COURS JAVASCRIPT

Mme: FATIMA ZAHRA HALOUI

Plan

01

01 - DÉFINIR LE RÔLE DE JAVASCRIPT DANS LE DÉVELOPPEMENT

- Comparer un langage de script avec un langage compilé
- Comprendre l’architecture client/serveur
- Découvrir l’écosystème de développement

02

02 - ACQUÉRIR LES FONDAMENTAUX DE JAVASCRIPT

- Maîtriser la syntaxe JavaScript et ses notions fondamentales
- Maîtriser les structures de contrôle
- Utiliser les fonctions & Manipuler les objets

03

03 - MANIPULER LES ÉLÉMENTS D’UNE PAGE AVEC DOM

- Comprendre l’arbre DOM, les nœuds parents et enfants
- Connaître les bases de la manipulation du DOM en JavaScript
- Manipuler les éléments HTML

04

04 - GÉRER LES ÉVÉNEMENTS UTILISATEUR

- Comprendre la notion d’événement pour gérer l’interactivité
- Gérer les éléments d’un formulaire
- Manipuler JQUERY/ AJAX

PARTIE 1

DÉFINIR LE RÔLE DE JAVASCRIPT DANS LE DÉVELOPPEMENT

- Comparer un langage de script avec un langage compilé
- Comprendre l’architecture client/serveur
- Découvrir l’écosystème de développement

JAVA SCRIPT: Définition

Le JavaScript est un langage de programmation créé en **1995**. Le JavaScript est aujourd'hui l'un des langages de programmation les plus populaires et il fait partie des **langages web dits « standards »** avec le **HTML et le CSS**. Son évolution est gérée par le groupe **ECMA International** qui se charge de publier les standards de ce langage.

On dit que le HTML, le CSS et le JavaScript sont des standards du web car les principaux navigateurs web (Google Chrome, Safari, Firefox, etc.) savent tous « lire » (ou « comprendre » ou « interpréter ») ces langages et les interprètent généralement de la même façon ce qui signifie qu'un même code va généralement produire le même résultat dans chaque navigateur.

Intérêts et usages du JavaScript :

- ❑ Le JavaScript est un langage dynamique ;
- ❑ Le JavaScript est un langage (principalement) côté client ;
- ❑ Le JavaScript est un langage interprété ;
- ❑ Le JavaScript est un langage orienté objet.

HTML

HTML + CSS

HTML + CSS + JavaScript

JavaScript vs Java : A ne pas confondre !!

Ces deux langages, bien que syntaxiquement assez proches à la base, reposent sur des concepts fondamentaux complètement différents et servent à effectuer des tâches totalement différentes.

Java est une technologie créée originellement par **Sun Microsystems** tandis que **JavaScript** est un langage créé par la société **Netscape**.

Avant sa sortie officielle, le nom original du JavaScript était **« LiveScript ».** Quelques jours avant la sortie du LiveScript, le langage est renommé JavaScript.

A l'époque, Sun et Netscape étaient partenaires et le Java était de plus en plus populaire. Il est donc communément admis que le nom « JavaScript » a été choisi pour des **raisons marketing** et pour créer une association dans la tête des gens avec le Java afin que les deux langages se servent mutuellement.

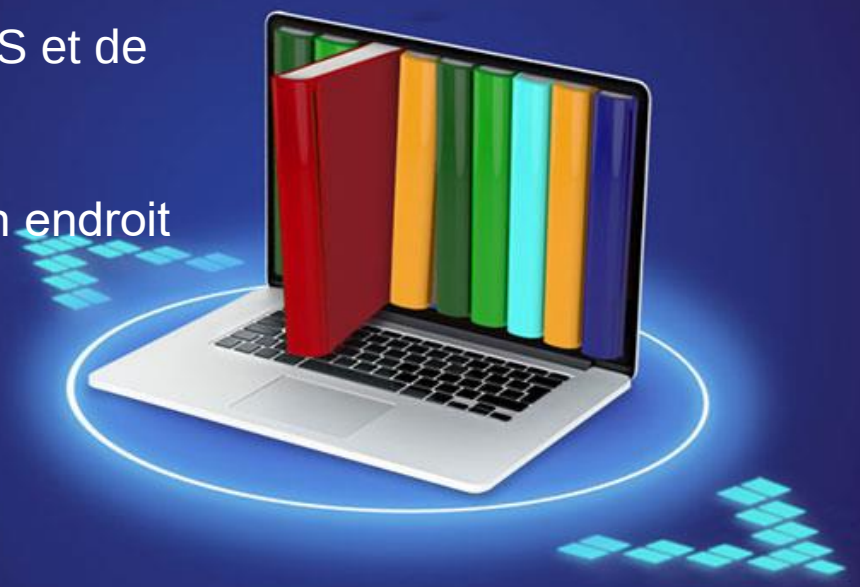
Le JavaScript, un langage dynamique:

Le JavaScript est un langage **dynamique**, c’est-à-dire un langage qui va nous permettre de générer du contenu dynamique pour nos pages web.

Un contenu « dynamique » est un contenu qui va se mettre à jour dynamiquement, c’est-à-dire changer sans qu’on ait besoin de modifier le code manuellement mais plutôt en fonction de différents facteurs externes

Ce langage permet de manipuler des contenus HTML ou des styles CSS et de les modifier en fonction de divers évènements ou variables.

Un évènement peut être par exemple un clic d’un utilisateur à un certain endroit de la page tandis qu’une variable peut être l’heure de la journée.



Le JavaScript, un langage côté client

Le Javascript est un langage dit client-side, c'est-à-dire que les scripts sont exécutés par le navigateur chez l'internaute (le client). Cela diffère des langages de scripts dits server-side qui sont exécutés par le serveur Web. C'est le cas des langages comme le PHP.

On peut classer les différents langages selon l'endroit où ils vont s'exécuter : **soit côté client**, **soit côté serveur.**

Pour comprendre ce que sont les langages **« côté client »** et **« côté serveur »**, il convient avant tout de comprendre ce qu'est un client et ce qu'est un serveur et pour cela il faut savoir ce qu'est un site.

Un site est un ensemble de ressources et de fichiers liés entre eux. Pour que notre site soit accessible sur le web pour tous, on va l'héberger sur un serveur, c'est-à-dire envoyer l'ensemble de nos fichiers sur le serveur et on va également acheter un nom de domaine qui va servir à identifier notre site.

Un **« serveur »** est une sorte de super ordinateurs, constamment accessibles et connectés aux autres serveurs (formant ainsi un réseau qu'on appelle le web) et qui va héberger les fichiers constituant un (ou plusieurs) site(s) web et le(s) « servir » sur demande du client.

Le JavaScript, un langage interprété

On peut séparer les langages selon qu'ils puissent être exécutés directement (**on parlera alors de langages interprétés**) ou qu'il faille les transformer en une autre forme pour pouvoir les exécuter (**on parlera alors le langages compilés**).

Le JavaScript est un **langage interprété**. Cela signifie qu'il va pouvoir être exécuté directement sous réserve qu'on possède le logiciel interpréteur. Tous les navigateurs connus possèdent leur interpréteur JavaScript.

Le JavaScript, un langage orienté objet

Langage de script vs langage compilé

Notion de compilateur / interpréteur:

- La compilation consiste à transformer le code écrit dans un langage de programmation de haut niveau (code lisible par l'homme) en code machine compréhensible par un processeur informatique (bits binaires 1 et 0).

Le compilateur s'assure également que le programme est correct du point de vue **TYPE** : on n'est pas autorisé à affecter une chaîne de caractères à une variable entière. Le compilateur veille également à ce que votre programme soit syntaxiquement correct. Par exemple, "x * y " est valide, mais "x @ y" ne l'est pas.

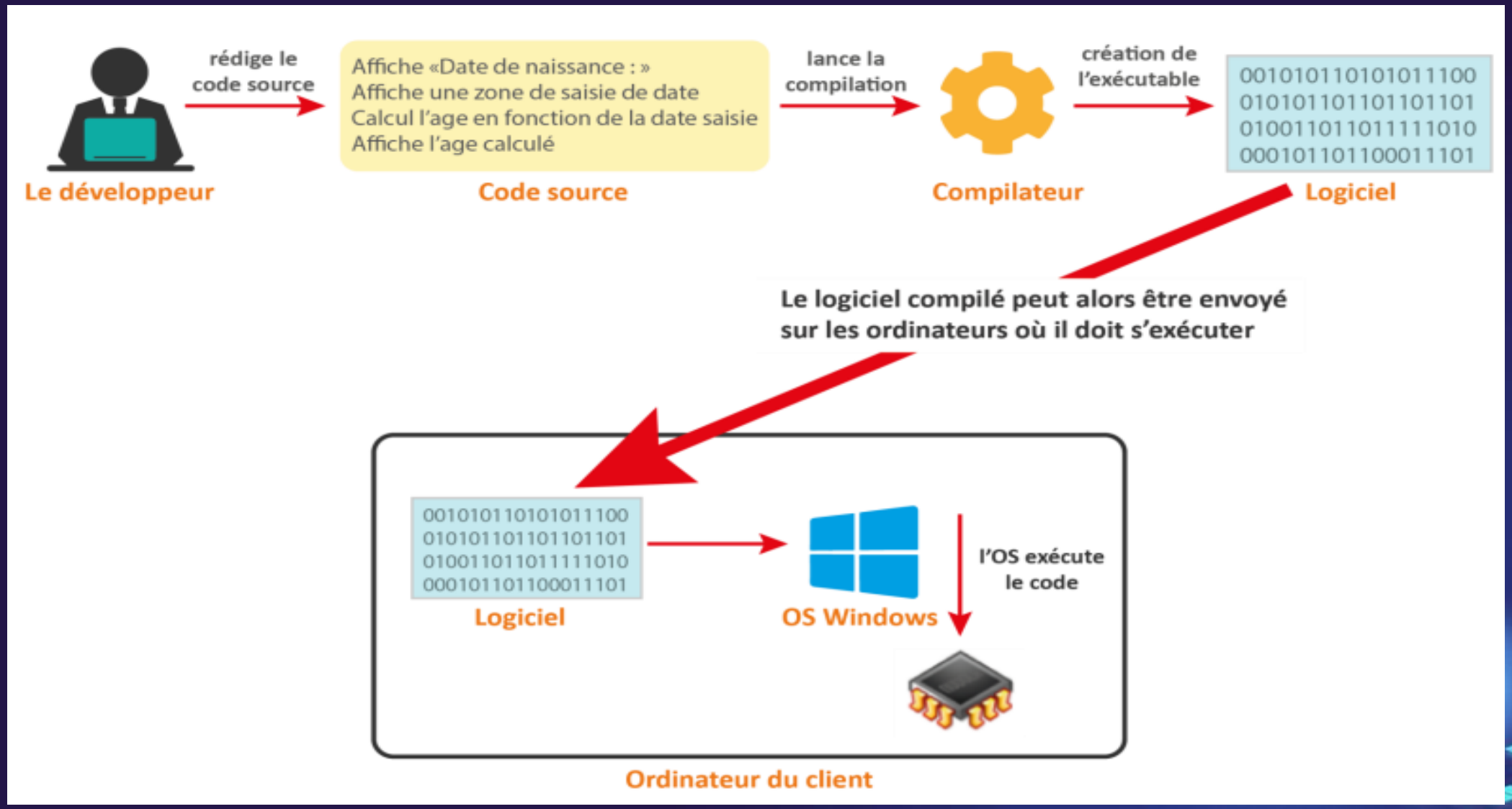
- Un interpréteur est un programme informatique qui convertit chaque déclaration de programme de haut niveau en code machine. Cela inclut le code source, le code précompilé et les scripts.

Différence : Un compilateur convertit le code en code machine avant l'exécution du programme. L'interpréteur convertit le code en code machine, ligne par ligne, au moment d'exécution du programme.

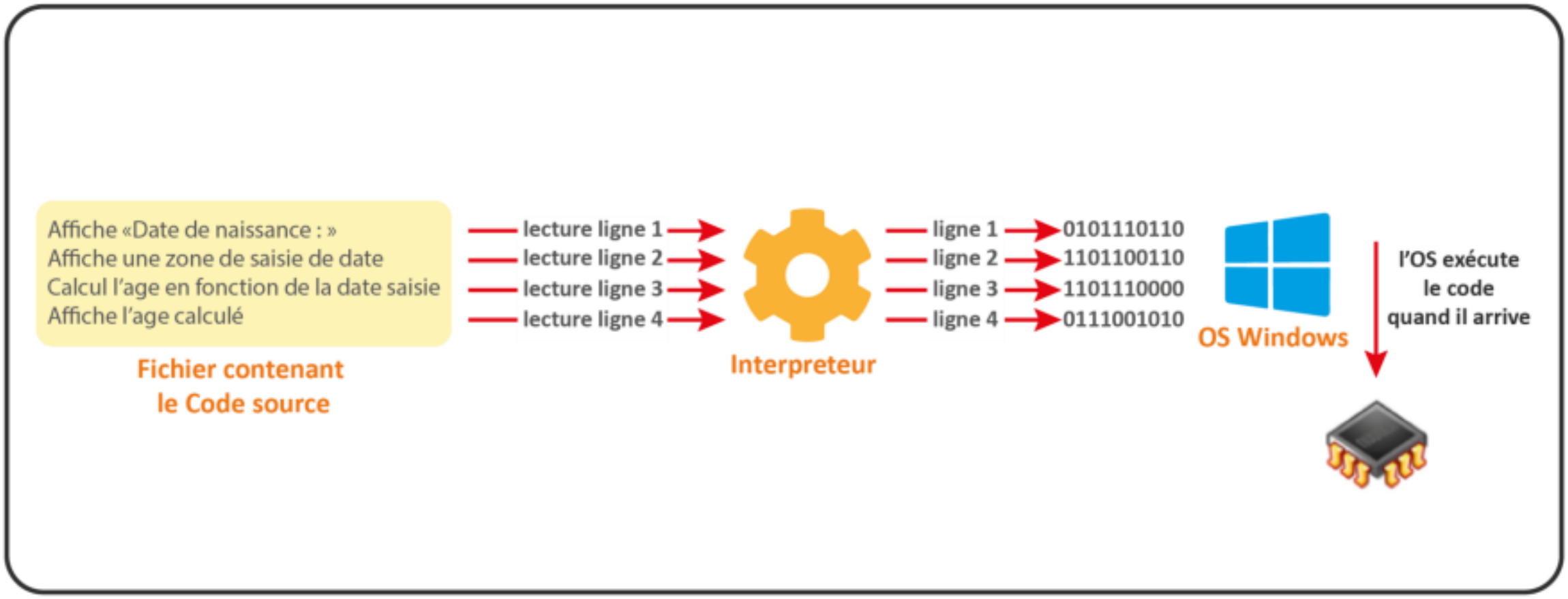
Exemples de langages compilés : **C, C++, C#, CLEO, COBOL, etc.**

Exemples de langages interprétés : **JavaScript, Perl, Python,**

Langage compilé



Langage interprété



Langage de script

Un langage de script (également appelé script) est une série de commandes qui peuvent être exécutées sans compilation.

Tous les langages de script sont des langages de programmation, mais tous les langages de programmation ne sont pas des langages de script.

Les langages de script utilisent un programme appelé interpréteur pour traduire les commandes.

Types d'un langage de script : coté client et coté serveur

- Les langages de script côté serveur s'exécutent sur un serveur Web. Lorsqu'un client envoie une requête, le serveur répond en envoyant du contenu via le protocole HTTP.

Exemples de langages de script côté serveur : PHP, ASP .NET, Node.Js, Java, Ruby, Perl.

- Les langages de script côté client s'exécutent du côté du client, sur son navigateur Web. L'avantage des scripts côté client est qu'ils peuvent réduire la demande sur le serveur, ce qui permet aux pages Web de se charger plus rapidement.

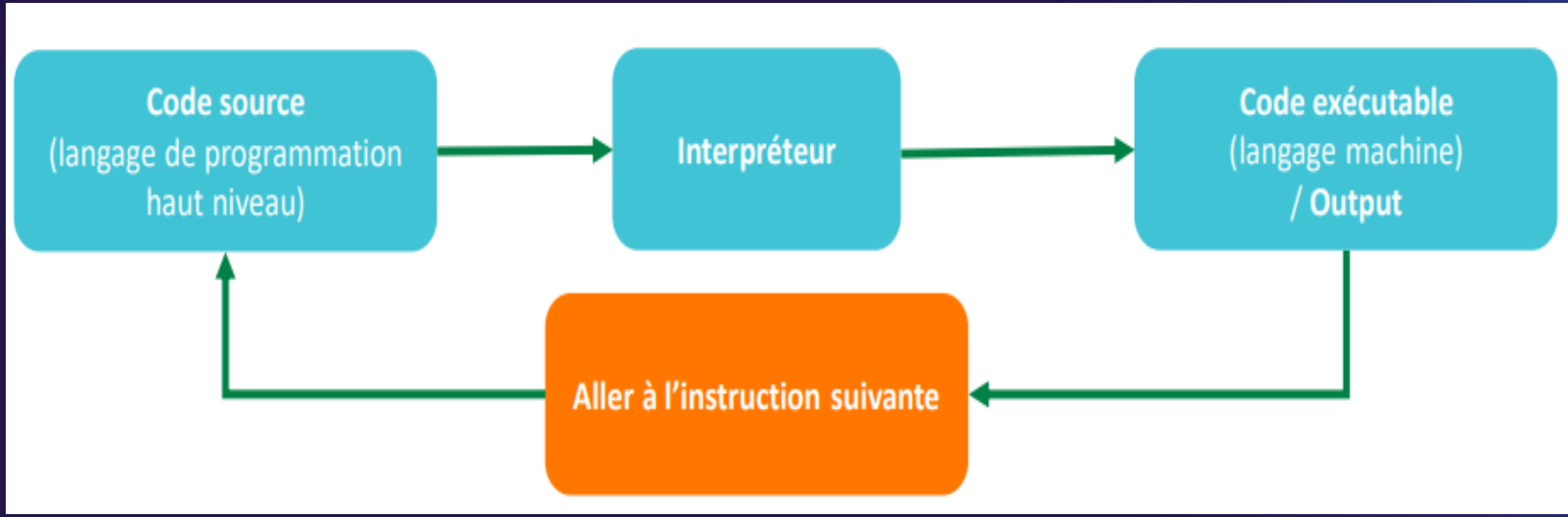
Exemples de langages de script côté client : HTML, CSS, JavaScript.

Fonctionnement d'un langage de script

▪ Le rôle de l'interpréteur

Le fonctionnement des langages de script est assuré par l'interpréteur. Son rôle réside dans la traduction des instructions du programme source en code machine:

- ✓ S'il y a une erreur dans l'instruction courante, l'interpréteur termine son processus de traduction à cette instruction et affiche un message d'erreur. L'interprète nepasse à la ligne d'exécution suivante qu'après avoir éliminé l'erreur.
- ✓ Un interpréteur exécute directement les instructions écrites dans un langage de script sans les convertir préalablement en code objet ou en code machine.



Résumé:

- **Langage compilé** : le code source est donné à un programme appelé compilateur qui va lire le code source et le convertir dans un langage que l'ordinateur sera capable d'interpréter : c'est le langage binaire, fait de 0 et de 1.

- **Langage précompilé** : le code source est compilé partiellement, généralement dans un code plus simple à lire par l'ordinateur, mais qui n'est pas encore du binaire. Ce code intermédiaire devra être lu par ce que l'on appelle une « **machine virtuelle** », qui exécutera ce code. Les langages comme le C# ou le Java sont dits précompilés.

- **Langage interprété** : dans ce cas, il n'y a pas de compilation. Le code source reste tel quel, et si on veut exécuter ce code, on doit le fournir à un interpréteur qui se chargera de le lire et de réaliser les actions demandées.

Remarque: Chaque navigateur possède un interpréteur Javascript, qui diffère selon le navigateur. Si vous utilisez Internet Explorer, son interpréteur Javascript s'appelle **JScript** (l'interpréteur de la version 9 s'appelle **Chakra**), celui de Mozilla Firefox se nomme **SpiderMonkey** et celui de Google Chrome est **V8**.

Fonctionnement d'un système client/serveur Web

Les ordinateurs clients fournissent une interface permettant à un utilisateur de demander des services auprès de serveur et d'afficher les résultats.
Cette interaction passe par les étapes suivantes :

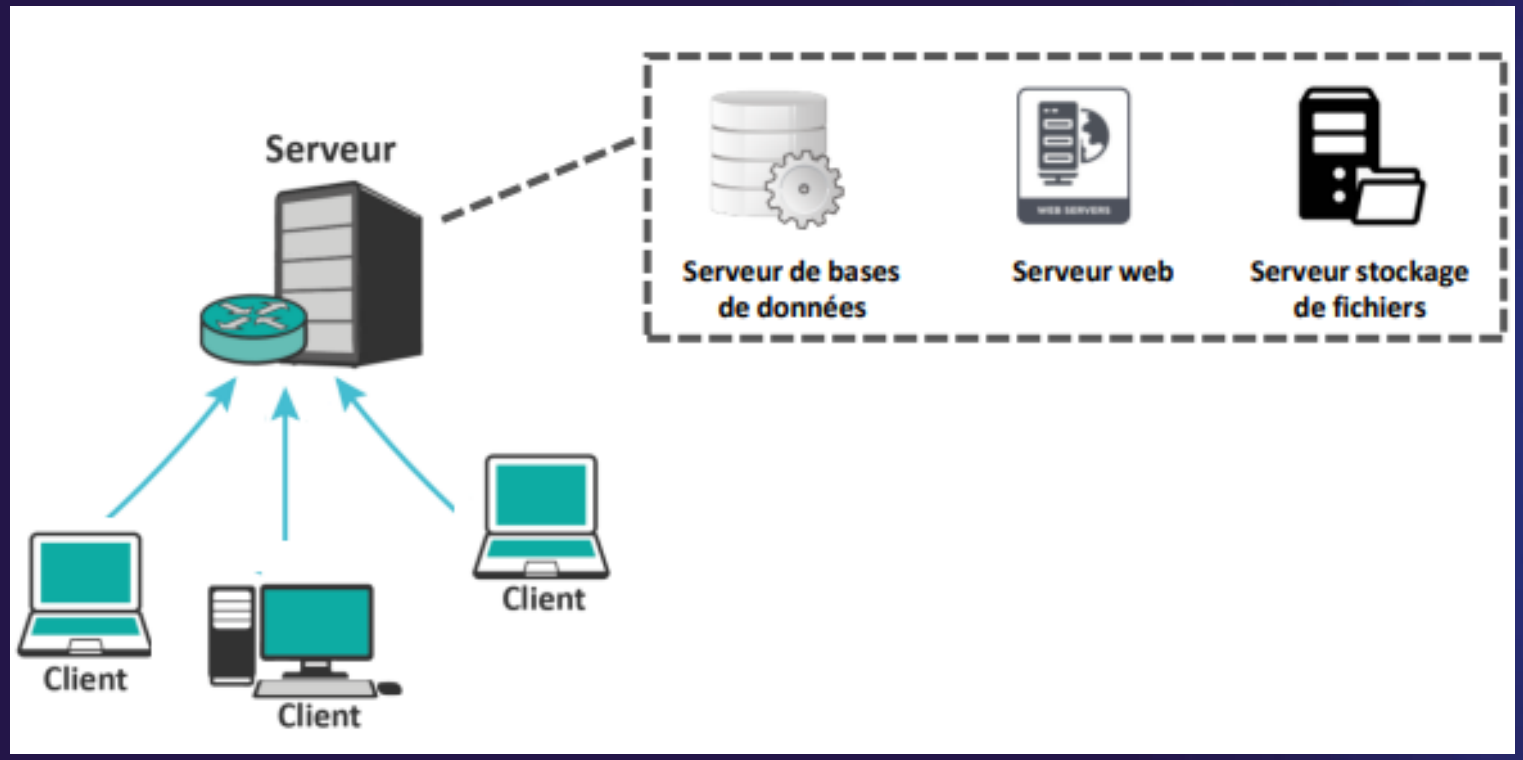
1. L'utilisateur saisit l'URL (Uniform Resource Locator) du site web ou du fichier. Le navigateur demande alors au serveur le DNS (DOMAIN NAME SYSTEM).
2. Le serveur DNS recherche l'adresse du serveur WEB.
3. Le serveur DNS répond avec l'adresse IP du serveur Web.
4. Le navigateur envoie une requête HTTP/HTTPS à l'adresse IP du serveur WEB (fournie par le serveur DNS).
5. Le serveur envoie les fichiers nécessaires du site web (pages html, documents, images, ...).
6. Le navigateur rend alors les fichiers et le site web s'affiche. Ce rendu est effectué à l'aide de l'interpréteur DOM (Document Object Model), de l'interpréteur CSS et du moteur JS, collectivement connus sous le nom de compilateurs JIT ou (Just in Time).

Comprendre l’architecture client/serveur

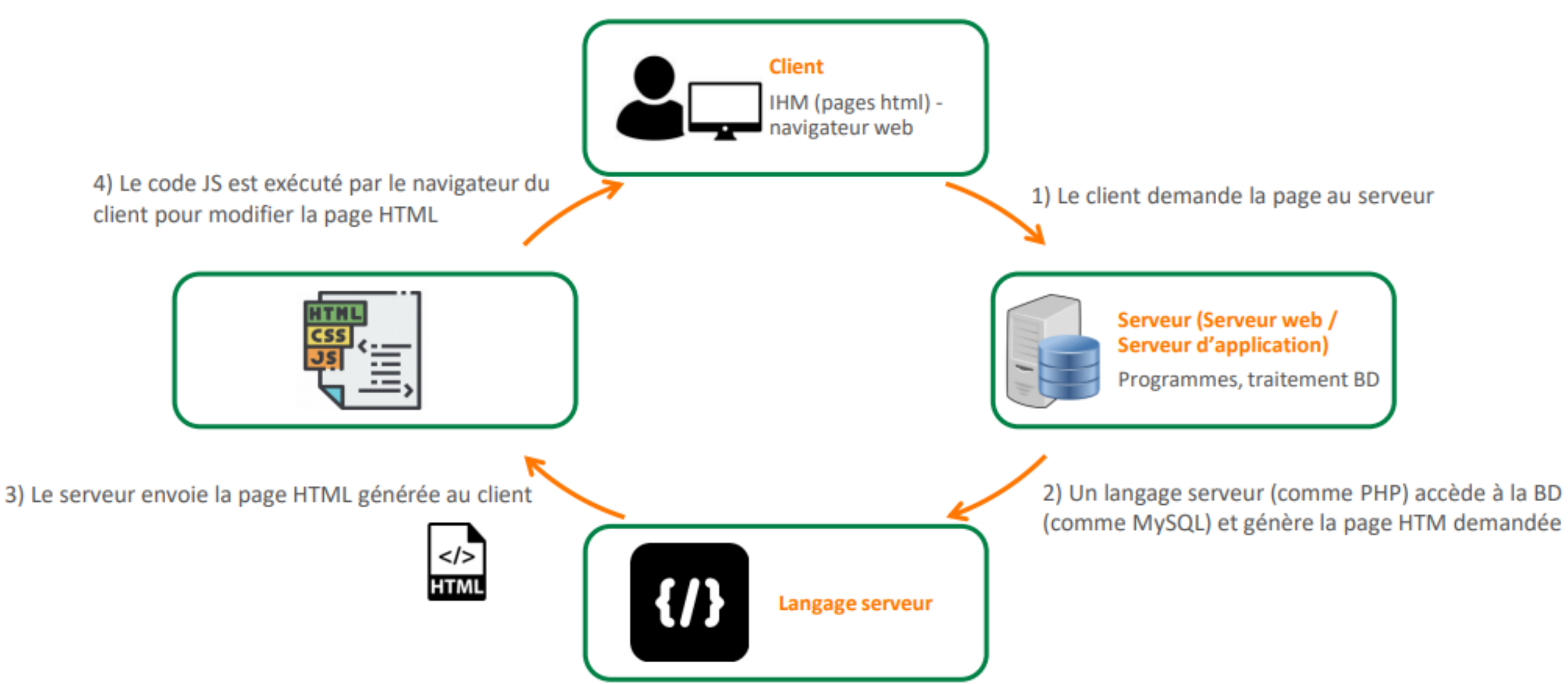
Composition d’une architecture client/serveur

L’architecture **client-serveur** correspond à l’architecture d'un réseau informatique dans lequel de nombreux clients demandent et reçoivent des services d'un serveur centralisé (Serveur).

Les clients sont souvent situés sur des postes de travail ou des ordinateurs personnels, tandis que les serveurs sont situés ailleurs sur le réseau, généralement sur des machines plus puissantes.



Le fonctionnement d'un système client/serveur peut être illustré par le schéma suivant :



Serveurs Web et HTTP

Les **navigateurs web (clients)** communiquent avec les serveurs web via le protocole **HTTP** (Hypertext Transfer Protocol). En tant que protocole de requête-réponse, ce protocole permet aux utilisateurs d'interagir avec des ressources Web telles que des fichiers HTML en transmettant des messages hypertextes entre les clients et les serveurs.

Les *clients HTTP* utilisent généralement des connexions **TCP (Transmission Control Protocol)** pour communiquer avec les serveurs. Une requête HTTP inclut :

- Une URL pointant sur la cible et la ressource (un fichier HTML, un document, …).
- Une méthode de requête spécifique afin d'effectuer diverses tâches (par exemple mise à jour des données, récupération d'un fichier, …).

Les différentes méthodes de requêtes et les actions associées sont présentées dans le tableau ci-dessous :

Méthode	Rôle
GET	Récupération d'une ressource spécifique (fichier html par exemple).
POST	Création d'une nouvelle ressource, enregistrement des données d'un formulaire d'inscription...
HEAD	Récupération des informations "metadata" d'une ressource spécifique sans le "body« .
PUT	Met à jour une ressource existante ou en crée une si elle n'existe pas.
DELETE	Suppression la ressource spécifiée.

Fonctionnement d'un système client/serveur Web

Serveurs Web et HTTP

La réponse HTTP (HTTP Response) est l'information fournie par le serveur suite à la demande du client. Elle sert à confirmer que l'action demandée a été exécutée avec succès. En cas d'erreur dans l'exécution de la demande du client, le serveur répond par un message d'erreur.

Les réponses HTTP se présentent sous la forme d'un texte brut formaté au format **JSON** ou **XML**, tout comme les demandes HTTP. Le corps d'une réponse aboutie à une requête GET contiendrait la ressource demandée.

Exemples de code d'état HTTP (HTTP status codes) :

- **"200 OK" :** succès
- **"404 Not Found" :** ressource introuvable
- **"403 Forbidden" :** accès non autorisé

Découvrir l’écosystème de développement

Environnements de développement

Choix de l'environnement de développement:

Un Environnement de Développement Intégré (**Integrated development environment IDE**) désigne un outil de développement dit tout en un qui permet aux programmeurs de consolider les différents aspects de l'écriture d'un programme informatique.
Les IDE assistent les programmeurs dans l’édition des logiciels en combinant les activités courantes de programmation en une seule application :

- ✓ **Édition du code source**
- ✓ **Mise en évidence de la syntaxe (colorisation)**
- ✓ **Saisie automatique (Auto-complétion)**
- ✓ **Création d'exécutables**
- ✓ **Débogage**

Rappel : Visual Studio Code

Le logiciel Visual Studio Code est un logiciel gratuit qui permet l'édition, la correction et le débogage du code source dans plusieurs langages informatiques : Visual Basic, JavaScript, XML, Python, HTML, CSS, ….

VS code offre :

- Une présentation sophistiquée du code.
- Une auto-complétion du code source.
- Un ensemble d'extensions permettant de simplifier la programmation.
- La détection du langage de programmation par l'extension du fichier.

Découverte des librairies appropriés

Front-end vs back-end

Front-End

- Le terme "**front-end**" désigne l'interface utilisateur.
- Le front-end est construit en utilisant une combinaison de technologies telles que le langage de balisage hypertexte (HTML), JavaScript et les feuilles de style en cascade (CSS).

Les frameworks front-end
1. Angular JS
2. React.js
3. JQuery
4. Vue.js

Back-End

- Le terme "**back-end**" désigne le serveur, l'application et la base de données qui travaillent en coulisses pour fournir des informations à l'utilisateur.
- La programmation back-end est définie comme la logique informatique d'un site Web ou d'un logiciel, depuis le stockage et l'organisation des données jusqu'à la création des algorithmes et des séquences logiques complexes qui fonctionnent, d'une manière transparente, sur le front-end.
- Les langages back-end les plus populaires pour sont Ruby, Python, Java, ASP .Net et PHP.

PARTIE 2

ACQUÉRIR LES FONDAMENTAUX DE JAVASCRIPT

- Maîtriser la syntaxe JavaScript et ses notions fondamentales
- Maîtriser les structures de contrôle
- Utiliser des fonctions & Manipuler les objets

Maîtriser la syntaxe JavaScript et ses notions fondamentales

Où écrire le code JavaScript ?

On va pouvoir placer du code JavaScript à trois endroits différents :

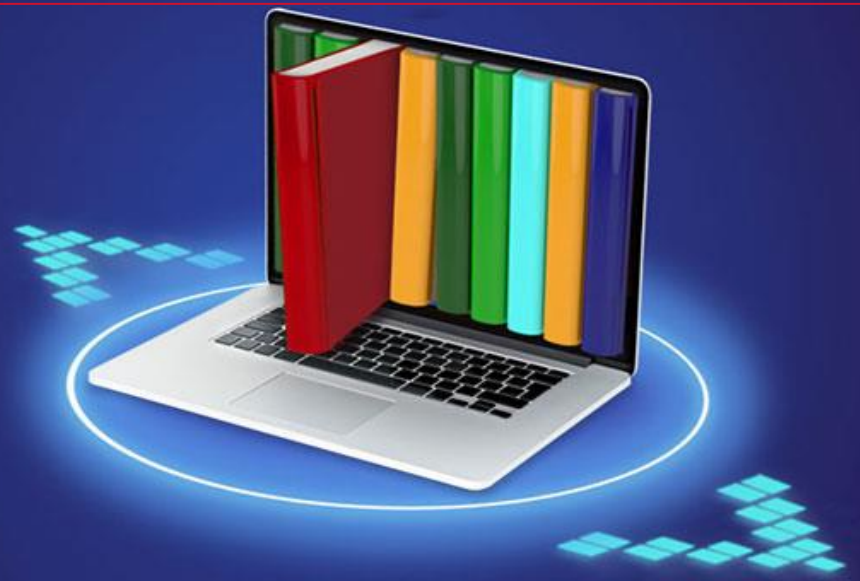
- Directement dans la balise ouvrante d'un élément HTML ;
- Dans un élément script, au sein d'une page HTML ;
- Dans un fichier séparé contenant exclusivement du JavaScript et portant l'extension **.js**.

1ier cas : Placer le code JavaScript dans la balise ouvrante d'un élément HTML

```
    <head>
        <title>Cours JavaScript</title>
        <meta charset="utf-8">
        <meta name="viewport"
          content="width=device-width, initial-scale=1, user-scalable=no">
        <link rel="stylesheet" href="cours.css">
    </head>

    <body>
        <h1>Titre principal</h1>
        <button onclick="alert('Bonjour !')">Cliquez moi</button>

        <button onclick="(function(){
            let para = document.createElement('p');
            para.textContent = 'Paragraphe ajouté';
            document.body.appendChild(para);
        })();">
            Ajouter un paragraphe
        </button>
    </body>
</html>
```

N.B: il est déconseillé et considéré comme une mauvaise pratique d'écrire du code JavaScript dans des balises ouvrantes d'éléments HTML.

Le web et les éléments le composant sont de plus en plus complexes et on doit donc être de plus en plus rigoureux pour exploiter cette complexité.



2 ième cas : Placer le code JavaScript dans un élément script, au sein d'une page HTML

On peut placer le code JavaScript dans un élément **script** qui est l'élément utilisé pour indiquer qu'on code en JavaScript.

On placer un **script** n'importe où dans une page HTML, aussi bien dans l'élément **head** qu'au sein de l'élément **body**.

On peut indiquer plusieurs éléments **script** dans une page HTML pour placer plusieurs bouts de code JavaScript à différents endroits de la page. Cette méthode est meilleure que la précédente mais pas l'idéale.

La séparation des codes n'est pas optimale ici puisqu'on mélange du JavaScript et du HTML ce qui peut rendre l'ensemble confus et complexe à comprendre dans le cadre d'un gros projet.

```html
        <meta charset="utf-8">
        <meta name="viewport"
          content="width=device-width, initial-scale=1, user-scalable=no">
        <link rel="stylesheet" href="cours.css">
        <script>
            document.addEventListener('DOMContentLoaded', function(){
                let bonjour = document.getElementById('b1');
                bonjour.addEventListener('click', alerte);

                function alerte(){
                    alert('Bonjour');
                }
            });
        </script>
    </head>

    <body>
        <h1>Titre principal</h1>
        <button id='b1'>Cliquez moi</button>
        <button id='b2'>Ajouter un paragraphe</button>
        <script>
            let ajouter = document.getElementById('b2');
            ajouter.addEventListener('click', ajout);
            function ajout(){
                let para = document.createElement('p');
                para.textContent = 'Paragraphe ajouté';
                document.body.appendChild(para);
            }
        </script>
    </body>
</html>
```

Exemple: Affichage d’une boite de dialogue

```
<!DOCTYPE html>
<html>
 <head>
   <title>Hello World!</title>
 </head>
 <body>
   <script>
     alert('Hello world!');
   </script>
 </body>
</html>
```

La boîte de dialogue alert():

alert() est une instruction simple, appelée fonction, qui permet d'afficher une boîte de dialogue contenant un message. Ce message est placé entre apostrophes, elles-mêmes placées entre les parenthèses de la fonction alert().

3 ième cas: Placer le code JavaScript dans un fichier séparé

Placer le code JavaScript dans un fichier séparé ne contenant que du code JavaScript est la méthode la plus recommandée

On crée un nouveau fichier et on l'enregistrer avec une extension **.js.** Ensuite, on fait appel à notre fichier JavaScript depuis notre fichier HTML.

cours.html × cours.js × cours.css ×

... Cours > JavaScript > Supports JavaScript > cours.js > Ln: 23 Col: 1 UTF-8

```javascript
let bonjour = document.getElementById('b1');
let ajouter = document.getElementById('b2');

bonjour.addEventListener('click', alerte);
ajouter.addEventListener('click', ajout);

function alerte(){
    alert('Bonjour');
}
function ajout(){
    let para = document.createElement('p');
    para.textContent = 'Paragraphe ajouté';
    document.body.appendChild(para);
}
```

N.B: Pseudo-URL dans Javascript !!!!

Certains attributs de balises HTML attendent comme valeur une URL, par exemple l'attribut href d'une balise <a> ou action pour une balise <form>.

Il peut aussi arriver que l'on souhaite contourner les actions par défaut de ces attributs, par exemple en remplaçant un lien par une fonction JavaScript. Une « ancienne » façon de coder consiste à remplacer l'URL attendue par une pseudo-url :

```
<a href="javascript:ma_fonction()">Exécuter ma fonction</a>
```

Cette notation permet d'indiquer au navigateur qu'il faut considérer le contenu de l'attribut comme du code JavaScript et l'exécuter.

Positionner l'élément <script>

Une page Web est lue par le navigateur de façon linéaire, c'est-à-dire qu'il lit d'abord le <head>, puis les éléments de <body> les uns à la suite des autres. Si vous appelez un fichier Javascript dès le début du chargement de la page, le navigateur va donc charger ce fichier, et si ce dernier est volumineux, le chargement de la page s'en trouvera ralenti. C'est normal puisque le navigateur va charger le fichier avant de commencer à afficher le contenu de la page.

Pour pallier ce problème, il est conseillé de placer les éléments <script> juste avant la fermeture de l'élément <body>

Remarque:

Il est à noter que certains navigateurs modernes chargent automatiquement les fichiers Javascript en dernier, mais ce n'est pas toujours le cas. C'est pour cela qu'il vaut mieux s'en tenir à cette méthode.

La compression des scripts

Certains scripts sont disponibles sous une forme dite compressée, c'est-à-dire que tout le code est écrit à la suite, sans retours à la ligne. Cela permet d'alléger considérablement le poids d'un script et ainsi de faire en sorte que la page soit chargée plus rapidement.

Des programmes existent pour « compresser » un code Javascript. Mais si vous avez oublié un seul point-virgule, votre code compressé ne fonctionnera plus, puisque les instructions ne seront pas correctement séparées. C'est aussi une des raisons qui fait qu'il faut toujours mettre les points-virgules en fin d'instruction.

Les espaces

Le JavaScript n'est pas sensible aux espaces. Cela veut dire que vous pouvez aligner des instructions comme vous le voulez, sans que cela ne gêne en rien l'exécution du script. Par exemple, ceci est correct :

Code : JavaScript

```
1 instruction_1;
2   instruction_1_1;
3   instruction_1_2;
4 instruction_2;    instruction_3;
```

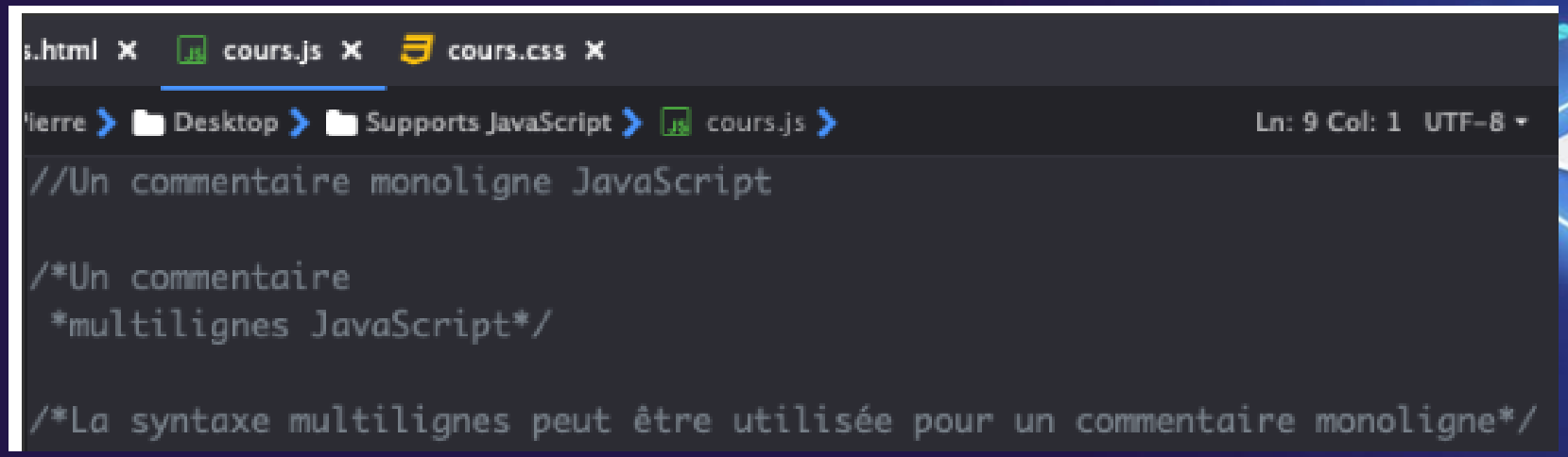
Commentaires, indentation et syntaxe de base en JavaScript

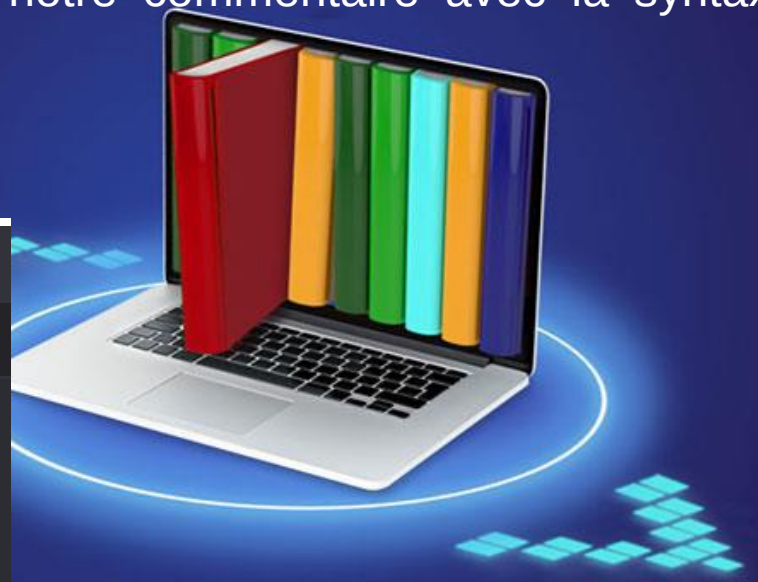
Les commentaires en JavaScript

Les commentaires sont des lignes de texte (des indications) placées au milieu d'un script et servant à documenter le code, c'est-à-dire à expliquer ce que fait tel ou tel bout de script et éventuellement comment le manipuler.

En JavaScript, il existe deux types : les commentaires **mono-ligne** et les commentaires **multi-lignes**.

- Pour écrire un commentaire **multi-lignes**, il faudra entourer le texte de notre commentaire avec la syntaxe suivante ***/* */.***
- Pour écrire un commentaire **mono-ligne**, on utilisera un double slash ***//***





L’indentation en JavaScript

L’indentation correspond au fait de décaler certaines lignes de code par rapport à d’autres. Cela est généralement utilisé pour rendre son code plus lisible et donc plus simple à comprendre.

Une bonne pratique est d’effectuer un retrait vers la droite équivalent à une tabulation à chaque fois qu’on écrit une nouvelle ligne de code à l’intérieur d’une instruction JavaScript.

```
function alerte(){
    alert('Bonjour');
}
function ajout(){
    let para = document.createElement('p');
    para.textContent = 'Paragraphe ajouté';
    document.body.appendChild(para);
}
```

Présentation des variables JavaScript

Qu’est-ce qu’une variable ?

Une variable est un conteneur servant à stocker des informations de manière temporaire, comme une chaine de caractères (un texte) ou un nombre par exemple.

Le propre d’une variable est de pouvoir varier, c’est-à-dire de pouvoir stocker différentes valeurs au fil du temps et c’est cette particularité qui les rend si utiles.

N.B:

- Une variable en soi et la valeur qu’elle va stocker sont deux éléments différents et qui ne sont pas égaux.
- Une variable n’est qu’un conteneur. On peut imaginer une variable comme une boite dans laquelle on va pouvoir placer différentes choses au cours du temps.

Les règles de déclaration des variables en JavaScript

Pour pouvoir utiliser les variables et illustrer leur intérêt, il va falloir les créer. Lorsqu'on crée une variable en JavaScript, on dit également qu'on **« déclare »** une variable

Pour déclarer une variable en JavaScript, nous allons devoir utiliser le mot clef **var** ou le mot clef **let** suivi du nom qu'on souhaite donner à cettevariable.

Concernant le nom des variables, on doit respecter les règles suivantes :

- ✓ Le nom d'une variable doit obligatoirement commencer par une **lettre** ou un **underscore (_)** et ne doit pas commencer par un chiffre ;
- ✓ Le nom d'une variable ne doit contenir que des lettres, des chiffres et des underscores mais pas de caractères spéciaux ;
- ✓ Le nom d'une variable ne doit pas contenir d'espace.

N.B: Le nom des variables est sensible à la casse en JavaScript. Cela signifie que l'usage de majuscules ou de minuscules avec un même nom va permettre de définir des variables différentes.

Par exemple, les noms **texte, TEXTE et tEXTe** vont pouvoir définir des variables différentes.

Remarque:

On utilise généralement la convention **lower camel case** pour définir les noms de variable en JavaScript. Cette convention stipule simplement que lorsqu'un nom de variable est composé de plusieurs mots, on colle les mots ensemble en utilisant une majuscule pour chaque mot sauf le premier.

Par exemple, si on décide de nommer une variable **« monage »** on écrit en JavaScript **let monAge** ou **var monAge**

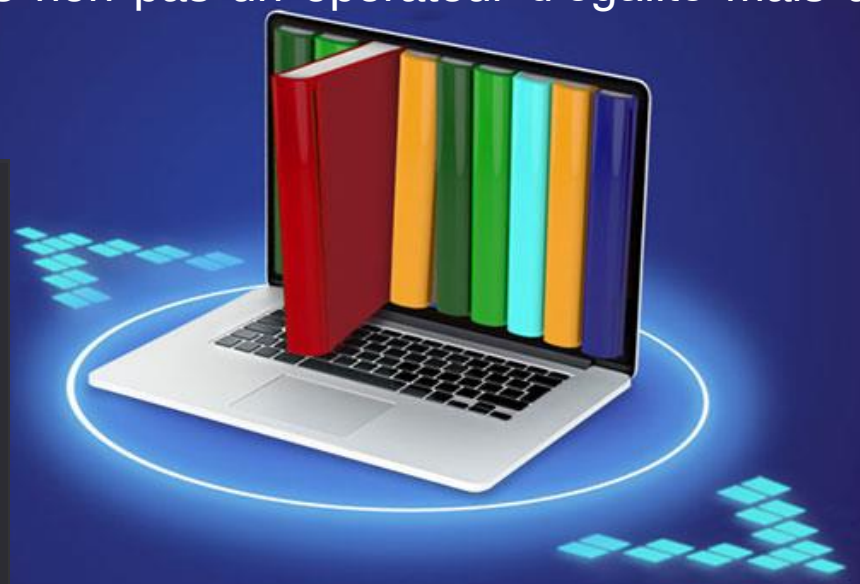
Initialiser une variable

Lorsqu'on assigne une valeur pour la première fois à une variable, c'est-à-dire lorsqu'on stocke une valeur dans une variable qui n'en stockait pas encore, on dit également qu'on initialise une variable.

On va pouvoir **initialiser une variable après l'avoir déclarée ou au moment de sa déclaration**. Les deux façons de faire sont équivalentes en termes de résultat mais il est plus rapide (en termes d'écriture de code) d'initialiser une variable lors de sa déclaration puisque cela va éviter d'avoir à réécrire le nom de la variable.

Pour initialiser une variable, on utilise **l'opérateur =** qui est dans ce cas non pas un opérateur d'égalité mais un opérateur d'assignation ou d'affectation comme ceci :

```
//On déclare et on initialise la variable en même temps
let prenom = 'Pierre';

//On déclare la variable puis on l'initialise ensuite
let monAge;
monAge = 29;
```



Modifier la valeur stockée dans une variable

L’intérêt principal des variables est de pouvoir stocker différentes valeurs.

Pour affecter une nouvelle valeur dans une variable déjà initialisée, on va se contenter d’utiliser à nouveau **l’opérateur d’affectation =**

En faisant cela, la nouvelle valeur va venir écraser l’ancienne valeur stockée qui sera alors supprimée.

```
//On déclare et on initialise la variable en même temps
let prenom = 'Pierre';

//On déclare la variable puis on l'initialise ensuite
let monAge;
monAge = 29;

/*On modifie la valeur stockée dans preonm.
 *Notre variable stocke désormais la valeur "Mathilde"*/
prenom = 'Mathilde';
```

```
var myVariable = 5.5; //  les nombres à virgule s'écrivent avec un point
```

```
var myVariable1, myVariable2 = 4, myVariable3;
```

nous avons déclaré trois variables en une ligne mais seulement la deuxième s'est vu attribuer une valeur.

Lorsqu’on utilise une seule fois l'instruction var pour déclarer plusieurs variables, on doit placer une virgule après chaque variable et on ne doit utiliser le point-virgule (qui termine une instruction) qu'à la fin de la déclaration de toutes les variables.

```
var myVariable1, myVariable2;
myVariable1 = myVariable2 = 2;
```

Les deux variables contiennent maintenant le même nombre : 2

Types de données JavaScript

JavaScript est un langage faiblement typé : le type d'une variable est défini au moment de l'exécution.

On peut déclarer une variable JS en utilisant les mots-clés suivants :

- ✓ **var** : utilisé pour déclarer une variable globale;
- ✓ **let** : utilisé pour déclarer une variable dont la portée est limitée à un bloc;
- ✓ **const** : permet de déclarer une variable qui doit avoir une valeur initiale et ne peut pas être réassignée.

Remarque:

La déclaration de variables se fait soit:

- Soit de **façon explicite**, en faisant précéder la variable du mot clé var qui permet d'indiquer de façon rigoureuse qu'il s'agit d'une variable : var chaine= "bonjour"
- Soit de **façon implicite**, en laissant le navigateur déterminer qu'il s'agit d'une déclaration de variable.

La différence entre les mots clefs let et var

Lorsqu'on utilise la syntaxe **avec var**, on n'est pas obligé de déclarer la variable avant de la manipuler dans le code, on peut très bien effectuer des manipulations en haut du code et la déclarer en fin de code.

Cela est possible car le JavaScript va traiter les déclarations de variables effectuées avec **var** avant le reste du code JavaScript. Ce comportement est appelé remontée.

Ce comportement a été jugé comme inadapté dans les versions récentes de JavaScript et a donc été corrigé dans la déclaration de variables **avec let** : les variables utilisant la syntaxe **let** doivent obligatoirement être déclarées avant de pouvoir être utilisées.

Le but de ce comportement est de pousser les développeurs à créer des scripts plus compréhensibles et plus clairs en apportant de la structure au code avec notamment la déclaration des différentes variables au début de chaque script.

Types de données JavaScript

Contrairement à de nombreux langages, le Javascript est un langage typé dynamiquement. Cela veut dire, généralement, que toute déclaration de variable se fait avec le mot-clé **var** sans distinction du contenu, tandis que dans d'autres langages, comme le C, il est nécessaire de préciser quel type de contenu la variable va devoir contenir.

En Javascript, les variables sont typées dynamiquement, ce qui veut dire que l'on peut y mettre du texte en premier lieu puis l'effacer et y mettre un nombre quel qu'il soit, et ce, sans contraintes.

Les trois types principaux en Javascript :

Le type numérique (alias number) : il représente tout nombre, que ce soit un entier, un négatif, un nombre scientifique. Pour assigner un type numérique à une variable, il suffit d'écrire : **var number = 5**;

Tout comme de nombreux langages, le Javascript reconnaît plusieurs écritures pour les nombres, comme l'écriture décimale var number = 5.5; ou l'écriture scientifique var number = 3.65e+5; ou encore l'écriture hexadécimale var number = 0x391;

Les chaînes de caractères (alias string) : ce type représente n'importe quel texte. On peut l'assigner de deux façons différentes :

```
var text1 = "Mon premier texte"; // Avec des guillemets
var text2 = 'Mon deuxième texte'; // Avec des apostrophes
```

Types de données JavaScript

Remarque:

- Il est important de préciser que si on écrit var **myVariable = '2';** alors le type de cette variable est une chaîne de caractères et non pas un type numérique.

- Si on utilise les apostrophes pour « encadrer » un texte et qu'on souhaite utiliser des apostrophes dans ce même texte, il faudra alors « échapper » les apostrophes comme suit : **var text = 'Ça c\'est un test !';**

Les booléens (alias boolean) : un booléen est un type à deux états qui sont les suivants : vrai ou faux. Ces deux états s'écrivent de la façon suivante :

```
var isTrue = true;
var isFalse = false;
```

Tester l'existence de variables avec typeof

- Pour vérifier le type d’une variable, on utilise l'instruction **typeof**:

```
var number = 2;
alert(typeof number); // Affiche : « number »
var text = 'Mon texte';
alert(typeof text); // Affiche : « string »
var aBoolean = false;
alert(typeof aBoolean); // Affiche : « boolean »
```

- Pour tester l'existence d'une variable :

```
alert(typeof nothing); // Affiche : « undefined »
```

Si l'instruction typeof vous renvoie undefined, c'est soit que votre variable est inexistante, soit qu'elle est déclarée mais ne contient rien.

Résumé :

En JavaScript, on distingue deux familles de types de variables :

Types primitifs	Les types structurels
• **String :** chaines de caractères • **Number :** nombres entiers et réels • **Boolean :** les booléens (rue, false) • **Undefined :** valeur prise par les variables déclarées non initialisées	• **Date** : pour représenter les dates • **Array** : pour représenter les tableaux • **Object** : tel que les enregistrements • **Function**

Différence entre non défini et nul:

Undefined et null sont de valeur égale mais de type différent :

```
typeof undefined          // undefined
typeof null               // object
null === undefined        // false
null == undefined         // true
```

Types de données JavaScript (SUITE)

- ✓ **String** ou « chaine de caractères » ;
- ✓ **Number** ou « nombre » ;
- ✓ **Boolean** ou « booléen » ;
- ✓ **Null** ou « nul / vide »;
- ✓ **Undefined** ou « indéfini » ;
- ✓ **Symbol** ou « symbole »;
- ✓ **Object** ou « objet » ;

Portée des variables (variable scope)

La portée d'une variable détermine son accessibilité (visibilité). En JS, trois types de portées sont distinguées

Portée du bloc : (Block scope)

- En utilisant le mot clé **let**, les variables déclarées à l'intérieur d'un bloc { } ne sont pas accessibles depuis l'extérieur du bloc :

```
{        let x = 2;        }
 // x n'est pas accessible ici
```

- En utilisant le mot clé **var**, les variables déclarées à l'intérieur d'un bloc { } sont accessibles depuis l'extérieur du bloc.

```
{        var x = 2;        }
 // x est accessible ici
```

Portée locale : (Function scope)

- Les variables déclarées dans une fonction JavaScript deviennent LOCALES à la fonction : Ils ne sont accessibles qu'à partir de la fonction.

```
function Test()
{
        var x = "test1";
        let y = "test2";
        const z = "test3";
}
//x, y et z ne sont pas
accessibles en dehors de la
fonction
```

Portée globale : (Global scope)

- Une variable déclarée en dehors d'une fonction, devient GLOBAL. Les variables globales sont accessibles de n'importe où dans un programme JavaScript..

Expressions et opérateurs:

Opérateurs arithmétiques JavaScript:

Les opérateurs arithmétiques sont utilisés pour effectuer des opérations arithmétiques sur des nombres :

Opérateur	Signification
+	Addition
-	Soustraction
*	Multiplication
**	Puissance
/	Division
%	Modulo (Reste de la division)
++	Incrémentation
--	Décrémentation

Opérateurs d'affectation JavaScript:

Les opérateurs d'affectation permettent d'assigner des valeurs aux variables JavaScript. Le tableau suivant regroupe ces opérateurs

Opérateur	Exemple d'utilisation	Signification	Exemple complet	Résultat
=	x = y	x prend la valeur de y	let x = 5	x vaut 5
+=	x += y	x = x + y	`let x = 10;` `x += 5;`	x vaut 15
-=	x -= y	x = x – y	`let x = 10;` `x -= 5;`	x vaut 5
*=	x *= y	x = x * y	`let x = 10;` `x *= 5;`	x vaut 50
/=	x /= y	x = x / y	`let x = 10;` `x /= 5;`	x vaut 2
%=	x %= y	x = x % y	`let x = 10;` `x %= 5;`	x vaut 0
**=	x **= y	x = x ** y ⇔ x = x à la puissance y	`let x = 3;` `x %= 2;`	x vaut 9

Concaténation des chaînes de caractères en JavaScript

L' opérateur + , appliqué aux chaînes de caractères, permet de concaténer des chaînes.

Exemple 1 :

```
let texte1 = "OFPPT";
texte1 += " ";
let texte2 = "en force";
let texte3 = texte1 + texte2;

//Output : texte3 = "OFPPT en force"
```

L'application de l'opérateur + pour concaténer les chaines de caractères et les nombres :

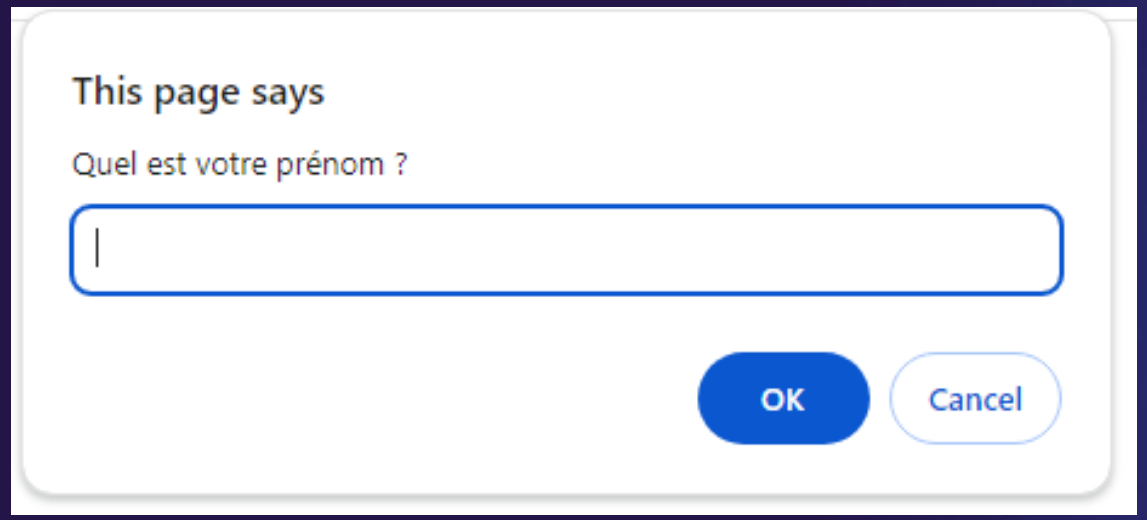
Exemple 2 :

```
var text = 'Voici un nombre : ', number = 42, result;
result = text + number;

alert(result); // Affiche : « Voici un nombre : 42 »
```

Exemple de concaténation pour interagir avec l'utilisateur:

```
var start = 'Bonjour ', name, end = ' !', result;
name   = prompt('Quel est votre prénom ?');
result = start + name + end;
alert(result);
```

La fonction **prompt()** s'utilise comme **alert()** mais a une petite particularité. Elle renvoie ce que l'utilisateur a écrit sous forme d'une chaîne de caractères:



Convertir une chaîne de caractères en nombre

On peut convertir une chaîne de caractères en nombre. Pour cela, on a besoin de la fonction **parseInt()**

```
var first, second, result;

first = prompt('Entrez le premier chiffre :');
second = prompt('Entrez le second chiffre :');
result = parseInt(first) + parseInt(second);

alert(result);
```

Convertir un nombre en chaîne de caractères

```
var text, number1 = 4, number2 = 2;
text = number1 + '' + number2;
alert(text); // Affiche : « 42 »
```

On ajoute une chaîne de caractères vide entre les deux nombres, ce qui aura eu pour effet de les convertir en chaînes de caractères.

Opérateurs de comparaison en JavaScript

Les opérateurs de comparaison permettent de comparer des opérandes (qui peuvent être des valeurs numériques ou des chaînes de caractères) et renvoie une valeur logique : true (vrai) si la comparaison est vraie, false (faux) sinon.

Opérateur	Signification	Exemple
==	Egal à (comparaison des valeurs)	let x = 5; let y = "5"; let z=(x==y); //z=true
===	Egal à (comparaison de la valeur et du type)	let x = 5; let y = "5"; let z=(x===y); //z=false
!=	Différent de (n'est pas égal à)	let x = 5; let y = 5; let z=(x!=y); //z=false
!==	Type ou valeur différente	let x = 5; let y = "5"; let z=(x!==y); //z=true
>	Supérieur à	let x = 5; let y = 5; let z=(x>y); //z=false
<	Inférieur à	let x = 5; let y = 5; let z=(x<y); //z=false
>=	Supérieur ou égal à	let x = 5; let y = 5; let z=(x>=y); //z=true
<=	Inférieur ou égal à	let x = 5; let y = 5; let z=(x<=y); //z=true

Opérateurs logiques en JavaScript

Les opérateurs logiques, aussi appelés opérateurs booléens, sont utilisés pour combiner des valeurs booléennes (des conditions qui peuvent avoir les valeurs true ou false) et produire une nouvelle valeur booléenne.

Opérateur	Signification
&&	ET logique
||	OU logique
!	NON logique

Les conditions en JavaScript

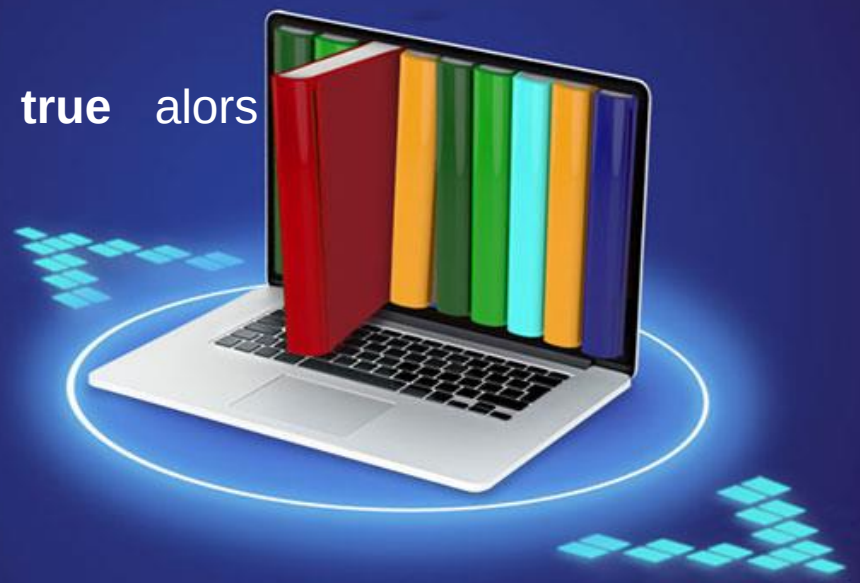
- **La condition « if else »:**

La structure « if»:

```
if (true) {
   alert("Ce message s'est bien affiché.");
}
if (false) {
   alert(« Ce message ne s'affichera pas.");
}
```

Le code d'une condition est exécuté si le booléen reçu est **true** alors que **false** empêche l'exécution du code.

```
if (2 < 8 && 8 >= 4) { // Cette condition renvoie « true », le code est donc exécuté
   alert('La condition est bien vérifiée.');
}

if (2 > 8 || 8 <= 4) { // Cette condition renvoie « false », le code n'est donc pas exécuté
   alert("La condition n'est pas vérifiée .");
}
```



La fonction confirm()

On lui passe en paramètre une chaîne de caractères qui sera affichée à l'écran et elle retourne un booléen en fonction de l'action de l'utilisateur :

```
if (confirm('Voulez-vous exécuter le code Javascript de cette page ?')) {
   alert('Le code a bien été exécuté !');
}
```

Le code s'exécute lorsqu'on clique sur le bouton OK et ne s'exécute pas lorsqu'on clique sur Annuler

Exemple:

```
<script>
if (confirm('Voulez-vous exécuter le code Javascript de cette
page ?')) {
alert('Le code a bien été exécuté !');}

var text, number1 = 4, number2 = 2;
text = number1  + '' + number2;
alert(text); // Affiche : « 42 »
</script>
```

La structure « else»:

Pour exécuter un code suite à la vérification d'une condition et exécuter un autre code si elle n'est pas vérifiée.

```
if (confirm('Pour accéder à ce site vous devez avoir 18 ans ou plus, cliquez sur "OK" si c\'est le cas.')) {

   alert('Vous allez être redirigé vers le site.');
}

else {
   alert("Désolé, vous n'avez pas accès à ce site.");
}
```

La structure else if

- Une première condition est à tester ;
- Une deuxième condition est présente et sera testée si la première échoue ;
- Et si aucune condition ne se vérifie, la structure else fait alors son travail

Exemple:

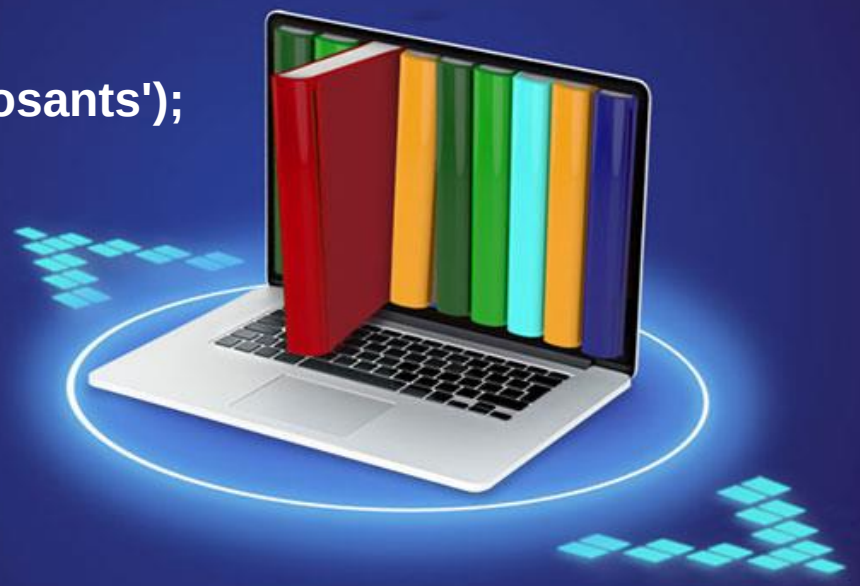
```
var floor = prompt("Entrez l'étage où l'ascenseur doit se rendre (de -2 à 30) :");

if (floor == 0) {
   alert('Vous vous trouvez déjà au rez-de-chaussée.');
} else if (-2 <= floor && floor <= 30) {
   alert("Direction l'étage n°" + floor + ' !');
} else {
   alert("L'étage spécifié n'existe pas.");
}
```

La condition « switch »

Le fonctionnement de la condition **if else** qui est très utile dans de nombreux cas, toutefois elle n'est pas très pratique pour faire du cas par cas ; c'est là qu'intervient la condition switch

Exemple : nous avons un meuble avec quatre tiroirs contenant chacun des objets différents, et il faut que l'utilisateur puisse connaître le contenu du tiroir dont il entre le chiffre

```
var drawer = parseInt(prompt('Choisissez le tiroir à ouvrir (1 à 4) :'));
switch (drawer) {
   case 1:
      alert('Contient du papier, des crayons');
   break;
   case 2:
      alert('Contient du matériel informatique : des câbles, des composants');
   break;
   case 3:
      alert('Ah ? Ce tiroir est fermé à clé ');
   break;
   case 4:
      alert('Contient  des chemises, des pantalons’);
   break;
   default:
      alert(‘ Le meuble ne contient que 4 tiroirs.’);
}
```



Exercice 1:

Modifier le code ci-dessous pour calculer la moyenne des notes.

```
let note_maths = 11;
let note_francais = 9;
let note_hg = 16;
let ??? = ???;
alert('La moyenne est de ' + ???);
```

Exercice 2:

- Déclarer une variable budget qui contient le budget de votre choix.
- Déclarer une variable achats qui contient le montant de votre choix.
- Afficher si le budget permet de payer les achats.

Exercice 3:

Grâce à la fonction prompt(), demander l'âge de l'utilisateur, puis à l'aide d'une condition, déterminer et afficher via une boite de dialogue si l'utilisateur est mineur ou majeur.

Les ternaires

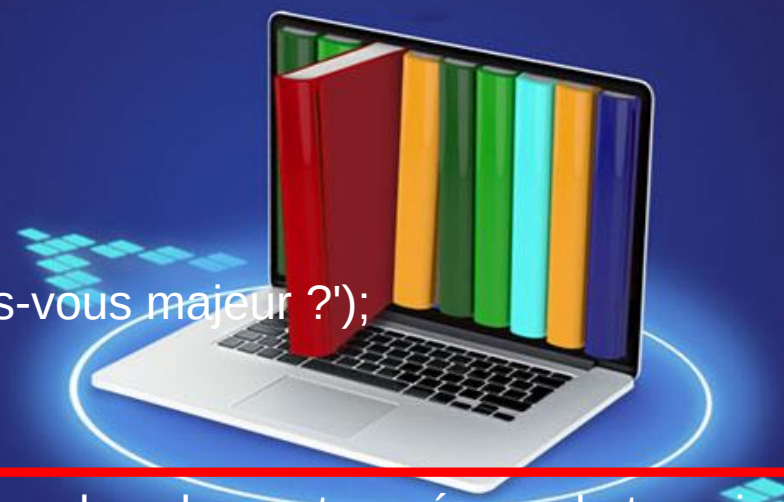
Les ternaires sont des conditions très simples et rapides à écrire

- **Exemple:**

```
var startMessage = 'Votre catégorie : ', endMessage, adult = confirm('Êtes-vous majeur ?');
if (adult) { // La variable « adult » contient un booléen, on peut donc directement la soumettre à la structure if sans opérateur conditionnel
   endMessage = '18+';
} else {
   endMessage = '-18';
}
alert(startMessage + endMessage);
```

- **Avec application des conditions ternaires**

```
var startMessage = 'Votre catégorie : ', endMessage, adult = confirm('Êtes-vous majeur ?');

endMessage = adult ? '18+' : '-18';

alert(startMessage + endMessage);
```



Si la variable **adult** vaut **true** alors la valeur retournée par la ternaire sera celle écrite juste après le point d'interrogation, si elle vaut **false** alors la valeur retournée sera celle après les deux points.

Les conditions sur les variables

Pour tester l'existence de contenu d'une variable, il faut tout d'abord savoir que tout se joue au niveau de la conversion des types :

```
var conditionTest = 'Fonctionnera ? Fonctionnera pas ?';
if (conditionTest) {
   alert('Fonctionne !');
} else {
   alert('Ne fonctionne pas !');
}
```

Ce code affiche le texte « Fonctionne ! ». Pourquoi ? Tout simplement parce que la variable conditionTest a été convertie en booléen et que son contenu est évalué comme étant vrai (true).
Qu'est-ce qu'un contenu vrai ou faux ? il suffit simplement de lister les contenus faux pour le savoir : un nombre qui vaut zéro ou bien une chaîne de caractères vide. Ces deux cas sont les seuls à être évalués comme étant à false.

Bien entendu, la valeur **undefined** est aussi évaluée à false.

Les boucles en Javascript

L'incrémentation:

Permet d'ajouter une unité à un nombre au moyen d'une syntaxe courte. À l'inverse, la décrémentation permet de soustraire une unité.

```
var number = 0;

number++;
alert(number); // Affiche : « 1 »

number--;
alert(number); // Affiche : « 0 »
```

Il s'agit donc d'une méthode assez rapide pour ajouter ou soustraire une unité à une variable (on dit incrémenter et décrémenter)

L'ordre des opérateurs:

Il existe deux manières d'utiliser l'incrémentation en fonction de la position de l'opérateur ++ (ou --).
On a vu qu'il pouvait se placer après la variable, mais il peut aussi se placer avant.

```
var number_1 = 0;
var number_2 = 0;

number_1++;
++number_2;

alert(number_1); // Affiche : « 1 »
alert(number_2); // Affiche : « 1 »
```

number_1 et number_2 ont tous deux été incrémentés.
Quelle est donc la différence entre les deux procédés ?

La différence réside en fait dans la priorité de l'opération, et ça a de l'importance si on veut récupérer le résultat de l'incrémentation. Dans l'exemple suivant, ++number retourne la valeur de number incrémentée, c'est-à-dire 1.

```
var number = 0;
var output = ++number;

alert(number); // Affiche : « 1 »
alert(output); // Affiche : « 1 »
```

Si on place l'opérateur après la variable à incrémenter, l'opération retourne la valeur de number avant qu'elle ne soit incrémentée :

```
var number = 0;
var output = number++;

alert(number); // Affiche : « 1 »
alert(output); // Affiche : « 0 »
```

L'opération number++ a retourné la valeur de number non incrémentée.

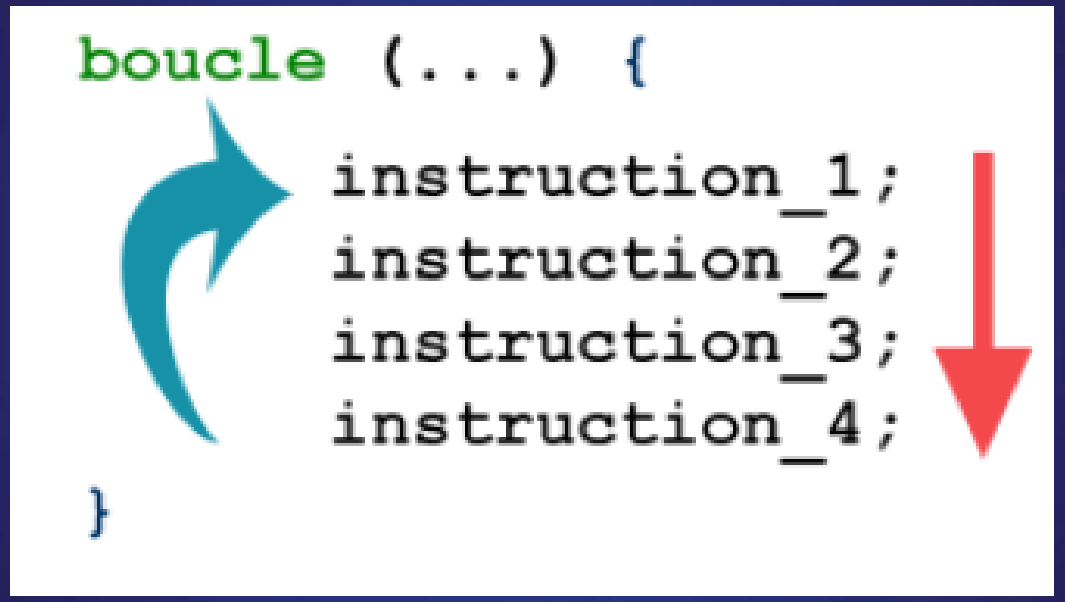
La boucle while

Il permet de répéter une série d'instructions. La répétition se fait jusqu'à ce qu'on dise à la boucle de s'arrêter. À chaque fois que la boucle se répète on parle d'itération (qui est en fait un synonyme de répétition).

Pour faire fonctionner une boucle, il est nécessaire de définir une condition. Tant que celle-ci est vraie (true), la boucle se répète. Dès que la condition est fausse (false), la boucle s'arrête.

Exemple de la syntaxe d'une boucle while :

```
while (condition) {
   instruction_1;
   instruction_2;
   instruction_3;
}
```



La **boucle while** se répète tant que la condition est validée. Cela veut donc dire qu'il faut s'arranger, à un moment, pour que la condition ne soit plus vraie, sinon la boucle se répéterait à l'infini.

Exemple, on va incrémenter un nombre, qui vaut 1, jusqu'à ce qu'il vaille 10 :

```
var number = 1;
while (number < 10) {
   number++;
}
alert(number); // Affiche : « 10 »
```

Au départ, number vaut 1. Arrive ensuite la boucle qui va demander si number est strictement plus petit que 10. Comme c'est vrai, la boucle est exécutée, et number est incrémenté. À chaque fois que les instructions présentes dans la boucle sont exécutées, la condition de la boucle est réévaluée pour savoir s'il faut réexécuter la boucle ou non. Dans cet exemple, la boucle se répète jusqu'à ce que number soit égal à 10. Si number vaut 10, la condition number < 10 est fausse, et la boucle s'arrête. Quand la boucle s'arrête, les instructions qui suivent la boucle (la fonction alert() dans notre exemple) sont exécutées normalement.

Exemple pratique

Ecrivons script qui va demander à l'internaute d’entrer à chaque fois un prénom. Nous allons utiliser une boucle while, qui va demander, à chaque passage dans la boucle, un prénom supplémentaire. La boucle ne s'arrêtera que lorsque l'utilisateur choisira de ne plus entrer de prénom.

```
var affiche = '', prenom, poursuivre = true;
while (poursuivre) {
   prenom = prompt('Entrez un prénom :');
   if (prenom) {
      affiche += prenom + ' '; // Ajoute le nouveau prénom ainsi
qu'une espace juste après
   } else {
      poursuivre = false; // Aucun prénom n'a été entré, donc
on fait en sorte d'invalider la condition
   }
}

alert(affiche); // Affiche les prénoms à la suite
```

La variable poursuivre est ce qu'on appelle une **variable témoin**, ou bien une variable de boucle. C'est une variable qui n'intervient pas directement dans les instructions de la boucle mais qui sert juste pour tester la condition.

À chaque passage dans la boucle, un prénom est demandé et sauvé temporairement dans la variable **prenom**. On effectue alors un test sur **prenom** pour savoir si elle contient quelque chose, et dans ce cas, on ajoute le prénom à la variable **affiche**. Si par contre **prenom** contient la valeur null — ce qui veut dire que l'utilisateur n'a pas entré de prénom ou a cliqué sur Annuler — on change la valeur de **poursuivre** en false, ce qui invalidera la condition, et cela empêchera la boucle de refaire une itération.

Utilisation de break

Dans l'exemple des prénoms, on a utilisé une variable de boucle pour pouvoir arrêter la boucle. Cependant, il existe un mot-clé pour arrêter la boucle d'un seul coup. Ce mot-clé est **break**, et il s'utilise exactement comme dans la structure conditionnelle switch:

```
var affiche = '', prenom;
while (true) {
   prenom = prompt('Entrez un prénom :');
   if (prenom) {
      affiche += prenom + ' '; // Ajoute le nouveau prénom ainsi qu'une espace juste après
   } else {
      break; // On quitte la boucle
   }
}
alert(affiche); // Affiche les prénoms à la suite
```

Utilisation de continue

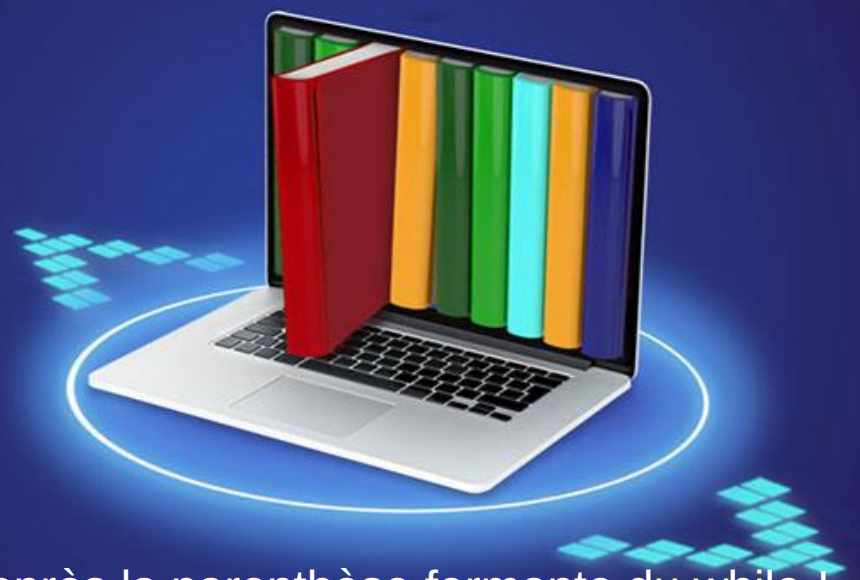
Cette instruction est plus rare, car les opportunités de l'utiliser ne sont pas toujours fréquentes. continue, un peu comme break, permet de mettre fin à une itération, mais attention, elle ne provoque pas la fin de la boucle : l'itération en cours est stoppée, et la boucle passe à l'itération suivante.

La boucle do while

La boucle do while ressemble très fortement à la boucle while, sauf que dans ce cas la boucle est toujours exécutée au moins une fois. Dans le cas d'une boucle while, si la condition n'est pas valide, la boucle n'est pas exécutée.

Avec do while, la boucle est exécutée une première fois, puis la condition est testée pour savoir si la boucle doit continuer.

```
do {
   instruction_1;
   instruction_2;
   instruction_3;
} while (condition);
```



Remarque: à la syntaxe de la boucle do while : il y a un point-virgule après la parenthèse fermante du while !

La boucle for

La boucle **for** ressemble dans son fonctionnement à la boucle **while**. Dans les parenthèses de la boucle ne se trouve plus juste la condition, mais trois blocs : initialisation, condition, et incrémentation.

```
for (initialisation; condition; incrémentation) {
   instruction_1;
   instruction_2;
   instruction_3;
}
```

Exemple:

On affichercinq fois une boîte de dialogue à l'aide de alert(), qui affichera le numéro de chaque itération :

```
for (var iter = 0; iter < 5; iter++) {
   alert('Itération n°' + iter);
}
```

- ✓ Dans le premier bloc, **l'*initialisation***, on initialise une variable appelée iter qui vaut 0 ; le **mot-clé var** est requis, comme pour toute initialisation.
- ✓ On définit dans la *condition* que la boucle continue tant qu'**iter** est strictement inférieure à 5.
- ✓ Enfin, dans le bloc d'*incrémentation*, on indique qu'**iter** sera incrémentée à chaque itération terminée.

Exemple:

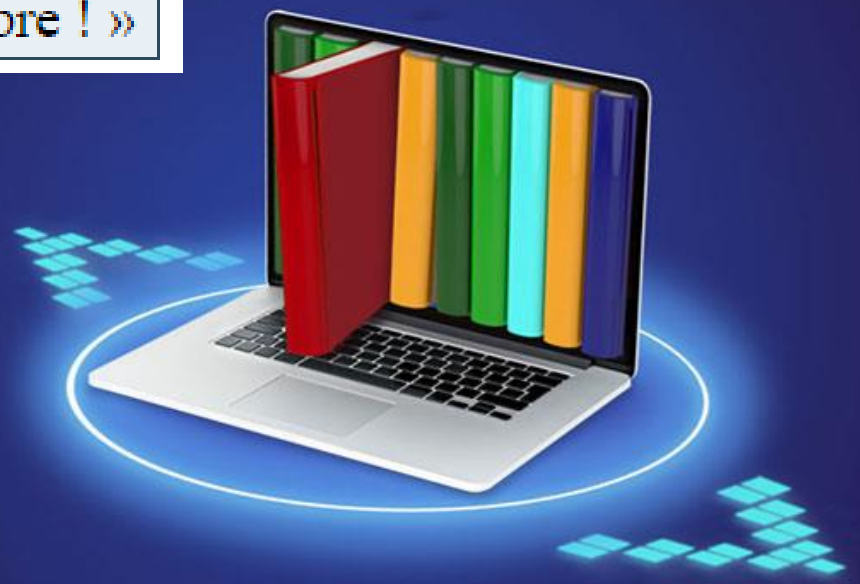
```
For (var i = 0, affiche = '', prenom; true; i++) {
    prenom = prompt('Entrez un prénom :');
        if (prenom) {
            affiche += prenom + ' ';
        } else {
         break;
        }
}
alert('Il y a ' + i + ' prénoms :\n\n' + affiche);
```

N.B: Les variables de boucles **for** sont généralement nommées i. Si une boucle se trouve dans une autre boucle, la variable de cette boucle sera nommée j, puis k et ainsi de suite. C'est une sorte de convention implicite, que l'on retrouve dans la majorité des langages de programmation.

Exercice :

L’objectif est de fournir un commentaire selon l'âge de la personne. Vous devez fournir un commentaire sur quatre tranches d'âge différentes qui sont les suivantes :

Tranche d'âge	Exemple de commentaire
1 à 17 ans	« Vous n'êtes pas encore majeur. »
18 à 49 ans	« Vous êtes majeur mais pas encore senior. »
50 à 59 ans	« Vous êtes senior mais pas encore retraité. »
60 à 120 ans	« Vous êtes retraité, profitez de votre temps libre ! »



- L’utilisateur doit taper son âge dans une fenêtre d'interaction ;
- Une fois l'âge fourni l'utilisateur obtient un petit commentaire.

L'intérêt de cet exercice est d’utiliser la structure conditionnelle la plus adaptée

Exercice :

Calculer le salaire d'une personne sur base de son salaire brut en décomptant les frais suivants:

- Impôts sur le revenu : 20%
- Assurance employé : 5%

Il est possible de recevoir des réductions sur les impôts sur le revenu sous certaines conditions :

- Si le travailleur est une femme, elle reçoit 2% de réduction.
- Si le travailleur a plusque 3 personnes à charge, il reçoit 3% de réduction.

Exercices structures conditionnelles et boucles

Exercice 1 :

Créer un script qui demande à l'utilisateur de saisir trois variables numériques via une boite prompt.
L'utilisateur est averti s'il ne saisit pas un nombre, le script lui demande de saisir une valeur numérique.
Une fois l'utilisateur à saisit trois valeurs numériques, le script teste les trois variables :

- Si elles sont égales, écrire : "Les trois variables sont identiques."
- Si seulement deux sont identiques, écrire :"deux des variables sont de valeurs égales."
- Sinon, écrire :"Les trois variables sont différentes."

Exercice 2:

Déclarer une variable avec le nom de votre choix et avec la valeur 0.
Tant que cette variable ne dépasse pas 20, il faut :

- l'afficher dans la page HTML;
- incrémenter sa valeur de 2 ;

Exercice 3:

Ecrivez un programme qui ouvre une porte après que l'utilisateur ait fourni un mot de passe. Le mot de passe est une chaîne de caractères donnée (et stockée dans une variable du programme). L'ouverture de la porte est simulée par l'affichage d'une fenêtre avec le message "la porte est ouverte".
Tant que le mot de passe fourni est erroné, le programme le redemande et n'ouvre pas la porte. Lorsque le bon mot de passe est fourni, la porte s'ouvre.

Exercice 4:

Utilisez une (ou deux) boucle for() pour afficher la figure ci-dessous :

```
1
1 1
1 1 1
1 1 1 1
1 1 1 1 1
1 1 1 1 1 1
1 1 1 1 1 1 1
1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
```

Les fonctions en JavaScript

L’avantage de création des fonctions est de pouvoir exécuter un code assez long et complexe juste en appelant la fonction le contenant. Cela réduit considérablement le code et le simplifie.
En plus des fonctions **natives du JavaScript (alert(), prompt(), confirm(), etc…).** On peut créer nos propres fonctions.

Syntaxe création de fonction:

```
function nomFonction(arguments) {
   // Le code que la fonction va devoir exécuter
}
```

- ✓ Le mot-clé function est présent à chaque déclaration de fonction Suivi du nom de la fonction
- ✓ Un couple de parenthèses contenant ce que l'on appelle des arguments. Ces arguments servent à fournir des informations à la fonction lors de son exécution. Par exemple, avec la fonction alert() quand on lui passe en paramètre ce qu’on veut afficher à l'écran ;
- ✓ Un couple d'accolades contenant le code que la fonction devra exécuter.

N.B:

- Il est important de préciser que tout code écrit dans une fonction ne s'exécutera que si on appelle cette dernière (« appeler une fonction » signifie « exécuter »). Sans ça, le code qu'elle contient ne s'exécutera jamais.
- Tout comme les variables, les noms de fonctions sont limités aux caractères alphanumériques (dont les chiffres) et aux deux caractères suivants : _ et $.

Exemple :

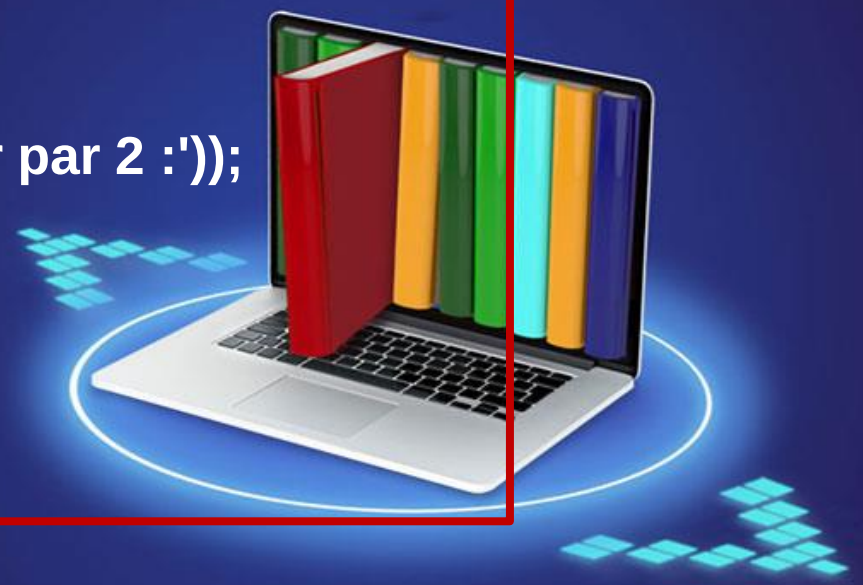
```
function showMsg() {

   alert(‘Notre première fonction !');
}
showMsg();
```

Code sans fonction:

```
var result;
result = parseInt(prompt('Donnez le nombre à multiplier par 2 :'));
alert(result * 2);
alert('Vous en êtes à la moitié !');
result = parseInt(prompt('Donnez le nombre à multiplier par 2 :'));
alert(result * 2);
```

Code avec fonction:

```
function byTwo() {
   var result = parseInt(prompt('Donnez le nombre à multiplier par 2 :'));
   alert(result * 2);
}
byTwo();
alert('Vous en êtes à la moitié !');
byTwo();
```



La portée des variables

Portée locale:

```
var essai = 'Hello world !';
function sayHello() {
   alert(essai);
}
sayHello();
```

```
function sayHello() {
  var essai = 'Hello world !';
}
sayHello();
alert(essai);
```

N.B: Toute variable déclarée dans une fonction n'est utilisable que dans cette même fonction ! Ces variables spécifiques à une seule fonction ont un nom : les **variables locales**.

Portée Globale:

À l'inverse des variables locales, celles déclarées en-dehors d'une fonction sont nommées les variables globales car elles sont accessibles partout dans le code, y compris à l'intérieur des fonctions.

```
var message = 'Ici la variable globale !';
function showMsg() {
   var message = 'Ici la variable locale !';
   alert(message);
}
showMsg();
alert(message);
```

Les arguments et les valeurs de retour

Ils permettent de faire communiquer les fonctions avec le reste du code. Ainsi, les arguments permettent d'envoyer des informations à une fonction tandis que les valeurs de retour représentent tout ce qui est retourné par la fonction une fois que celle-ci a fini de travailler.

Les arguments: sont des informations envoyées à une fonction; Selon les fonctions, on n'aura parfois pas besoin de spécifier d'arguments, parfois il faudra en spécifier un, voire plusieurs

```
// Voici la fonction alert sans argument, elle n'affiche rien :
   alert();
// Et avec un argument, elle affiche ce qu’on lui a envoyé :

   alert('Mon message à afficher');
```

Exemple 1:

```
function maFonction (arg) {
   alert('Votre argument : ' + arg);
}
maFonction (‘Premier test des arguments');
```

Exemple 2:

```
function maFonction(arg) {

alert('Votre argument : ' + arg);

}
maFonction(prompt('Tapez ce que vous
voulez'));
```

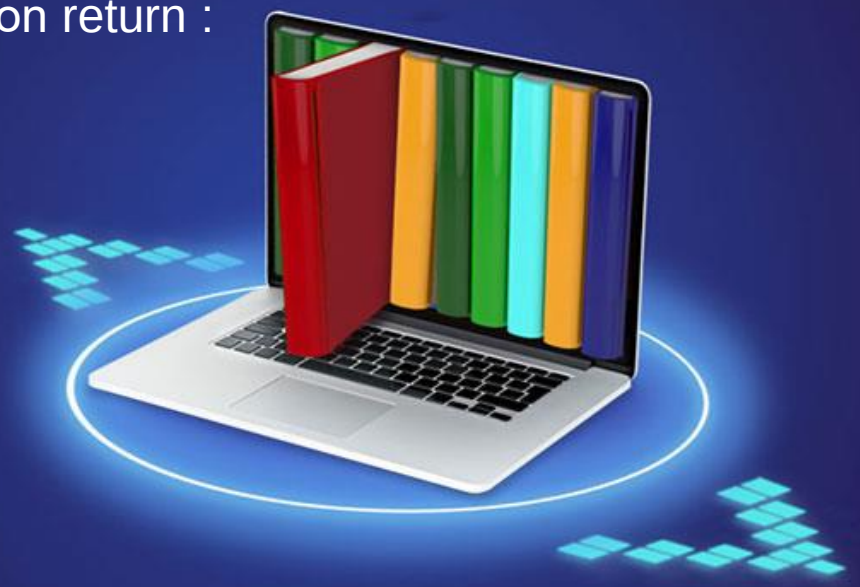
Les valeurs de retour: Ce sont les valeurs qu'on peut retourner avec une fonction. Exemple: les fonctions prompt(), confirm() et parseInt(), chacune d'entre elles renvoyait une valeur que l'on stocke dans une variable.
Il est tout important de préciser que les fonctions ne peuvent retourner qu'une seule et unique valeur chacune. Pour faire retourner une valeur à une fonction, on utilise **l'instruction return** suivie de la valeur à retourner.

Exemple :

```
function sayHello() {
   return 'Bonjour !';}
alert(sayHello()); // Ici on affiche la valeur retournée par la fonction sayHello()
```

Exemple 2: Ajouter une ligne de code après la ligne contenant l'instruction return :

```
function sayHello() {
   return 'Bonjour !';
   alert('Attention ! Le texte arrive !');
}
alert(sayHello());
```



Remarque:

Le premier **alert()** ne s'est pas affiché. Cela s'explique par la présence du return : cette instruction met fin à la fonction, puis retourne la valeur.

Ce fonctionnement explique pourquoi on ne peut pas faire plusieurs renvois de valeurs pour une même fonction : si on écrit deux return à la suite, seul le premier sera exécuté puisque le premier return aura déjà mis un terme à l'exécution de la fonction.

Fonction anonyme

Les fonctions anonymes sont, comme leur nom l'indique, des fonctions qui ne vont pas posséder de nom.

En effet, lorsqu'on crée une fonction, nous on est pas obligé de lui donner un nom.

Généralement, on utilisera les fonctions anonymes lorsqu'on n'a pas besoin d'appeler la fonction par son nom c'est-à-dire lorsque le **code de notre fonction n'est appelé qu'à un endroit dans notre script et n'est pas réutilisé.**

En d'autres termes, les fonctions anonymes vont très souvent permettre de gagner un peu de temps dans l'écriture du code et à le rendre plus clair en ne le polluant pas avec des noms inutile

Création et exécution ou appel d’une fonction anonyme

On va pouvoir créer une fonction anonyme de la même façon qu’une fonction classique, en utilisant le mot clef **function** mais en omettant le nom de la fonction après.

Exemple de définition d'une fonction anonyme en JavaScript:

```
function(){
    alert('Alerte exécutée par une fonction anonyme');
}
```

Nous avons ici déclaré une fonction anonyme dont le rôle est d’exécuter une fonction alert() qui va elle-même renvoyer le message « Alerte exécutée par une fonction anonyme » dans une boite d’alerte.

Comment appeler une fonction qui n’a pas de nom ?

Pour exécuter une fonction anonyme, on peut :

- Enfermer le code de la fonction anonyme dans une variable et utiliser la variable comme une fonction ;
- Auto-invoquer la fonction anonyme ;
- Utiliser un évènement pour déclencher l’exécution de la fonction.

1. Exécuter une fonction anonyme en utilisant une variable

Enfermer la fonction dans une variable et utiliser la variable comme une fonction.

```
let alerte = function(){
    alert('Alerte exécutée par une fonction anonyme');
}

alerte();
```

- ✓ On affecte la fonction anonyme à une variable nommée alerte. cette variable contient alors une valeur complexe qui est une fonction

et on va désormais pouvoir l'utiliser comme si c'était une fonction elle-même.

- ✓ Pour « appeler notre variable » et pour exécuter le code de la fonction anonyme qu'elle contient, il va falloir écrire le nom de la variable suivi d'un couple de parenthèses. Ces parenthèses sont des parenthèses dites « appelantes » car elles servent à exécuter la fonction qui les précède.

2. Auto-invoquer une fonction anonyme

La deuxième façon d'exécuter une fonction anonyme est de créer une fonction anonyme qui va s'auto-invoquer c'est-à-dire qui va s'invoquer (ou s'appeler ou encore s'exécuter) elle-même dès sa création.

Pour créer une fonction auto-invoquée à partir d'une fonction, il va falloir rajouter un couple de parenthèses autour de la fonction et un second après le code de la fonction.

On a vu précédemment que le couple de parenthèses suivant le nom de la variable stockant notre fonction anonyme servait à lancer l'exécution de la fonction. De la même manière, le couple de parenthèses après la fonction va faire en sorte que la fonction s'appelle elle-même.

```
//Fonction anonyme auto-invoquée
(function(){alert('Alerte exécutée par une fonction anonyme')})();

//Fonction nommée auto-invoquée
(function bonjour(){alert('Bonjour !')})();
```

Remarque :

- Il faut savoir que la notion d'auto-invocation n'est pas réservée qu'aux fonctions anonymes : on va tout à fait pouvoir auto-invoquer une fonction qui possède un nom. Cependant, en pratique, cela n'aura souvent pas beaucoup d'intérêt (puisque si une fonction possède un nom, on peut tout simplement l'appeler en utilisant ce nom).

- Il faut bien comprendre que lorsqu'on auto-invoque une fonction, la fonction s'exécute immédiatement et on n'a donc pas de flexibilité par rapport à cela : une fonction auto-invoquée s'exécutera toujours juste après sa déclaration.

3. Exécuter une fonction anonyme lors du déclenchement d’un évènement

On peut rattacher les fonctions anonymes à ce qu’on appelle des « gestionnaires d’évènements » en JavaScript.

Le langage JavaScript va en effet nous permettre de répondre à des évènements, c’est-à-dire d’exécuter certains codes lorsqu’un évènement survient.

Le JavaScript permet de répondre à de nombreux types d’évènements : clic sur un élément, pressage d’une touche sur un clavier, ouverture d’une fenêtre, etc.

Pour indiquer comment on veut répondre à tel évènement, on utilise des gestionnaires d’évènements qui sont des fonctions qui vont exécuter tel code lorsque tel évènement survient.

On va pouvoir passer une fonction anonyme à un gestionnaire d‘évènement qui va l’exécuter dès le déclenchement de l’évènement que le gestionnaire prend en charge.

Exercice sur les fonctions

Exercice 1:

Créer une fonction qui s'appelle **concatenation().** Elle prendra deux arguments de type string (chaine de caractères). Elle devra retourner la concaténation des deux. Exemple : argument 1 = Steven et Argument 2 = Hanse => Resultat : StevenHans

Exercice 2:

Crée une fonction qui s'appelle **somme().** Elle prendra deux arguments de type int (nombre entier). Elle devra retourner la somme des deux.

Exercice 3:

Créer une fonction qui s'appelle **estMajeure().** Elle prendra un argument de type int.
Elle devra retourner un boolean.
Si age >= 18 elle doit retourner true,
si age < 18 elle doit retourner false

Exemple : age = 5 ==> false age = 34 ==> true

Exercice 4:

Créer une fonction qui s'appelle **plusGrand().** Elle prendra deux arguments de type int. Elle devra retourner le plus grand des deux.

Exercice 5:

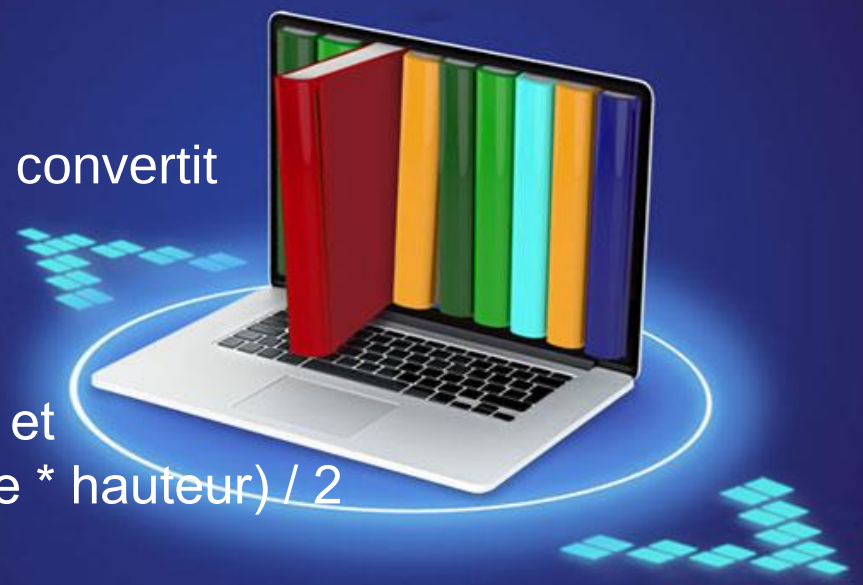
Créer une fonction qui s'appelle plusPetit(). Elle prendra trois arguments de type int. Elle devra retourner le plus petit des trois.

Exercice 6:

Écrivez une fonction qui prend un nombre entier de minutes et le convertit en secondes. Exemple: minuteToSecond(4)➡ 240

Exercice 7:

Écrivez une fonction qui prend la base et la hauteur d'un triangle et retourne sa surface. Notez que la surface d'un triangle est: (base * hauteur) / 2



Exercice 8:

Créez une fonction qui prend un nombre en argument, incrémente le nombre+1 et renvoie le résultat

Exercice 9:

Écrivez un programme JavaScript pour renvoyer le reste de deux nombres. Deux nombres sont transmis comme paramètres. Le premier paramètre divisé par le deuxième paramètre.

Exercice 10:

Créer une fonction qui s'appelle verificationPassword(). Elle prendra un argument de type string. Elle devra retourner un boolean qui vaut true si le password fait au moins 8 caractères et false si moins.(utiliser la propriété **.length** pour calculer le nombre de caractère)

Exercice 11:

Créer une fonction qui s'appelle capital(). Elle prendra un argument de type string. Elle devra retourner le nom de la capitale des pays suivants :

France ==> Paris
Allemagne ==> Berlin
Italie ==> Rome
Maroc ==> Rabat
Espagne ==> Madrid
Portugal ==> Lisbonne
Angleterre ==> Londres
Tout autre pays ==> Inconnu

Pensez à utiliser SWITCH pour cet exercice.

Introduction à l’orienté objet en JavaScript

Les paradigmes de programmation

Un « paradigme » de programmation est une façon d’approcher la programmation informatique, c’est-à-dire une façon de voir (ou de construire) son code et ses différents éléments.
Il existe plusieurs paradigmes de programmation, c’est-à-dire la façon de penser son code, on cite parmi eux :

La programmation procédurale ;
La programmation orientée objet.

- ✓ La programmation procédurale est le type de programmation le plus commun et le plus populaire. C’est une façon d’envisager son code sous la forme d’un enchainement de procédures ou d’étapes qui vont résoudre les problèmes un par un. Cela correspond à une approche verticale du code où celui-ci va s’exécuter de haut en bas, ligne par ligne.

- ✓ La programmation orientée objet est une façon de concevoir un code autour du **concept d’objets**. Un objet est une entité qui peut être vue comme indépendante et qui va contenir un ensemble de variables (qu’on va appeler propriétés) et de fonctions (qu’on appelle méthodes). Ces objets vont pouvoir interagir entre eux.

Définition de l’orienté objet et des objets en JavaScript:

Un objet, est un ensemble cohérent de données et de fonctionnalités qui vont fonctionner ensemble. En effet, c’est un conteneur qui va pouvoir stocker plusieurs variables qu’on va appeler ici des propriétés. Lorsqu’une propriété contient une fonction en valeur, on appelle alors la propriété une méthode. Un objet est donc un conteneur qui va posséder un ensemble de propriétés et de méthodes.

Quels avantages et intérêts de coder en orienté objet en JavaScript ?

Les grands enjeux et avantages de la programmation orientée objet sont de nous permettre d’obtenir des scripts mieux organisés, plus clairs, plus facilement maintenables et plus performants en groupant des ensembles de données et d’opérations qui ont un rapport entre elles au sein d’objets qu’on va pouvoir manipuler plutôt que de réécrire sans cesse les mêmes opérations. On va généralement utiliser la programmation orientée objet dans le cadre de gros projets où on doit répéter de nombreuses fois des opérations similaires

Création d’un objet JavaScript littéral et manipulation de ses membres

Le JavaScript dispose d’objets **natifs (objets prédéfinis)** qui possèdent des propriétés et des méthodes qu’on va pouvoir directement utiliser et nous permet également de définir nos propres objets.

On peut créer des objets de 4 manières différentes en JavaScript. On va pouvoir :

- Créer un objet;
- Utiliser le constructeur Object() ;
- Utiliser une fonction constructeur personnalisée ;
- Utiliser la méthode create().

Ces différents moyens de procéder peuvent être utilisés dans des contextes différents, selon ce que l’on souhaite réaliser.

❑ Création d'un objet :

```
let pierre = { // "pierre" est une variable qui contient un objet. nom, age et mail sont des propriétés de l'objet "pierre"
    nom : ['Pierre', 'Giraud'],
    age : 29,
    mail : 'pierre.giraud@edhec.com',

    //Bonjour est une méthode de l'objet pierre
    bonjour: function(){
        alert('Bonjour, je suis ' + this.nom[0] + ', j\'ai ' + this.age + ' ans');
    }
};
```

On parle ici d'objet « littéral » car on a défini chacune de ses propriétés et de ses méthodes lors de la création, c'est-à-dire littéralement.

L'utilisation du mot clef this: **C'**est un mot clef qui apparait fréquemment dans les langages orientés objets. Il sert à faire référence à l'objet qui est couramment manipulé. C'est un prête nom qui va être remplacé par le nom de l'objet lorsqu'on écrit this.bonjour(), le mot clef this va être remplacé par pierre.

Pour créer un objet, on utilise une syntaxe utilisant une paire d'accolades { … } qui indique au JavaScript que nous créons un objet. Ces objets vont généralement être stockés dans des variables

Un objet est composé de différents couples de **« nom : valeur »** qu'on appelle membres (**exemple: age : 29,)**
Chaque nom d'un membre doit être séparé de sa valeur par un caractère deux-points **:** et les différents membres d'un objet doivent être séparés les uns des autres par une virgule.

La partie « nom » de chaque membre suit les mêmes règles que le nommage d'une variable. La partie valeur d'un membre peut être n'importe quel type de valeur : une chaine de caractère, un nombre, une fonction, un tableau…

Les membres d'un objet qui ne servent qu'à stocker des données sont appelés des propriétés tandis que ceux qui manipulent des données (c'est-à-dire ceux qui contiennent des fonctions) sont appelés des méthodes.

Utiliser le point pour accéder aux membres d’un objet, les modifier ou en définir de nouveaux

Pour accéder aux propriétés et aux méthodes d’un objet, on utilise le caractère point **.** qu’on appelle également un accesseur. On commence par préciser le nom de l’objet puis l’accesseur puis enfin le membre auquel on souhaite accéder.

Cet accesseur va permettre non seulement d’accéder aux valeurs des différents membres mais également de modifier ces valeurs:

```
let pierre = {
    //nom, age et mail sont des propriétés de l'objet "pierre"
    nom : ['Pierre', 'Giraud'],
    age : 29,
    mail : 'pierre.giraud@edhec.com',
    bonjour: function(){
        alert('Bonjour, je suis ' + this.nom[0] + ', j\'ai ' + this.age + ' ans');
    }
};
pierre.age = 30; //On modifie la valeur de la propriété "age" de "pierre"
pierre.taille = 170;// on ajoute une propriété taille et une méthode prez() à notre objet pierre
pierre.mesure = function(){
    alert('Bonjour, je suis ' + this.nom[0] +  ', j\'ai ' + this.age + ' ans et je mesure ' + this.taille + 'cm');}
```

L’utilité d’un constructeur d’objets

Exemple: On souhaite créer un objet à chaque fois qu’un utilisateur enregistré se connecte sur notre site. Chaque objet « utilisateur » va posséder des propriétés (un pseudonyme, une date d’inscription, etc.) et des méthodes similaires (possibilité de mettre à jour ses informations, etc.).

Dans ces cas-là, plutôt que de créer les objets un à un , il serait pratique de créer une sorte de plan ou de schéma à partir duquel on pourrait créer des objets similaires à la chaine.
On peut faire cela en JavaScript en utilisant ce qu’on appelle un **constructeur d’objets** qui n’est autre qu’une **fonction constructeur**

La fonction construction d’objets : définition et création d’un constructeur

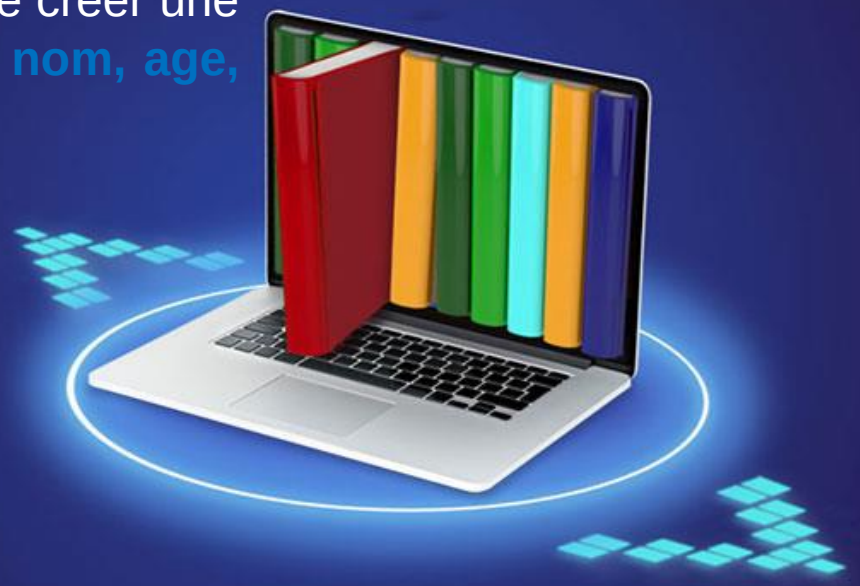
Une fonction constructeur d’objets est une fonction qui permet de créer des objets semblables.

Pour construire des objets à partir d’une fonction constructeur, on doit suivre deux étapes : il faut définir la fonction constructeur et ensuite on doit appeler ce constructeur avec une syntaxe spéciale utilisant le mot clef new.

Dans une fonction constructeur, on va pouvoir définir un ensemble de propriétés et de méthodes. Les objets créés à partir de ce constructeur vont automatiquement posséder les (« hériter des ») propriétés et des méthodes définies dans le constructeur.

Remarque: Les fonctions sont en fait un type particulier d'objets en JavaScript. Et comme tout autre objet, une fonction peut donc contenir des propriétés et des méthodes.

Exemple: on va se baser sur l'objet crée précédemment . L'objectif est de créer une fonction qui permet de créer des objets possédant les mêmes propriétés **nom, age, mail et méthode bonjour()**

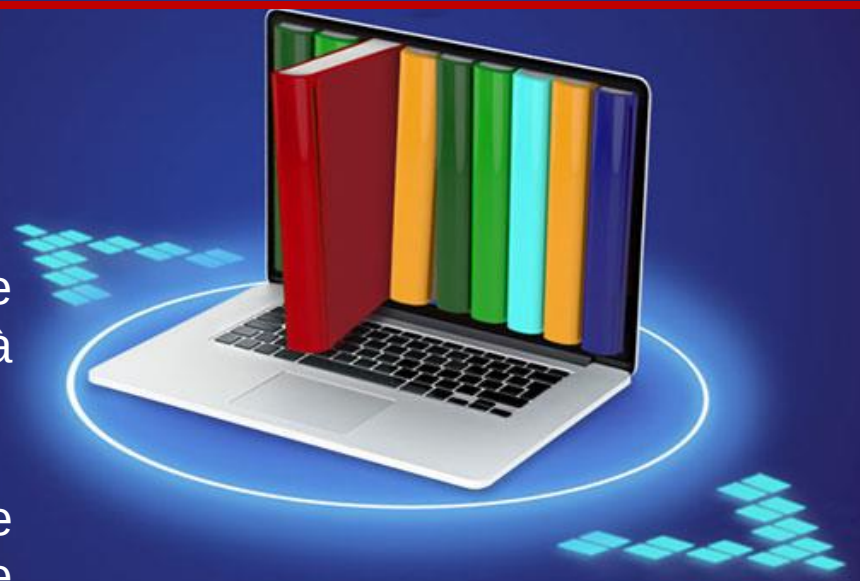


On définit ici une fonction Utilisateur() qu'on va utiliser comme constructeur d'objets.
Lorsqu'on définit un constructeur, on utilise par convention une majuscule au début du nom de la fonction afin de bien différencier les constructeurs des fonctions classiques dans un script.

```
function Utilisateur(n, a, m){
    this.nom = n;
    this.age = a;
    this.mail = m;

    this.bonjour = function(){
        alert('Bonjour, je suis ' + this.nom[0] + ', j\'ai ' +
this.age + ' ans');
    }
}
let pierre = new Utilisateur(['Pierre', 'Giraud'], 29,
'pierre.giraud@edhec.com');
```

L'utilisation du mot clef **this** va permettre de définir et d'initialiser les propriétés ainsi que les méthodes de chaque objet créé.

Le constructeur possède trois paramètres qu'on a ici nommé n, a et m qui vont nous permettre de transmettre les valeurs liées aux différentes propriétés pour chaque objet.



A chaque création d'objet, c'est-à-dire à chaque appel de notre constructeur, on va passer en argument les valeurs de l'objet relatives à ses propriétés nom, age et mail.

Pour créer ensuite de manière effective des objets à partir de notre constructeur, nous allons simplement appeler le constructeur en utilisant le mot **clef new**. On dit également qu'on **crée une nouvelle instance**.

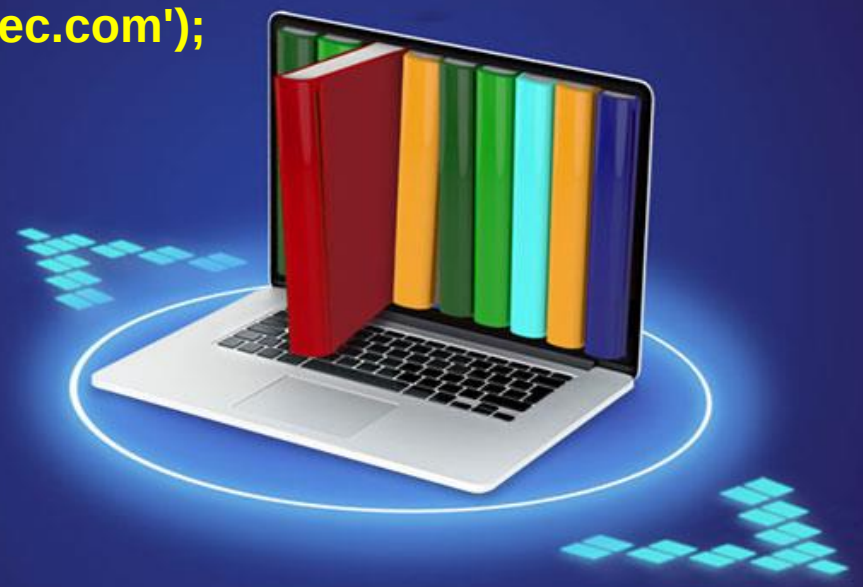
Lorsqu’on écrit **let pierre = new Utilisateur(['Pierre', 'Giraud'], 29, 'pierre.giraud@edhec.com'),** on crée un nouvel objet pierre en appelant la fonction **constructeur Utilisateur().**

Lors de l’exécution du constructeur, la ligne this.nom = n va donc être remplacée par pierre.nom = ['Pierre', 'Giraud'] ce qui crée une propriété nom pour notre objet pierre avec la valeur ['Pierre', 'Giraud'] et etc.

Comme le constructeur est une fonction, on va pouvoir l’appeler autant de fois qu’on le veut et donc créer autant d’objets que souhaité à partir de celui-ci

Exemple:

```
let pierre = new Utilisateur(['Pierre', 'Giraud'], 29, 'pierre.giraud@edhec.com');
let mathilde = new Utilisateur(['Math', 'Ml'], 27, 'math@edhec.com');
let florian = new Utilisateur(['Flo', 'Dchd'], 29, 'flo.dchd@gmail.com');
```

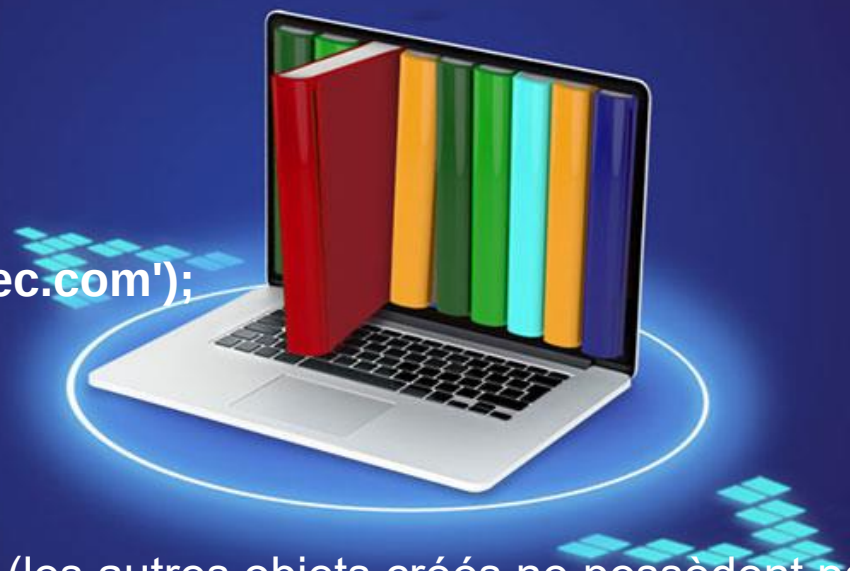


Constructeur et différenciation des objets

La fonction constructeur en JavaScript est comme un plan de base pour la création d'objets similaires et comme un moyen pour gagner du temps. On ne va définir dans cette fonction que les caractéristiques communes de nos objets et on pourra ensuite rajouter à la main les propriétés particulières à un objet.

Exemple: on peut rajouter une propriété taille à l'objet pierre après sa création

```
function Utilisateur(n, a, m){
    this.nom = n;
    this.age = a;
    this.mail = m;

    this.bonjour = function(){
        alert('Bonjour, je suis ' + this.nom[0] + ', j\'ai ' + this.age + ' ans');
    }}
let pierre = new Utilisateur(['Pierre', 'Giraud'], 29, 'pierre.giraud@edhec.com');
let florian = new Utilisateur(['Flo', 'Dchd'], 29, 'flo.dchd@gmail.com');

pierre.taille = 170;
```



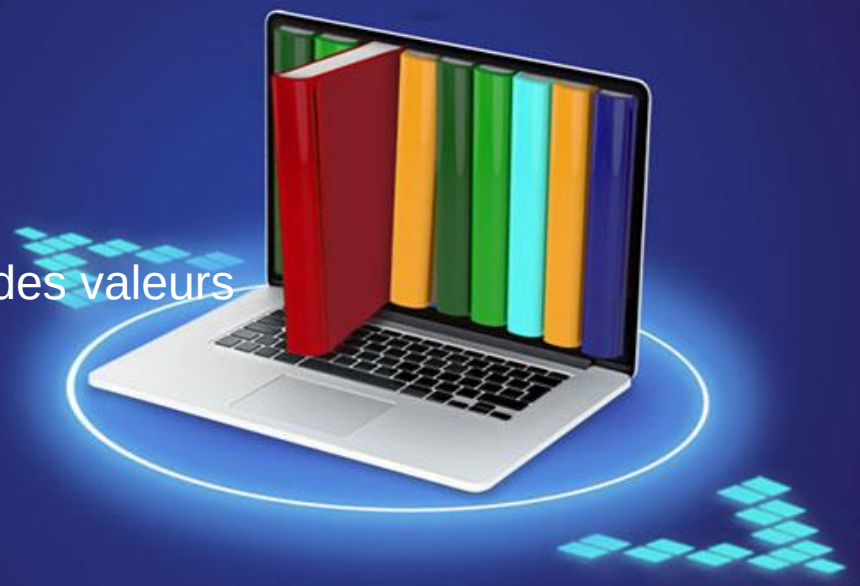
L'objet pierre dispose désormais d'une propriété taille qui lui est exclusive (les autres objets créés ne possèdent pas cette propriété)

Rappel sur les types de valeurs

En JavaScript, il existe 7 types de valeurs différents. Chaque valeur qu'on va pouvoir créer et manipuler en JavaScript va obligatoirement appartenir à l'un de ces types. Ces types sont les suivants :

1. string ou « chaine de caractères » ;
2. number ou « nombre »;
3. boolean ou « booléen »;
4. null ou « nul / vide » ;
5. undefined ou « indéfini »;
6. symbol ou « symbole »;
7. object ou « objet »;

Les valeurs appartenant aux 6 premiers types de valeurs sont appelées des valeurs primitives. Les valeurs appartenant au type object sont des objets



Définition des valeurs primitives et différence avec les objets:

Le JavaScript possède deux grandes catégories de types de données : les valeurs primitives et les objets. On appelle valeur primitive en JavaScript une valeur qui n’est pas un objet et qui ne peut pas être modifiée.

En effet, une fois un nombre ou une chaine de caractères définis, on ne va plus pouvoir les modifier en JavaScript. Si on stocke une chaine de caractères dans une variable, on va pouvoir écraser cette chaine pour stocker une autre valeur mais la chaine de caractères stockée n’aura pas été modifiée : elle aura été écrasée et c’est une nouvelle valeur différente qui va être stockée . Cela va être différent pour les objets : on va tout à fait pouvoir modifier les membres d’un objet.

Autre différence notable entre valeurs primitives et objets :

les valeurs primitives sont passées et comparées par valeur tandis que les objets sont passés et comparés par référence.

- Si deux valeurs primitives ont la même valeur, elles vont être considérées égales.

- Si deux objets définissent les mêmes propriétés et méthodes avec les mêmes valeurs, ils ne vont pas être égaux. Pour que deux objets soient égaux, il faut que les deux fassent référence aux mêmes membres

L’objet global JavaScript String

Propriétés de l’objet String:

La propriété **length** permet d’obtenir la longueur d’une chaine de caractères. Cette longueur est exprimée en points de code (appelées « codets ») sur la base du format UTF-16.

La plupart des caractères comptent pour une unité ou un codet mais certains caractères spéciaux vont être représentés par deux codets.

N.B : la propriété **length** ne renverra pas toujours une valeur égale au nombre de caractères présents dans la chaine.

let ch1 = 'Pierre'; ***// renvoie 6***

let ch2 = 'Pierre Giraud'; ***// renvoie 13***

Les méthodes de l’objet String:

Le constructeur String() dispose d’une trentaine de méthodes. Les plus utilisés sont:

❑ La méthode includes():

La méthode includes() permet de déterminer si une chaine de caractères est inclue dans une autre. Cette méthode prend l’expression (la chaine) à rechercher en argument.

Si la chaine passée en argument est trouvée dans la chaine dans laquelle on effectue la recherche, includes() renvoie le booléen true. Dans le cas contraire, la méthode renvoie le booléen false.

N.B: cette méthode est sensible à la casse, ce qui signifie qu’une lettre majuscule et une lettre minuscule correspondent à deux entités différentes pour includes().

```
<script>
let phrase = 'Bonjour, je suis étudiant en première année dev 103';

if(phrase.includes(‘dev')){
    alert ('"dev" présent dans la phrase');
}
</script>
```

Les méthodes startsWith() et endsWith():

- La méthode **startsWith()** permet de déterminer si une chaine commence par une certaine sous chaine (ou expression). Si c'est le cas, cette méthode renvoie true. Dans le cas contraire, c'est le booléen false qui est renvoyé.

- La méthode **endsWith()** permet de déterminer si une chaine se termine par une certaine sous chaine. Elle va fonctionner exactement de la même manière que startsWith().

```
<script>

let phrase = 'Bonjour,je suis étudiant en première année dev 103' ;

if(phrase .startsWith('Bonjour')){
   alert('La chaine commence par "Bonjour"');
}
if(phrase .endsWith('dev 103')){
   alert('La chaine se termine par "dev 103 "');
}
</script>
```

La méthode substring():

La méthode **substring()** retourne une sous-chaîne de la chaîne courante à partir d'un indice de départ.

Cette méthode demande un indice de départ en argument obligatoire qui va servir à indiquer la position de départ de la sous-chaine. On peut passer un deuxième indice facultatif pour préciser une position de fin.

N.B: dans le cas où on fournit une position de départ qui se situe après la position de fin, la méthode substring() intervertira automatiquement les deux valeurs.

```
<script>

let phrase = 'Bonjour, je suis étudiant en première année dev 103';

let p1 = phrase.substring(9);
let p2 = phrase.substring(9, 29);
let p3 = phrase.substring(29, 9);
alert(p1);
alert(p2);
alert(p3);

</script>
```

Les méthodes indexOf() et lastIndexOf():

- La méthode **indexOf()** permet de déterminer la position de la première occurrence d'un caractères ou d'une chaine de caractères dans une chaîne de caractères de base. Cette méthode va prendre l'expression à rechercher dans la chaine de caractères en argument et va renvoyer la position à laquelle cette expression a été trouvée la première fois dans la chaine si elle est trouvée ou la valeur -1 si l'expression n'a pas été trouvée dans la chaine.On peut aussi passer un deuxième argument **optionnel à indexOf()** qui correspond à l'endroit où on souhaite faire démarrer la recherche dans la chaine. Par défaut, la recherche se fait dans toute la chaine.

N.B: la méthode **indexOf()** est sensible à la casse, ce qui signifie qu'une lettre majuscule et une lettre minuscule correspondent à deux entités différentes pour elle.

- La méthode **lastIndexOf()** fonctionne de la même manière que indexOf() à la différence que c'est la position de la dernière occurrence de l'expression cherchée qui va être renvoyée (ou -1 si l'expression n'est pas trouvée dans la chaine).

```
<script>
let phrase = 'Bonjour,je suis étudiant en première année dev 103';
alert(phrase.indexOf('e'));
alert(phrase.indexOf('Salut'));
alert(phrase.lastIndexOf('e'));
</script>
```

La méthode slice():

La méthode **slice()** extrait une section d'une chaine de caractères et la retourne comme une nouvelle chaine de caractères. La chaîne de caractères de départ n'est pas modifiée. On doit fournir en argument de départ obligatoire la position de départ dans la chaine de caractères de départ où doit démarrer l'extraction. On peut également passer en deuxième argument optionnel la positon où l'extraction doit s'arrêter.

Cette méthode va donc fonctionner comme **substring()** à deux différences :

- En passant des valeurs négatives en argument à slice(), les positions de départ et de fin d'extraction seront calculées à partir de la fin de la chaine de caractères à partir de laquelle on extrait ;
- En passant une position de départ plus lointaine que la position d'arrivée à slice(), cette méthode n'inverse pas les valeurs mais renvoie une chaine de caractères vide.
- La méthode slice() ne modifie pas la chaine de caractères d'origine mais renvoie une nouvelle chaine.

```
<script>

let phrase = 'Bonjour,je suis étudiant en première année dev 103';

alert (phrase.slice(0, 29)) ;
alert(phrase.slice(29, 9));
alert(phrase.slice(-34, -14));

 </script>
```

La méthode replace():

La méthode **replace()** permet de rechercher une expression dans une chaine de caractères et de la remplacer par une autre. On passe deux arguments à cette méthode : l'expression à rechercher et l'expression de remplacement.

La méthode **replace()** va renvoyer une nouvelle chaine de caractères avec les remplacements faits. La chaine de caractères de départ ne sera pas modifiée.

N.B: dans le cas où on passe une expression de type chaine de caractères à rechercher, seule la première occurrence dans la chaine sera remplacée. Pour pouvoir remplacer toutes les occurrences, il faudra passer une expression régulière comme schéma de recherche à cette méthode

```
<script>

let phrase = 'Bonjour,je suis étudiant en première année dev 103';

alert(phrase.replace('103', '104'));
alert(phrase.replace('e', 'E'));

</script>
```

Les méthodes toLowerCase() et toUpperCase():

La méthode **toLowerCase()** retourne une chaine de caractères en minuscules à l'inverse, la méthode **toUpperCase()** qui retourne une chaine de caractères en majuscules.

Ces deux méthodes retournent une nouvelle chaine de caractères et ne modifient pas la chaine de caractères de base.

```
<script>

let phrase = 'Bonjour,je suis étudiant en première année dev 103';

alert(phrase.toLowerCase());
alert(phrase.toUpperCase());

</script>
```

L’objet global JavaScript Number

La méthode isInteger():

La méthode **isInteger()** permet de déterminer si une valeur est un entier valide. Si la valeur testée est bien un entier, la méthode **isInteger()** renverra le booléen true. Dans le cas contraire, cette méthode renverra la booléen false.

N.B: si la valeur testée est NaN (Not a Number) ou l’infini, la méthode renverra également false.

```
<script>

let nb1 = 10;
let nb2 = 10.5

if(Number.isInteger(nb1)){
    alert('Le nombre ' + nb1 + ' est entier');
}

if(Number.isInteger(nb2)){
    alert('Le nombre ' + nb2 + ' est entier');
}

</script>
```

La méthode trim():

La méthode **trim()** supprime les espaces ou les « blancs » en début et en fin de chaîne. Cette méthode va s'avérer très pratique lorsqu'on voudra nettoyer des données pour ensuite effectuer des opérations dessus.

Cette méthode renvoie une nouvelle chaine de caractères sans blancs ni au début ni à la fin. Elle ne modifie pas la chaine de caractères de départ.

```
<script>

let phrase = '    Bonjour,je suis étudiant en première année dev 103    ';

alert(phrase.trim());

alert(phrase);

</script>
```

La méthode isNaN():

La méthode **isNaN()** permet de déterminer si la valeur passée en argument est la valeur NaN.On va lui passer en argument la valeur qui doit être comparée à NaN. Si la valeur passée est bien égale à NaN, notre méthode renverra le booléen true. Dans le cas contraire, le booléen false sera renvoyé.

```
<script>

let nb1 = 10;
let nb2 = NaN;

if(Number.isNaN(nb1)){
    alert('nb1 stocke la valeur NaN');
}

if(Number.isNaN(nb2)){
    alert('nb2 stocke la valeur NaN');
}

</script>
```

La méthode parseFloat():

La méthode **parseFloat()** permet de convertir une chaîne de caractères en un nombre décimal. On lui passe la chaine à transformer en argument et la méthode renverra un nombre décimal en retour.

L'analyse de la chaîne s'arrête dès qu'un caractère qui n'est pas +,-, un chiffre, un point ou un exposant est rencontré. Ce caractère et tous les suivants vont alors être ignorés. Si le premier caractère de la chaîne ne peut pas être converti en un nombre, parseFloat() renverra la valeur NaN

```
<script>

let nb1 = '29 Pierre';
let nb2 = '29.5 Pierre 30'
let nb3 = 'Pierre 29';

alert(Number.parseFloat(nb1));
alert(Number.parseFloat(nb2));
alert(Number.parseFloat(nb3));

</script>
```

La méthode parseInt():

La méthode **parseInt()** permet de convertir une chaine de caractères en un entier selon une base et va renvoyer ce nombre en base 10. On va lui passer deux arguments : la chaine de caractères à convertir et la base utilisée pour la conversion.

```
<script>

alert(Number.parseInt('0F', 16));
alert(Number.parseInt('FFF', 16));
alert(Number.parseInt('1000', 2));

</script>
```

La méthode toFixed():

La méthode **toFixed()** permet de formater un nombre en indiquant le nombre de décimales (nombre de chiffres après la virgule) qu'on souhaite conserver.

On va indiquer en argument de cette méthode le nombre de décimales souhaitées et notre méthode va renvoyer une chaine de caractères qui représente le nombre avec le nombre de décimales souhaitées.

```
<script>
let nb1 = 1234.450;

alert(nb1.toFixed(0));
alert(nb1.toFixed(1));
alert(nb1.toFixed(2));

</script>
```

La méthode toString():

La méthode **toString()** permet de transformer un nombre en une chaine de caractères. On va pouvoir lui passer une base en argument pour formater notre nombre. Elle renverra une chaine de caractères représentant notre nombre.

```
<script>

let nb1 = 255;

alert(nb1.toString(16));
alert(nb1.toString(2));
alert(typeof(nb1.toString(2)));

</script>
```

L’objet global JavaScript Math:

L’objet **Math** va être utilisé lorsqu’on aura besoin d’effectuer des opérations mathématiques. Il possède des méthodes qui permettent d’arrondir des nombres, de générer des nombres aléatoires ou encore de calculer le cosinus, sinus, tangente, logarithme ou l’exponentielle d’un nombre.

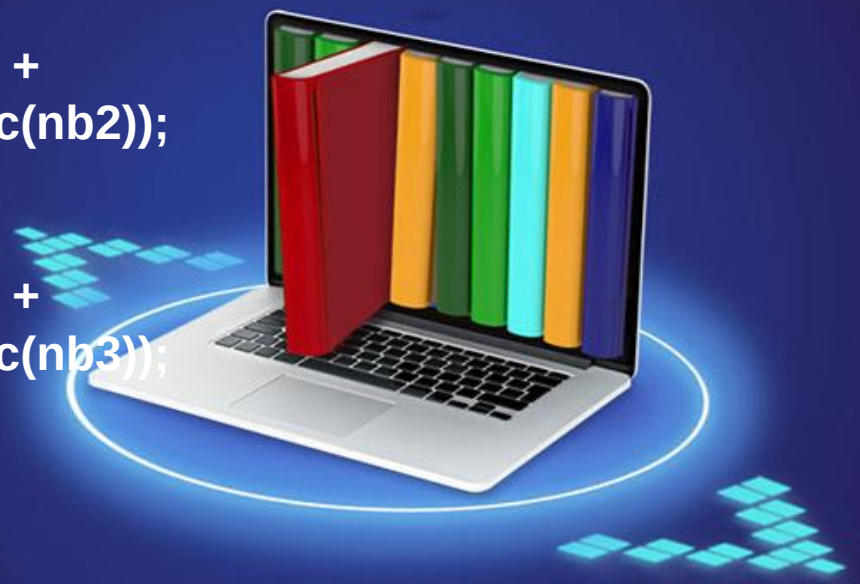
Les méthodes floor(), ceil(), round() et trunc():

- La méthode **floor()** va arrondir la valeur passée en argument à l’entier immédiatement inférieur (ou égal) à cette valeur.
- La méthode **ceil(),** au contraire, va arrondir la valeur passée en argument à l’entier immédiatement supérieur (ou égal) à cette valeur.

- La méthode **round()** va arrondir la valeur passée en argument à l’entier le plus proche. Ainsi, si la partie décimale de la valeur passée est supérieure à 0.5, la valeur sera arrondie à l’entier supérieur. Dans le cas contraire, la valeur sera arrondie à l’entier inférieur. Dans le cas où la partie décimale vaut exactement 0.5, la valeur sera arrondie à l’entier supérieur.

- La méthode **trunc()** va ignorer la partie décimale d’un nombre et ne retourner que sa partie entière.

```
<script>

let nb1 = 12.3456;
let nb2 = 2.45;
let nb3 = 2.54;

alert('Nombre : ' + nb1 +
    '<br>floor() : ' + Math.floor(nb1) + '<br>ceil() : ' + Math.ceil(nb1) +
    '<br>round() : ' + Math.round(nb1) + '<br>trunc() : ' + Math.trunc(nb1));

alert('Nombre : ' + nb2 +
    '<br>floor() : ' + Math.floor(nb2) + '<br>ceil() : ' + Math.ceil(nb2) +
    '<br>round() : ' + Math.round(nb2) + '<br>trunc() : ' + Math.trunc(nb2));

alert('Nombre : ' + nb3 +
    '<br>floor() : ' + Math.floor(nb3) + '<br>ceil() : ' + Math.ceil(nb3) +
    '<br>round() : ' + Math.round(nb3) + '<br>trunc() : ' + Math.trunc(nb3));

</script>
```



La méthode random():

La méthode **random()** permet de générer un nombre décimal compris entre 0 (inclus) et 1 (exclu) de manière aléatoire.

On va ensuite pouvoir multiplier son résultat par un autre nombre afin d'obtenir un nombre aléatoire compris dans l'intervalle de notre choix.

```
<script>

var min=1;
var max=10;
var random = Math.floor(Math.random() * (max – min+1)) + min;
alert(random);

</script>
```

Les méthodes min() et max():

La méthode **min()** renvoie le plus petit nombre d'une série de nombres passés en arguments.
La méthode **max(),** au contraire, va renvoyer le plus grand nombre d'une série de nombres passés en arguments.

Dans les deux cas, si l'une des valeurs fournies en argument n'est pas un nombre et ne peut pas être convertie en nombre, alors ces méthodes renverront la valeur NaN.

```
<script>

alert(Math.min(50, 2.5, 5, 14));

alert(Math.max(2, 75,-156));

</script>
```

La méthode abs():

La méthode **abs()** renvoie la valeur absolue d’un nombre, c’est-à-dire le nombre en question sans signe.

Si la valeur fournie en argument n’est pas un nombre et ne peut pas être convertie en nombre, alors elle renverra NaN

```
<script>

alert(Math.abs(4));
alert(Math.abs(-4));
alert(Math.abs('Pierre'));

</script>
```

Les méthodes cos(), sin(), tan(), acos(), asin() et atan():

Les méthodes **cos(), sin(), tan(), acos(), asin() et atan()** retournent respectivement le cosinus, le sinus, la tangente, l'arc cosinus, l'arc sinus et l'arc tangente d'une valeur passée en argument.

Les valeurs passées et retournées sont exprimées en radians. Pour convertir une valeur en radians en une valeur en degrés, il suffit de multiplier la valeur en radians par 180 et de diviser par pi (180° = pi radian, 360° = 2pi radian).

```
<script>

alert(Math.cos(0));
alert(Math.sin(0));

</script>
```

Les méthodes exp() et log():

Les méthodes exp() et log() renvoient respectivement l’exponentielle et le logarithme népérien (ou logarithme naturel) d’une valeur passée en argument.

```
<script>

alert(Math.exp(0));

alert(Math.log(1));

</script>
```

Méthodes	Description
abs()	Renvoie la valeur absolue du nombre donné.
acos(), asin(), atan()	Renvoie respectivement l'arc cosinus, l'arc sinus et l'arc tangente du nombre donné en radians.
ceil()	Renvoie une plus petite valeur entière, supérieure ou égale au nombre donné.
exp()	Renvoie la forme exponentielle du nombre donné.
floor()	Renvoie la plus grande valeur entière, inférieure ou égale au nombre donné.
log()	Renvoie le logarithme népérien d'un nombre.
max() , max()	Renvoie respectivement la valeur maximale et minimale des nombres donnés.
pow()	Renvoie la valeur de la base à la puissance de l'exposant.
random()	Renvoie un nombre aléatoire compris entre 0 (inclus) et 1 (exclusif).
round()	Renvoie la valeur entière la plus proche du nombre donné.
sin(), cos(), tan()	Renvoie respectivement le sinus, le cosinus et la tangente du nombre donné.
sqrt()	Il renvoie la racine carrée du nombre donné
trunc()	Il renvoie une partie entière du nombre donné.

Les tableaux en JavaScript et l’objet global Array:

Les variables de type **Array,** ou variables tableaux, sont des variables particulières qu’on retrouve dans de nombreux langages de programmation et qui permettent de stocker plusieurs valeurs à la fois avec un système de clef ou d’indice associé à chaque valeur. En JavaScript, les tableaux sont des objets qui dépendent de **l’objet global Array**

Présentation de tableaux en JavaScript:

Les tableaux sont des éléments qui vont pouvoir contenir plusieurs valeurs. Le principe des tableaux est relativement simple : un indice ou clef va être associé à chaque valeur du tableau. Pour récupérer une valeur dans le tableau, on va utiliser les indices qui sont chacun unique dans un tableau.

Dans la majorité des langages de programmation, on distingue deux types de tableaux : les tableaux dont les clefs ou indices sont des chiffres et qu’on appelle **tableaux numérotés** et les tableaux dont les clefs ou indices sont des chaines de caractères définies par le développeur et qu’on appelle **tableaux associatifs**.

Le JavaScript ne gère qu’un type de tableau : les tableaux numérotés. Les clefs numériques associées à chaque valeur vont être générées automatiquement. La première valeur d’un tableau va posséder la clef 0, la deuxième valeur possèdera la clef 1, et etc. On peut stocker n’importe quel type de valeur en valeurs d’un tableau

Création d'un tableau en JavaScript

Les tableaux ne sont pas des valeurs primitives. Cependant, on est pas obligé d'utiliser le **constructeur Array()** avec le mot clef **new** pour créer un tableau en JavaScript.

En effet, une syntaxe alternative et plus performante est disponible en JavaScript et permet des créer des tableaux qui vont tout de même pouvoir utiliser les propriétés et méthodes du constructeur Array().

Cette syntaxe utilise les crochets **[…]** :

```
let prenoms = ['Pierre', 'Mathilde', 'Florian', 'Camille'];
let ages = [29, 27, 29, 30];
let produits = ['Livre', 20, 'Ordinateur', 5, ['Magnets', 100]];
```

Accéder à une valeur dans un tableau

Lorsqu’on crée un tableau, un indice est automatiquement associé à chaque valeur du tableau. Chaque indice dans un tableau est toujours unique et permet d’identifier et d’accéder à la valeur qui lui est associée. Pour chaque tableau, l’indice 0 est automatiquement associé à la première valeur, l’indice 1 à la deuxième et etc.

Pour accéder à une valeur en particulier dans un tableau, il suffit de préciser le nom du tableau puis l’indice associé à la valeur à laquelle on souhaite accéder entre crochets.

Dans le cas où un tableau stocke un autre tableau, il faudra utiliser deux paires de crochets : la première paire va mentionner l’indice relatif à la valeur à laquelle on souhaite accéder dans notre tableau de base tandis que la deuxième parie de crochets va nous permettre de préciser l’indice lié à la valeur à laquelle on souhaite accéder dans notre sous tableau.

```
let prenoms = ['Pierre', 'Mathilde', 'Florian', 'Camille'];
let ages = [29, 27, 29, 30];
let produits = ['Livre', 20, 'Ordinateur', 5, ['Magnets', 100]];

alert(prenoms[0] + ' possède 1 ' + produits[2]);
alert(prenoms[1] + ' a ' + ages[1] + ' ans');
alert(produits[4][1] + ' ' + produits[4][0]);
```

Utiliser une boucle for…of pour parcourir toutes les valeurs d’un tableau:

Pour parcourir un tableau élément par élément, on utilise une boucle spécialement créée dans ce but qui est la boucle **for…of**.

```
let prenoms = ['Pierre', 'Mathilde', 'Florian', 'Camille'];
let ages = [29, 27, 29, 30];
let produits = ['Livre', 20, 'Ordinateur', 5, ['Magnets', 100]];

for(let valeur of prenoms){
   alert(valeur);
}
```

On définit une variable **let valeur** qui va stocker les différentes valeurs du tableau une à une. La boucle for… of va en effet exécuter son code en boucle jusqu’à ce qu’elle arrive à la fin du tableau.

Tableaux associatifs en JavaScript, objets littéraux et boucle for… in:

Dans d'autres langages informatiques (dont le PHP, par exemple), on peut créer des tableaux en choisissant d'attribuer une clef textuelle à chaque nouvelle valeur. On appelle **ces tableaux des tableaux associatifs**.

En JavaScript, ce type de tableau n'existe pas. La chose qui va le plus se rapprocher d'un tableau associatif en JavaScript est un objet. On utilise une boucle **for… in** pour parcourir les propriétés d'un objet une à une. La boucle for…in est l'équivalent de la boucle for…of mais pour les objets.

```
<script>

let pierre = {
    'prenom' : 'Pierre',
    'age' : 29,
    'sport' : 'trail',
    'cours' : ['HTML', ' CSS', ' JavaScript']
};

for(let propriete in pierre){
   alert(pierre[propriete]);
}
</script>
```

Les méthodes du constructeur Array():

Les méthodes push() et pop():

La méthode **push()** permet d'ajouter des éléments en fin de tableau et retourne la nouvelle taille du tableau.
La méthode **pop()** permet de supprimer le dernier élément d'un tableau et va retourner l'élément supprimé.

```
<script>

let prenoms = ['Pierre', 'Mathilde'];
let ages = [29, 27, 32];

/*On ajoute 2 éléments à "prenoms" et on récupère la nouvelle taille du tableau renvoyée par push() dans une variable "taille"*/
let taille = prenoms. push('Florian', 'Camille');

//On supprime le dernier élément de ages et on récupère l'élément supprimé
 dans del
let del = ages.pop();


alert(taille + ' éléments dans prenoms');
alert(del + '" supprimé de ages');

</script>
```

Les méthodes unshift() et shift()

La méthode **unshift()** permet d'ajouter des éléments en début de tableau et retourne la nouvelle taille du tableau.
La méthode **shift()** permet de supprimer le premier élément d'un tableau et retourne l'élément supprimé.

Ces deux méthodes sont les équivalentes des méthodes push() et pop() à la différence que les éléments vont être ajoutés ou supprimés en début de tableau et non pas en fin

```
<script>

let prenoms = ['Pierre', 'Mathilde'];
let ages = [29, 27, 32];

let taille = prenoms.unshift('Florian', 'Camille');

let del = ages.shift();

alert('Tableau prenoms : ' + prenoms);
alert(taille +' éléments dans prenoms');
alert('"' + del + '" supprimé de ages');

</script
```

La méthode **splice():** Pour ajouter, supprimer ou remplacer des éléments dans un tableau, on peut également utiliser splice(). L'avantage de cette méthode est qu'elle nous permet d'ajouter, de supprimer ou de remplacer des éléments n'importe où dans un tableau.

La méthode splice() prend trois arguments : une position de départ à partir d'où commencer le changement, le nombre d'éléments à remplacer et finalement les éléments à ajouter au tableau.

En précisant la position de départ 0, les changements seront effectués à partir du début du tableau. En précisant la position 1, ils se feront à partir du deuxième élément, etc. En précisant une position négative, les changements seront faits en comptant à partir de la fin : -1 pour commencer en partant du dernier élément, -2 pour commencer en partant de l'avant dernier élément, etc.

Si on précise 0 en nombre d'éléments à remplacer, alors aucun élément ne sera supprimé du tableau de base. Dans ce cas, il sera nécessaire de préciser des éléments à rajouter.

Enfin, si on ne précise pas d'éléments à rajouter au tableau, le nombre d'éléments à remplacer tel quel précisé en deuxième argument seront supprimés du tableau à partir de la position indiquée en premier argument.

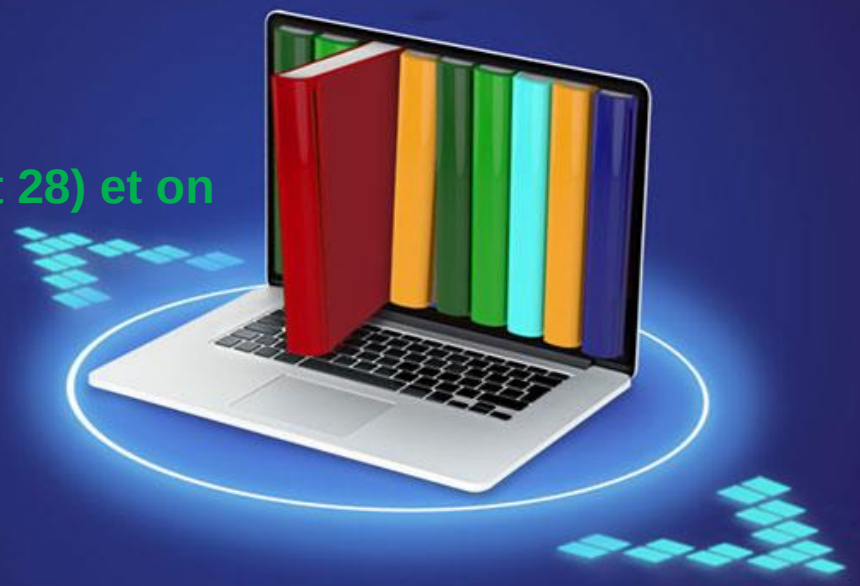
Cette méthode va également retourner un tableau contenant les éléments supprimés.

```
<script>

let prenoms = ['Pierre', 'Mathilde', 'Florian', 'Camille'];
let ages = [29, 27, 28, 30];

/*On insère 'Thomas' et 'Manon' dans le tableau prenoms, après le deuxième élément (Mathilde) et sans supprimer d'éléments*/

prenoms.splice(2, 0, 'Thomas', 'Manon');

/*On supprime les deux éléments après le premier ( c'est à dire 27 et 28) et on insère 35 après le premier élément (29)*/

let del = ages.splice(1, 2, 35);
alert(prenoms);
alert(ages);
alert('"' + del + '" supprimé de ages');

</script>
```

La méthode join()

La méthode join() retourne une chaine de caractères créée en concaténant les différentes valeurs d'un tableau. Le séparateur utilisé par défaut sera la virgule mais nous allons également pouvoir passer le séparateur de notre choix en argument de join().

```
let prenoms = ['Pierre', 'Mathilde', 'Florian', 'Camille'];
let ages = [29, 27, 28, 30];

/*On insère 'Thomas' et 'Manon' dans le tableau prenoms, après le deuxième élément (Mathilde) et sans supprimer d'éléments*/

prenoms.splice(2, 0, 'Thomas', 'Manon');

/*On supprime les deux éléments après le premier ( c'est à dire 27 et 28) et on
 insère 35 après le premier élément (29)*/

let del = ages.splice(1, 2, 35);

alert(prenoms.join(' - '));
alert(ages.join(' / '));
```



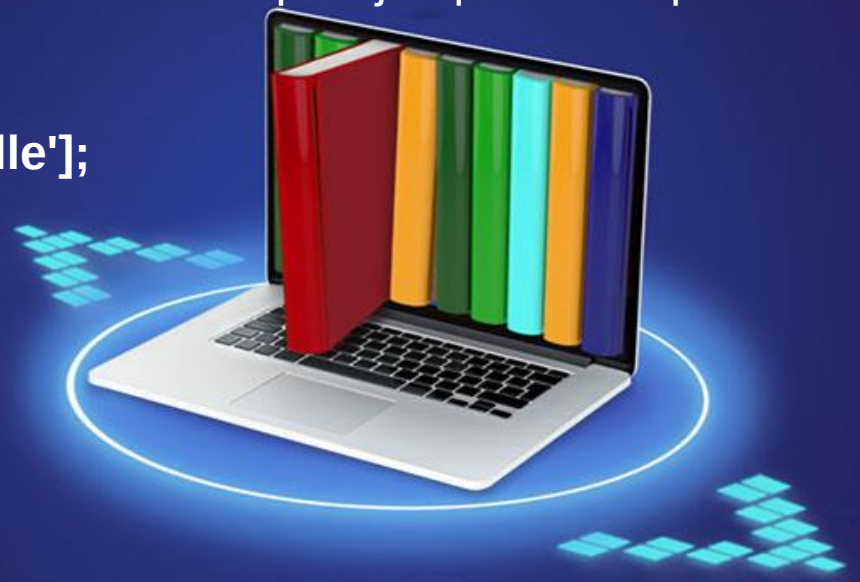
La méthode slice():

La méthode **slice()** renvoie un tableau créé en découpant un tableau de départ.

Cette méthode prend en premier argument facultatif la position de départ où doit commencer la découpe du tableau de départ. Si la position passée est un nombre négatif, alors le début de la découpe sera calculé à partir de la fin du tableau de départ. Si aucune position de départ n’est passée, la découpe commencera depuis le début du tableau de départ.

On peut lui passer en second argument facultatif la position où doit s’arrêter la découpe du tableau de départ. Si la position passée est un nombre négatif, alors la fin de la découpe sera calculé à partir de la fin du tableau de départ. Si aucune position de fin n’est passée, alors on récupèrera le tableau de départ jusqu’à la fin pour créer notre nouveau tableau.

```
let prenoms = ['Pierre', 'Mathilde', 'Thomas', 'Manon', 'Florian', 'Camille'];
let ages = [29, 27, 28, 30];

let sliceprenoms = prenoms.slice(2, 4);
let sliceages = ages.slice(2);

alert(sliceprenoms.join(' - '));
alert(sliceages.join(' / '));
```



La méthode includes()

La méthode **includes()** permet de déterminer si un tableau contient une valeur qu'on va passer en argument. Si c'est le cas, includes() renvoie true. Dans le cas contraire, cette méthode renvoie false.

Cette méthode est sensible à la casse (une majuscule est considérée comme une entité différente d'une minuscule).

```
let prenoms = ['Pierre', 'Mathilde', 'Thomas', 'Manon', 'Florian', 'Camille'];

if(prenoms.includes('Mathilde')){
    alert('Mathilde est dans le tableau‘);
}

if(prenoms.includes('mathilde')){
    alert('mathilde est dans le tableau’);
}
```

Exercice 1:

Écrivez un programme JavaScript simple pour joindre tous les éléments du tableau suivant dans une chaîne.

myColor = ["Red", "Green", "White", "Black"];

Sortie attendue :

"Rouge* Vert*Blanc*Noir"
"Rouge, Vert, Blanc, Noir"
"Rouge + Vert + Blanc + Noir"

Exercice 2:

Écrivez une fonction JavaScript pour supprimer un élément spécifique d'un tableau.
Données de test :

remove_array_element([2, 5, 9, 6], 5));

[2, 9, 6]

Exercice 3:

Soit le tableau suivant:

var semaine = ['lun', 'mra', 'mer', 'jeu', 'ven', 'sam', 'ddi'];

1. Retirer la dernière valeur du tableau jours
2. Afficher les valeurs du tableau
3. Ajouter la valeur ‘dim’ à la fin du tableau
4. Remplacer la valeur ‘mra’ par ‘mar’
5. Afficher le nombre de valeurs du tableau
6. Afficher la troisième valeur du tableau
7. Supprimez le premier élément du tableau puis afficher l'élément supprimé et le tableau mis à jour
8. Ajouter de nouveau la valeur ‘lun’ au tableau semaine
9. Vérifier si la valeur est bien ajoutée
10. Créer un sous tableau du tableau semaine contenant les 3 premiers jours de la semaine

Exercice 4:

Soit le tableau suivant:

var fruits = ['Mangue', 'Raisin', 'Figue', 'Kiwi'];

Écrire un programme qui:

1. Affiche la liste de fruits disponibles;
2. Demande au client quel fruit il désire acheter:
3. s’il est présent dans le tableau fruits: le retirer du tableau, et afficher ‘ok!’,
4. sinon, afficher ‘indisponible…’.
5. Affiche à nouveau la liste de fruits disponibles.

Exercice 5:

Écrire un script en JavaScript pour réaliser un petit jeu « Deviner un nombre ».
Le principe de ce jeu est le suivant :

1. L’ordinateur choisit un nombre aléatoire entre 10 et 100.
2. Le but est de deviner ce nombre, qui sera saisit dans une zone de texte.
3. À chaque fois qu’on propose un nombre, on indique si le nombre à deviner est plus petit ou plus grand que celui-ci.
4. Le nombre d’essai autorisé est de 10, l’utilisateur est averti pour chaque essai concernant le nombre d’essai restant

Trier un tableau:

- La méthode **sort()** retourne les éléments par ordre alphabétique (elle doivent être de la même nature) :

```
<script>
let tableau = ['E', 'A', 'D'];
alert(tableau.sort());

</script>
```

- La méthode **reverse()** Inverse l'ordre des éléments (sans tri) :

```
<script>

let tableau = ['A', 'B', 'C'];
alert(tableau.reverse());

</script>
```

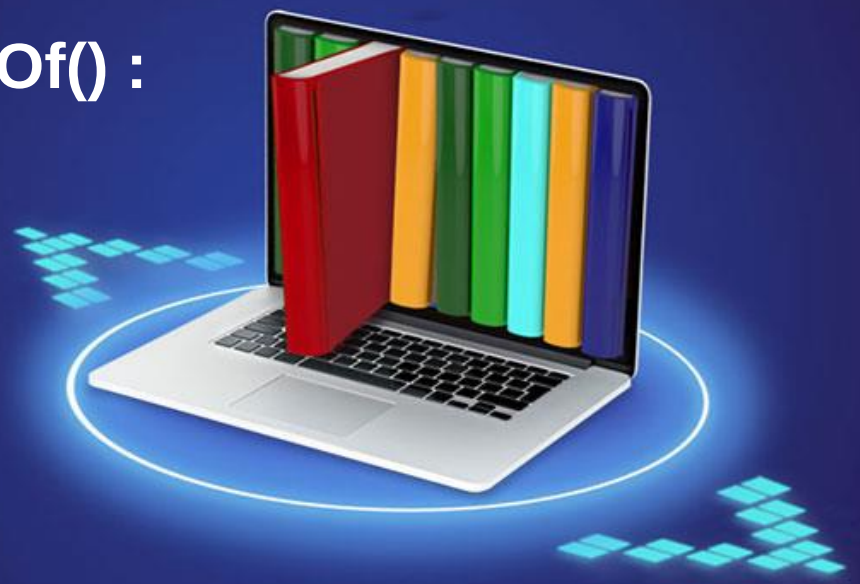
Recherche un élément dans le tableau:

La méthode **findIndex()** retourne l’indice du premier élément du tableau qui remplit une condition :

```
<script>
function findC(element) {
return element === 'C';
}
let tableau = ['A', 'B', 'C', 'D', 'C'];
tableau.findIndex(findC); // Retourne 2, l'indice du premier C
</script>
```

Récupérer l’indice d’un élément : la méthode indexOf() :

```
<script>
let tableau = ['A', 'B', 'C'];
alert(tableau.indexOf('C')); // Retourne 2
</script>
```



indexOf() a un deuxième paramètre optionnel qui permet aussi de choisir l'indice à partir duquel la recherche débute (Par défaut, ce deuxième paramètre est à 0) :

```
<script>

let tableau = ['A', 'B', 'C', 'B'];

alert(tableau.indexOf('B')); // Retourne 1, l'indice du premier B trouvé

alert(tableau.indexOf('B', 2)); // Retourne 3, l'indice du premier B trouvé après l'indice 2

</script>
```

Les dates en JavaScript et l'objet global Date

Le constructeur **Date()** possède en effet de nombreuses méthodes qui vont nous permettre d'obtenir ou de définir une date. Le JavaScript stocke les dates en interne sous forme d'un **Timestamp** exprimé en millisecondes (c'est-à-dire le Timestamp Unix multiplié par 1000).

Le **Timestamp** correspond au nombre de secondes écoulées depuis le premier janvier 1970 à minuit UTC ou GMT (c'est-à-dire minuit selon l'heure de Londres).

Récupérer la date actuelle:

Pour récupérer la date actuelle, on va utiliser Date().

Pour afficher cette même date sous forme de nombre (le nombre de millisecondes écoulées depuis le 1er janvier 1970), on peut utiliser la méthode statique now() du constructeur Date.

```
let date1 = Date();
let date2 = Date.now();

alert(date1);
alert(date2);
```

Créer un objet de type date

Pour créer et manipuler une date particulière, on doit **créer un objet de type date** en utilisant le constructeur **Date()** avec donc le mot clef **new**.

- On instancie **Date()** sans lui passer d'argument. Le constructeur va créer un objet date contenant la date actuelle complète en fonction de l'heure locale du système.
- On peut passer une suite de nombres à notre constructeur pour qu'il crée un objet date morceau par morceau. Ces nombres représentent, dans l'ordre :

- L'année (argument obligatoire) de la date qu'on souhaite créer ;
- Le numéro du mois (argument obligatoire) de la date , entre 0 (pour janvier) et 11 (pour décembre) ;
- Le numéro du jour du mois (facultatif) de la date, entre 1 et 31 ;
- L'heure (facultatif) de la date, entre 0 et 23 ;
- Le nombre de minutes (facultatif) de la date , entre 0 et 59 ;
- Le nombre de secondes (facultatif) de la date, entre 0 et 59 ;
- Le nombre de millisecondes (facultatif) de la date , entre 0 et 999.

```
let date1 = new Date();
let date2 = new Date('March 23, 2019 20:00:00');
let date3 = new Date(1553466000000);
let date4 = new Date(2019, 0, 25, 12, 30);

alert('Date 1 : ' + date1 + '<br>Date 2 : ' + date2 +'<br>Date 3 : ' + date3 + '<br>Date4 : ' + date4;)
```

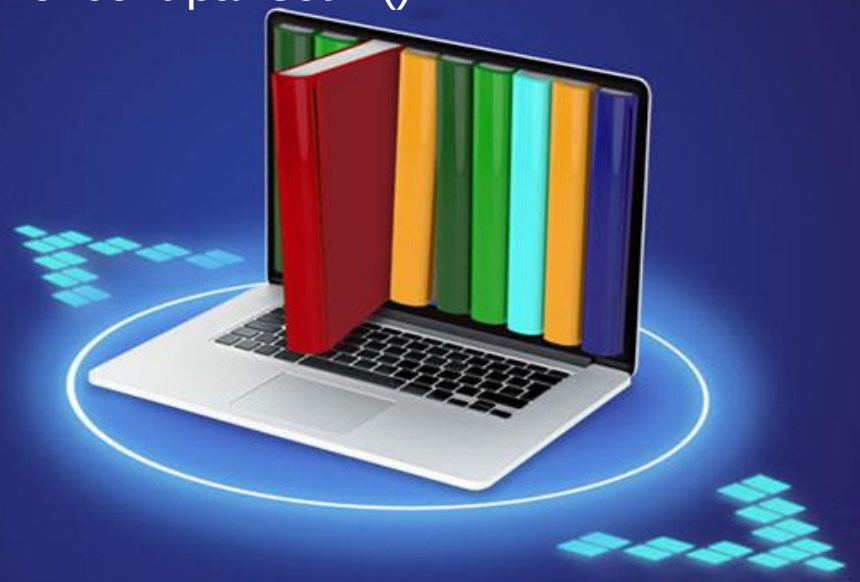
Les méthodes getters et setters de l’objet Date

L’objet Date possède de nombreuses méthodes qu’on peut classer en différents groupes : les méthodes qui permettent d’obtenir une date, celles qui permettent de définir une date et celles qui permettent de formater une date.

De manière générale en programmation on peut noter qu’on appelle les méthodes qui permettent d’obtenir / de récupérer quelque chose des « getters » (get signifie avoir, posséder en anglais). Ces méthodes sont souvent reconnaissables par le fait qu’elles commencent par **get…().**

De même, on appelle les méthodes qui permettent de définir quelque chose des « setters » (set signifie définir en anglais). Ces méthodes vont être reconnaissables par le fait qu’elles commencent par set…().

N.B: le système de « getters » et de « setters » n’est pas propre à l’objet Date en JavaScript mais est une convention partagée par de nombreux langages de programmation.



Les getters de l'objet Date:

L'objet Date possède différentes méthodes getters qui permettent de récupérer un composant d'une date (année, mois, jour, heure, etc.).

Les getters suivants vont renvoyer un composant de date selon l'heure locale :

- **getDay()** renvoie le jour de la semaine sous forme de chiffre (avec 0 pour dimanche, 1 pour lundi et 6 pour samedi) pour la date spécifiée selon l'heure locale ;
- **getDate()** renvoie le jour du mois en chiffres pour la date spécifiée selon l'heure locale ;
- **getMonth()** renvoie le numéro du mois de l'année (avec 0 pour janvier, 1 pour février…) pour la date spécifiée selon l'heure locale ;
- **getFullYear()** renvoie l'année en 4 chiffres pour la date spécifiée ;
- **getHours()** renvoie l'heure en chiffres pour la date spécifiée;
- **getMinutes()** renvoie les minutes en chiffres pour la date spécifiée;
- **getSeconds()** renvoie les secondes en chiffres pour la date spécifiée;
- **getMilliseconds()** renvoie les millisecondes en chiffres pour la date spécifiée.

```
<script>

let date1 = new Date();

let jourSemaine = date1.getDay();
let jourMois = date1.getDate();
let mois = date1.getMonth();
let annee = date1.getFullYear();
let heures = date1.getHours();
let heuresUTC = date1.getUTCHours();
let minutes = date1.getMinutes();
let secondes = date1.getSeconds();
let ms = date1.getMilliseconds();

alert(
    'Date : ' + date1 +'<br>Jour de la semaine : ' + jourSemaine +
    '<br>Jour du mois : ' + jourMois + '<br>Numéro du mois : ' + mois +
    '<br>Année : ' + annee +'<br>Heures : ' + heures + ' (heure UTC : ' +
heuresUTC + ')' +
    '<br>Minutes : ' + minutes +'<br>Secondes : ' + secondes +
    '<br>Millisecondes : ' + ms);
</script>
```

L’objet Date fournit également des getters qui renvoient un composant de date selon l’heure UTC (l'heure universelle) et qui sont les suivants :

- **getUTCDay()** renvoie le jour de la semaine sous forme de chiffre (avec 0 pour dimanche, 1 pour lundi et 6 pour samedi) pour la date spécifiée selon l’heure UTC ;
- **getUTCDate()** renvoie le jour du mois en chiffres pour la date spécifiée selon l’heure UTC ;
- **getUTCMonth()** renvoie le numéro du mois de l’année (avec 0 pour janvier, 1 pour février, 11 pour décembre) pour la date spécifiée selon l’heure UTC ;
- **getUTCFullYear()** renvoie l’année en 4 chiffres pour la date spécifiée selon l’heure UTC ;
- **getUTCHours()** renvoie l’heure en chiffres pour la date spécifiée;
- **getUTCMinutes()** renvoie les minutes en chiffres pour la date spécifiée;
- **getUTCSeconds()** renvoie les secondes en chiffres pour la date spécifiée;
- **getUTCMilliseconds()** renvoie les millisecondes en chiffres pour la date spécifiée.

Les setters de l’objet Date

Les setters de l’objet Date permettent de définir (ou de modifier) des composants de dates pour une date donnée. Ces setters vont correspondre exactement aux getters vus précédemment (de manière générale, en programmation, les getters et les setters marchent par paires).

- **setDate() et setUTCDate()** définissent le jour du mois en chiffres pour la date spécifiée selon l’heure locale ou l’heure UTC ;
- **setMonth() et setUTCMonth()** définissent le numéro du mois de l’année (avec 0 pour janvier, 1 pour février, 11 pour décembre) pour la date spécifiée selon l’heure locale ou l’heure UTC ;
- **setFullYear() et setUTCFullYear()** définissent l’année en 4 chiffres pour la date spécifiée selon l’heure locale ou l’heure UTC ;
- **setHours() et setUTCHours ()** définissent l’heure en chiffres pour la date spécifiée selon l’heure locale ou l’heure UTC ;
- **setMinutes() et setUTCMinutes()** définissent les minutes en chiffres pour la date spécifiée selon l’heure locale ou l’heure UTC ;
- **setSeconds() et setUTCSeconds()** définissent les secondes en chiffres pour la date spécifiée selon l’heure locale ou l’heure UTC ;
- **setMilliseconds() et setUTCMilliseconds()** définissent les millisecondes en chiffres pour la date spécifiée selon l’heure locale ou l’heure UTC.

```
<script>

let date1 = new Date(2023, 0, 25, 12, 30, 15);

//On modifie la date
date1.setDate(31);
date1.setMonth(2);
date1.setFullYear(2024);
date1.setHours(10);
date1.getUTCHours();
date1.setMinutes(0);
date1.setSeconds(0);
date1.setMilliseconds(0);

alert('Date : ' + date1);

</script>
```

Convertir une date au format local

L’objet Date dispose d’autres méthodes permettant de convertir un format de date en un format local.

Ces méthodes permettent d’afficher les éléments de date dans un ordre local (jour puis mois puis année) ou de pouvoir afficher une heure selon un format 12 heures ou 24 heures. Elles vont permettent de traduire des dates littérales anglaises dans la langue locale.

Ces trois méthodes prennent deux arguments : un premier argument qui est une locale et un second qui correspond à des options.

- La locale sert à définir la langue dans laquelle la date doit être formatée. Pour la France, on utilisera fr-FR.
- Les options vont permettre de modifier le comportement par défaut des méthodes et notamment d'expliciter si on souhaite que la date passée soit renvoyée sous forme de chiffres ou sous forme littérale.

Les options sont:

- ✓ **weekday** représente le jour de la semaine. Les valeurs possibles sont « narrow », « short » et « long » ;
- ✓ **day** représente le jour du mois. Les valeurs possibles sont numeric et 2-digit ;
- ✓ **month** qui représente le mois. Les valeurs possibles sont numeric et 2-digit
- ✓ **year** qui représente l'année. Les valeurs possibles sont numeric et 2-digit
- ✓ **hour** qui représente l'heure.
- ✓ **minute** qui représente les minutes.
- ✓ **second** qui représente les secondes.

- La méthode **toLocaleDateString()** renvoie la partie « jour-mois-année » d'une date, formatée en fonction d'une locale et d'options.

```
<script>

let date1 = new Date();
let dateLocale = date1. toLocaleDateString() ;
alert(dateLocale );

</script>
```

- La méthode **toLocaleTimeString()** renvoie la partie « heures-minutes-secondes » d'une date, formatée en fonction d'une locale et d'options.

```
<script>

let date1 = new Date();
let dateLocale = date1. toLocaleTimeString() ;
alert(dateLocale );

</script>
```

- La méthode **toLocaleString()** renvoie la date complète, formatée en fonction d'une locale et d'options.

```
<script>

let date1 = new Date();
let dateLocale = date1. toLocaleString() ;
alert(dateLocale );

</script>
```

N.B: A noter que les navigateurs sont obligés de supporter à minima les ensembles suivants. Si une autre combinaison est utilisée, celle-ci pourra ne pas être supportée :

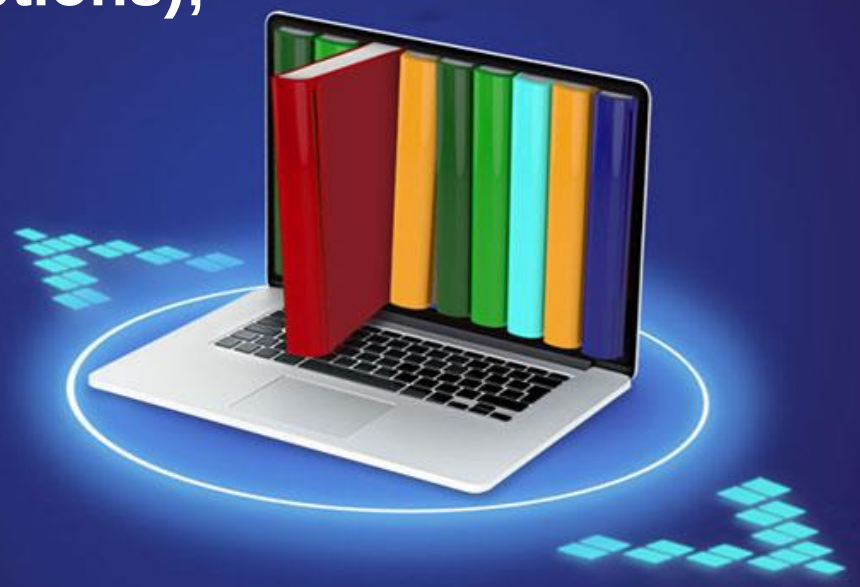
- **weekday, year, month, day, hour, minute, second ;**
- **weekday, year, month, day ;**
- **year, month, day ;**
- **year, month ;**
- **month, day ;**
- **hour, minute, second ;**
- **hour, minute.**

```
<script>

let date1 = new Date();

let dateLocale = date1.toLocaleString('en-EN',
{
    weekday: 'narrow',
    year: '2-digit',
    month: 'long',
    day: '2-digit',
    hour: '2-digit',
    minute: 'numeric',
    second: 'numeric'});

alert('Date : ' + dateLocale);

</script>
```

Remarque:

Une autre option intéressante est l’option **timeZone** qui permet de définir le fuseau horaire à utiliser.

```
<script>

let date = new Date();
let options = {timeZone: 'America/New_York'};
let eastCoastTime = date.toLocaleString('en-US', options);
alert (eastCoastTime);

</script>
```



N.B:

getTimezoneOffset() renvoie la différence entre l'heure UTC et l'heure locale. getTimezoneOffset() renvoie la différence en minutes. Par exemple, si le fuseau horaire est GMT+2, -120 sera renvoyé.

```
<script>

let date1 = new Date();
let dateLocale = date1.toLocaleTimeString();
let zoneLocale = date1.getTimezoneOffset();
alert('Date : ' + dateLocale);
alert('Zone : ' + zoneLocale);

</script>
```

Exercice 1:

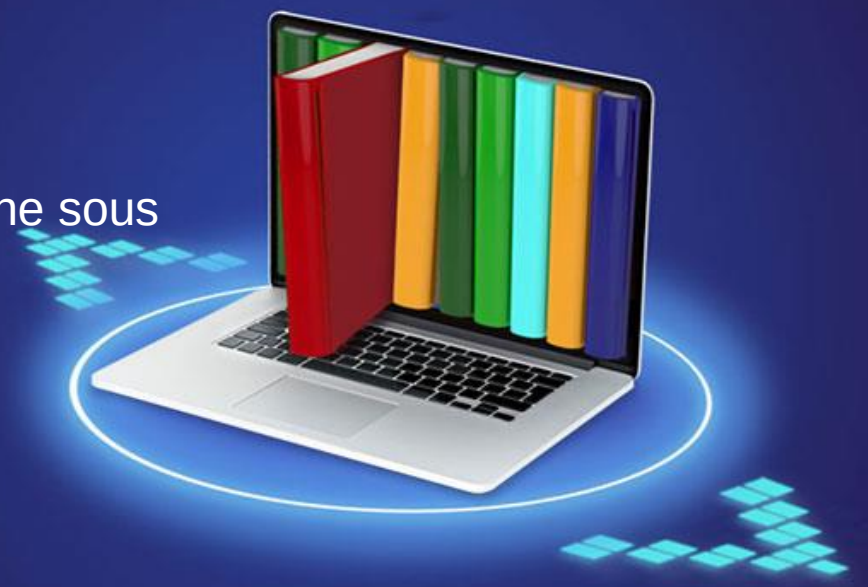
1. Ecrivez une fonction permettant l’affichage de la date complète. On ne s’occupera pas du format d’affichage, on se contentera d’un "simple" affichage de l’objet Date.

2. Testez l’utilisation de la méthode **getFullYear()** . On récupèrera l’année dans une variable que l’on fera afficher dans une page web.

3. Écrivez une fonction permettant l’affichage de la date sous la forme "jour mois année", par exemple : "Nous sommes 28 Février 2024, il est 8 h 15 min. 36 sec.

Exercice 2:

Ecrivez une fonction **getWeekDay(date)** pour afficher le jour de la semaine sous forme abrégée: ‘MO’, ‘TU’, ‘WE’, ‘TH’, ‘FR’, ‘SA’, ‘SU’.



Par exemple:

```
let date = new Date(2024, 0, 3);  // 3 Janvier 2024
alert( getWeekDay(date) );        // devrait afficher "TU"
```

Exercice 3:

1. Demander l'âge actuel de l'utilisateur et un âge visé
2. Afficher le nombre de jours vécus et l'écart avec l'âge visé

Exemple avec 21 et 45 : "Vous avez vécu 7665 jours. Il y a encore 8760 jours avant vos 45 ans.«

N.B : si l'âge visé est plus petit que l'âge actuel, signaler l'erreur (alert ou affichage dans la page)

Manipulation du BOM en JavaScript

JavaScript API, Browser Object Model et interface Window:

Définition et présentation des API JavaScript:

Une **API** (Interface de Programmation Applicative) est une interface, c'est-à-dire un ensemble de codes grâce à laquelle un logiciel fournit des services à des clients.

Le principe et l'intérêt principal d'une API est de permettre à des personnes externes de pouvoir réaliser des opérations complexes.

En effet, le développeur n'aura pas besoin de connaître les détails de la logique interne du logiciel tiers et n'y aura d'ailleurs pas accès directement puisqu'il devra justement passer par l'API qui va fournir un ensemble d'objets et donc de propriétés et de méthodes prêtes à l'emploi et en permettant de réaliser des opérations complexes.

Une API peut être comparée à une commande de voiture (pédale d'accélération, essuie-glace, etc.) : lorsqu'on accélère ou qu'on utilise les essuies glace, on ne va pas se préoccuper de comment la voiture fait pour effectivement avancer ou comment les essuies glace fonctionnent. On va simplement se contenter d'utiliser les commandes (l'API) qui vont cacher la complexité des opérations derrière et nous permettre de faire fonctionner la voiture.

Les API JavaScript vont pouvoir être classées dans deux grandes catégories :

- Les API intégrées aux navigateurs web et qu'on va donc pouvoir utiliser immédiatement pour du développement web comme **l'API DOM (Document Object Model)** qui va permettre de manipuler le HTML et le CSS d'une page, **l'API Geolocation** qui va permettre de définir des données de géolocalisation ou encore **l'API Canvas** qui permet de dessiner et de manipuler des graphiques dans une page ;

- Les API externes, proposées par certains logiciels ou sites comme la suite **d'API Google Map** qui permettent d'intégrer et de manipuler des cartes dans nos pages web ou encore **l'API Twitter** qui permet d'afficher une liste de tweets sur un site par exemple ou bien **l'API YouTube** qui permet d'intégrer des vidéos sur un site.

Introduction au Browser Object Model (BOM) et à l'objet Window

Le **BOM est une sorte de « super API »** elle-même composée de plusieurs API dont certaines sont elles mêmes composées de plusieurs API et etc.

A la base du BOM, on a l'interface **Window** qui représente une fenêtre de navigateur contenant une page ou un document. Dans un navigateur utilisant des onglets, comme Firefox, chaque onglet contient son propre objet Window.

Les objets suivants appartiennent au BOM et sont tous des enfants de Window :

- **L'objet Navigator** qui représente l'état et l'identité du navigateur et qu'on va utiliser avec l'API Geolocation ;
- **L'objet History** qui permet de manipuler l'historique de navigation du navigateur
- **L'objet Location** qui fournit des informations relatives à l'URL de la page courante ;
- **L'objet Screen** qui permet d'examiner les propriétés de l'écran qui affiche la fenêtre courante ;
- **L'objet Document** et le DOM

Propriétés, méthodes et fonctionnement de l'objet Window

L'objet **Window** représente la fenêtre du navigateur actuellement ouverte. Pour les navigateurs qui supportent les onglets, chaque onglet va posséder son propre objet **Window**.

Les propriétés de l'objet Window: L'objet **Window** possède de nombreuses propriétés dont notamment des propriétés **document**, **navigator**, **location** .

- Les propriétés **outerHeight et outerWidth** vont retourner la hauteur et la largeur de la fenêtre du navigateur.

- Les propriétés **innerHeight et innerWidth** vont retourner la hauteur et la largeur de la partie visible de la fenêtre de navigation (la partie dans laquelle le code est rendu).

```
<script>
alert('Taille de la fenêtre (ext) : ' + window.outerWidth + '*' +
window.outerHeight);
alert('Taille de la fenêtre (int) : ' + window.innerWidth + '*' +
window.innerHeight);
</script>
```

Les méthodes de Window

Ouvrir, fermer, redimensionner ou déplacer une fenêtre:

La méthode **open()** permet d'ouvrir une certaine ressource dans une fenêtre, un onglet ou au sein d'un élément iframe. On va passer en argument de cette méthode une URL qui correspond à la ressource à charger.

On peut passer à **open()** une suite d'arguments précisant les fonctionnalités de la nouvelle fenêtre, notamment sa position (avec **left et top**), sa taille (avec **outerWidth et outerHeight**).

Exemple:

```
<html>
   <head>
      <title>Cours JavaScript</title>
      <meta charset="utf-8">
       <script src='codej.js' async></script>
   </head>

   <body>
      <h1>Titre principal</h1>
      <p>Un paragraphe</p>
      <button id='b1'>OUVRIR</button>

   </body>
</html>
```

```
let b1 = document.getElementById('b1'); //On accède au bouton #b1
let winSize = 'width=500, height=500';
 //exécuter openWindow() dès qu'un utilisateur clique sur #b1
b1.addEventListener('click', openWindow);

//On définit notre fonction openWindow()
function openWindow(){
    fenetre = window.open('https://www.google.com', '', winSize);
}
```

On utilise dans le code ci-dessus la gestion des évènements avec **addEventListener().** On commence par récupérer une référence à notre élément HTML **button id='b1'** pour pouvoir le manipuler en JavaScript. Ensuite, on attache un gestionnaire d'évènement click à ce bouton.

L'idée est la suivante : la fonction **openWindow()** sera exécutée dès qu'une personne clique sur button id='b1'. Finalement, on définit **openWindow()** qui contient la méthode **open().**
On va passer à open() une URL qui devra être chargée dans la nouvelle fenêtre ainsi que la taille de la fenêtre.

Une fois une nouvelle fenêtre créée, on va pouvoir la redimensionner en ajoutant ou en enlevant à sa taille actuelle un certain nombre de pixels grâce à la méthode **resizeBy()** ou en lui passant une nouvelle taille avec **resizeTo().**

Chacune de ces deux méthodes va prendre deux arguments : une largeur et une hauteur et elles devront être appliquées sur la référence renvoyée par open().

En effet, la plupart des navigateurs dont Chrome bloquent ce genre de requêtes pour des raisons de sécurité. Pour tester le résultat de ces méthodes, on ne pas renseigner d'URL dans la méthode open().

Exemple:

```
<button id='b2'>window.resizeBy</button>
<button id='b3'>window.resizeto</button>
```

HTML

```
let b2 = document.getElementById('b2');
let b3 = document.getElementById('b3');
b2.addEventListener('click', resizeWindowBy);
b3.addEventListener('click', resizeWindowTo);

function resizeWindowBy(){
   fenetre.resizeBy(800, 500);
}
function resizeWindowTo(){
   fenetre.resizeTo(960, 720);
}
```

JS

Méthodes moveTo() et moveBy():

On peut déplacer la fenêtre sur un espace de travail avec les méthodes **moveBy()** qui va déplacer la fenêtre relativement à sa position de départ et **moveTo()** qui va la déplacer de manière absolue, par rapport à l'angle supérieur gauche de l'espace de travail.

Ces deux méthodes vont à nouveau prendre deux arguments qui correspondent au déplacement horizontal et vertical de la fenêtre.

Code HTML:

```
<button id='b1'>OUVRIR</button>
<button id='b2'>window.resizeBy</button>
<button id='b3'>window.resizeto</button>
<button id='b4'>window.moveBy</button>
<button id='b5'>window.moveTo</button>
```

```
let b1 = document.getElementById('b1');
let b2 = document.getElementById('b2');
let b3 = document.getElementById('b3');
let b4 = document.getElementById('b4');
let b5 = document.getElementById('b5');
let winSize = 'width=500, height=500';
```

suite →

```
b1.addEventListener('click', openWindow);
b2.addEventListener('click', resizeWindowBy);
b3.addEventListener('click', resizeWindowTo);
b4.addEventListener('click', moveWindowBy);
b5.addEventListener('click', moveWindowTo);
```

```
function openWindow(){
    //On recupère l'information renvoyée par open() dans une variable "fenetre"
    fenetre = window.open('', '', winSize);
}
function resizeWindowBy(){
    fenetre.resizeBy(800, 500);
}
function resizeWindowTo(){
    fenetre.resizeTo(560, 420);
}
function moveWindowBy(){
    fenetre.moveBy(500, 500); //Déplace la fenêtre 500px à droite et 500px en bas
}
function moveWindowTo(){
    fenetre.moveTo(0, 0); //Place la fenêtre contre le bord supérieur gauche
}
```

Méthodes close():

Pour pouvoir fermer une fenêtre avec la méthode close().

```
<button id='b6'>window.close</button>
```

HTML

```
let b6 = document.getElementById('b6');
b6.addEventListener('click', closeWindow);

function closeWindow()
{
 fenetre.close();
}
```

JS

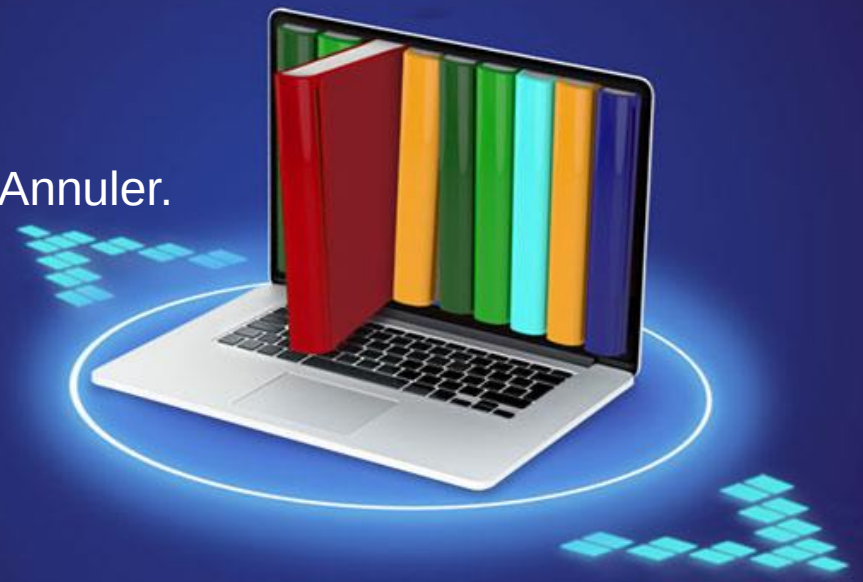
Rappel:

L’objet **Window** possède également des méthodes qui permettent d’afficher des boites d’alerte, de dialogue ou de confirmation dans la fenêtre.

Nous connaissons déjà bien les fonctions alert() et prompt() qui sont en fait des méthodes de l’objet Window. Comme ces deux méthodes sont très utilisées, et comme l’objet Window est implicite, nous les utiliserons généralement sans préciser window. avant la méthode.

Pour rappel, la méthode alert() permet d’afficher une boite d’alerte tandis que prompt() affiche une boite de dialogue permettant aux utilisateurs de nous envoyer du texte.

La méthode confirm(), quant-à-elle, ouvre une boite avec un message (facultatif) et deux boutons pour l’utilisateur : un bouton Ok et un bouton Annuler.



L’interface et l’objet Navigator

Présentation de l’objet Navigator:

L’objet Navigator permet de donner des informations sur le navigateur de vos visiteurs en soi ainsi que sur les préférences enregistrées (langue, etc.).

Les propriétés et méthodes de Navigator:

Les propriétés les plus intéressantes vont sont les suivantes :

- **language** : retourne la langue définie dans le navigateur ;
- **geolocation** : retourne un objet Geolocation qui peut être utilisé pour définir la localisation de l’utilisateur ;
- **cookieEnabled** : détermine si les cookies sont autorisés ou non et retourne true ou false ;
- **platform** : retourne la plateforme utilisée par le navigateur.

```
<html>
   <head><title></title></head>
<body>
   <h1>Titre principal</h1>
   <p>Un paragraphe</p>
   <p id='p1'></p>
   <p id='p2'></p>
   <p id='p3'></p>

<script>

    document.getElementById('p1').innerHTML =
   'Langue du navigateur : ' + navigator.language  +
   '<br>Cookies autorisés : ' + navigator.cookieEnabled +
   '<br>Plateforme utilisée : ' + navigator.platform;

</script>
</body>
</html>
```

L’interface et l’objet History en JavaScript

Présentation de l’objet History: Permet de manipuler l’historique du navigateur des visiteurs pour la session courante et de charger par exemple la page précédente.Lorsqu’on parle « d’historique » ici on parle de la liste des pages visitées au sein de l’onglet ou fenêtre dans laquelle la page actuelle est ouverte.

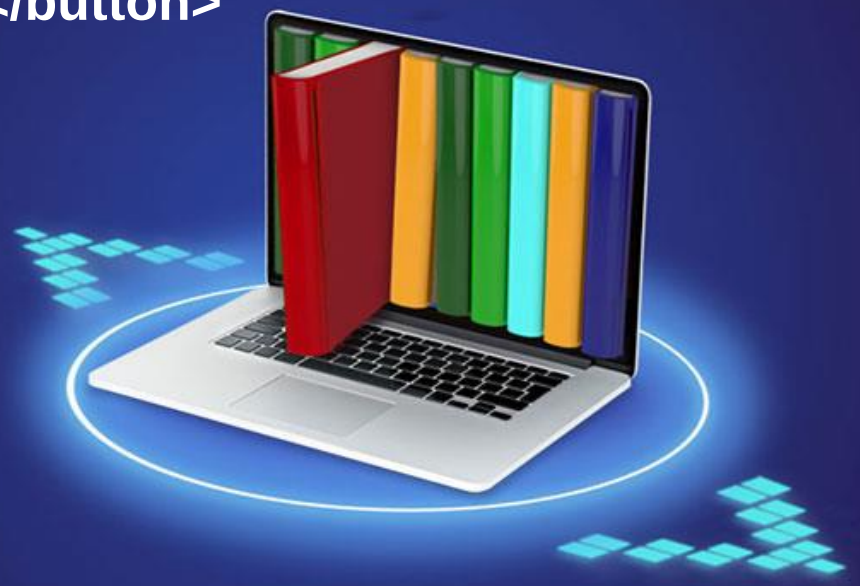
Les propriétés et méthodes de History:L’interface **History** implémente plusieurs propriétés et méthodes qu’on va pouvoir utiliser à partir d’un objet History.

- **La propriété length:** retourne le nombre d’éléments dans l’historique, c’est-à-dire le nombre d’URL parcourues durant la session ;
- **La méthode go():** permet de charger une page depuis l’historique de session. On va lui passer un nombre en argument qui représente la place de la page qu’on souhaite atteindre dans l’historique par rapport à la page actuelle (-1 pour la page précédente et 1 pour la page suivante par exemple) ;
- **La méthode back():** permet de charger la page précédente dans l’historique de session par rapport à la page actuelle. Utiliser back() est équivalent à utiliser go(-1) ;
- **La méthode forward() :** permet de charger la page suivante dans l’historique de session par rapport à la page actuelle. Utiliser forward() est équivalent à utiliser go(1).

Exemple:

Code HTML:

```
<body>
    <h1>Titre principal</h1>
    <p>Un paragraphe</p>
    <p id='p1'></p>
    <button id='b1'>Deux pages en arrières dans l'historique</button>
    <button id='b2'>Vers la page précédente dans l'historique</button>
    <button id='b3'>Vers la page suivante dans l'historique</button>
</body>
```



Code JS:

```
document.getElementById('p1').innerHTML= history.length + ' pages visitées';

//On accède aux boutons b1, b2 et b3
let b1 = document.getElementById('b1');
let b2 = document.getElementById('b2');
let b3 = document.getElementById('b3');

//On définit des gestionnaires d'évènement click pour ces boutons
b1.addEventListener('click', hgo);
b2.addEventListener('click', hback);
b3.addEventListener('click', hforward);

function hgo(){
    history.go(-2);
}
function hback(){
    history.back();
}
function hforward(){
    history.forward();
}
```

L’interface et l’objet Location en JavaScript

Présentation de l’objet Location: Permet de fournir des informations relatives à l’URL (c’est-à-dire la localisation) d’une page.

On peut accéder à Location à partir des interfaces Window ou Document, en utilisant leur propriété location respective c’est-à-dire Window.location et Document.location.

L’objet **Location** donne accès à 3 méthodes intéressantes :

- La méthode **assign()** qui va charger une ressource à partir d’une URL passée en argument ;
- La méthode **reload()** qui va recharger le document actuel ;
- La méthode **replace()** qui va remplacer le document actuel par un autre disponible à l’URL fournie en argument.

La différence fondamentale entre les méthodes **assign() et replace()** est qu’en utilisant assign(), on peut revenir en arrière pour revenir sur notre page de départ car celle-ci a été sauvegardée dans l’historique de navigation ce qui n’est pas le cas si on choisit d’utiliser replace().

```
//On accède aux boutons b1, b2 et b3

let b1 = document.getElementById('b1');
let b2 = document.getElementById('b2');
let b3 = document.getElementById('b3');

//On définit des gestionnaires d'évènement click pour ces boutons
 b1.addEventListener('click', recharge);
b2.addEventListener('click', assigne);
b3.addEventListener('click', remplace);
function recharge()
{ location.reload(); }
function assigne(){
location.assign('https://www.google.com'); }
function remplace(){
location.replace('https://www.google.com'); }
```

Exemple:

```
<body>
<h1>Titre principal</h1>
<p>Un paragraphe</p>
<button id='b1'>Recharger la page</button>
<button id='b2'>Charger un nouveau document</button>
 <button id='b3'>Remplacer le document</button>
 </body>
```

L’interface et l’objet Screen en JavaScript

Présentation de l’objet Screen:

L’objet **Screen** donne accès à des informations concernant l’écran des visiteurs, comme la taille ou la résolution de l’écran.
On peut utiliser ces informations pour proposer différents affichages à différents visiteurs par exemple.

Les propriétés et méthodes de l’objet Screen:

- **width** : retourne la largeur totale de l’écran ;
- **availWidth** : retourne la largeur de l’écran moins celle de la barre de tâches ;
- **height** : retourne la hauteur totale de l’écran ;
- **availHeight** : retourne la hauteur de l’écran moins celle de la barre de tâches ;

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Cours JavaScript</title>
        <meta charset="utf-8">
        <meta name="viewport"
          content="width=device-width, initial-
scale=1, user-scalable=no">
        <link rel="stylesheet"
href="cours.css">
        <script src='cours.js' async></script>
    </head>

    <body>
        <h1>Titre principal</h1>
        <p>Un paragraphe</p>

        <p id='p1'></p>
    </body>
</html>
```

```javascript
document.getElementById('p1').innerHTML =
    'Dimensions totales de l\'écran : ' + screen.width + '*' +
screen.height +
    '<br>Surface disponible : ' + screen.availWidth + '*' +
screen.availHeight ;
```

Présentation du DOM HTML

Le DOM ou Document Object Model, une interface qui fait partie du BOM (Browser Object Model) et grâce à laquelle on peut manipuler le contenu HTML et les styles de nos pages.

Présentation et définition du DOM ou Document Object Model

Le DOM est une interface de programmation pour des documents HTML ou XML qui représente le document (la page web actuelle) sous une forme qui permet aux langages de script comme le JavaScript d'y accéder et d'en manipuler le contenu et les styles.

Le DOM est ainsi une représentation structurée du document sous forme « d'arbre » crée automatiquement par le navigateur. Chaque branche de cet arbre se termine par ce qu'on appelle un nœud qui va contenir des objets. On va finalement pouvoir utiliser ces objets, leurs propriétés et leurs méthodes en JavaScript.

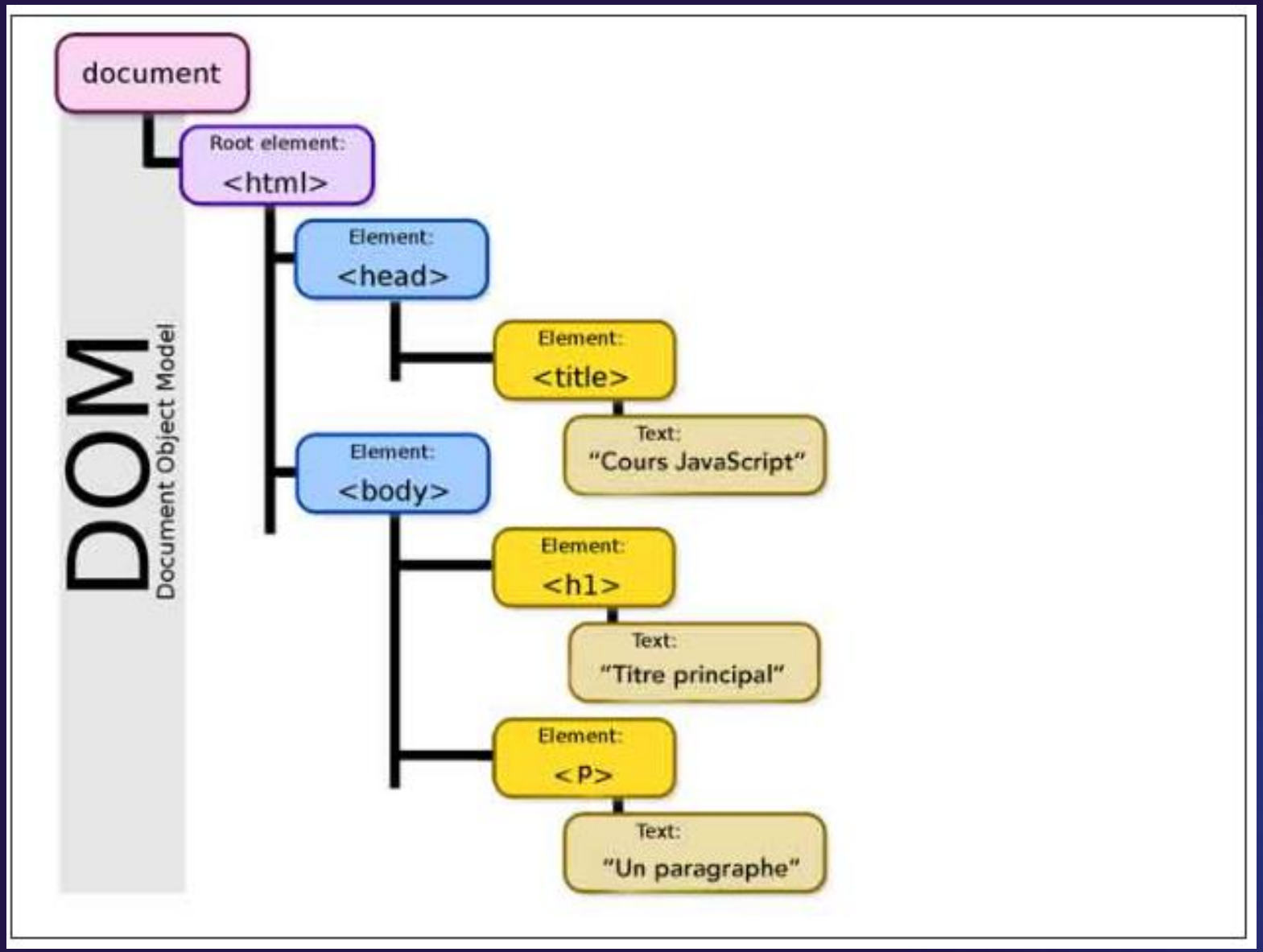
Le DOM contient ou correspond à un ensemble d'APIs qui font partie du BOM comme l'interface Document par exemple qui représente une page et sert de point d'entrée dans l'arborescence du DOM.

Une première présentation de la structure du DOM

Lorsqu'on demande à un navigateur d'afficher une page Web, celui-ci va automatiquement créer un modèle objet de la page ou du document. Ce modèle objet correspond à une autre représentation de la page sous **forme d'arborescence** contenant des objets qui sont de type Node (nœuds).

Les navigateurs utilisent eux-mêmes cette arborescence qui va s'avérer très pratique à manipuler pour eux et notamment pour appliquer les styles aux bons éléments.

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Cours JavaScript</title>
        <meta charset="utf-8">
    </head>
    <body>
        <h1>Titre principal</h1>
        <p>Un paragraphe</p>
    </body>
</html>
```



Le modèle objet d'un document HTML commence toujours avec l'élément html . On parle également de **« nœud » racine.**

Les objets Node ou nœuds du DOM

Le terme « **nœud** » est un terme générique qui sert à désigner tous les objets contenus dans le DOM. A l'extrémité de chaque branche du DOM se trouve un nœud.

A partir du **nœud racine** qui est le nœud HTML on voit deux branches se forment : une première qui va aboutir au nœud HEAD, une deuxième qui aboutit à un nœud BODY.

De nouvelles branches se créent ensuite à partir des nœuds HEAD et BODY et etc.

Les types de nœuds du DOM

On peut distinguer les nœuds les uns des autres en fonction de s'il s'agit d'un nœud constitué par un texte, par un élément, par un commentaire, etc. On va pouvoir utiliser des propriétés et méthodes différentes avec chaque type de nœud puisqu'ils vont dépendre d'interfaces différentes.

Constante	Description
ELEMENT_NODE	Représente un nœud élément (comme p ou div par exemple)>
TEXT_NODE	Représente un nœud de type texte
COMMENT_NODE	Représente un nœud commentaire

L'un des intérêts majeurs du DOM et des nœuds va être qu'on va pouvoir se déplacer de nœuds en nœuds pour manipuler des éléments en utilisant le JavaScript.

Les interfaces composant le DOM

Le DOM contient un ensemble des interfaces , parmi elles on cite:

- L’interface **Window**;
- L’interface **Event** qui représente tout événement qui a lieu dans le DOM
- L’interface **Node** qui est l’interface de base pour une grande partie des objets du DOM ;
- L’interface **Document** qui représente le document actuel et qui va être l’interface la plus utilisée ;
- L’interface **Element** qui est l’interface de base pour tous les objets d’un document ;
- L’interface **ParentNode**
- L’interface **ChildNode**

Accéder à un élément du DOM

On peut Chercher les éléments directement par nom en utilisant les méthodes suivantes :

- **document.getElementsByTagName()**
- **document.getElementsByClassName ()**
- **document.getElementById ()**
- **document.getElementsByName ()**

Ou bien en utilisant un sélecteur CSS associé :

- **document.querySelector()**
- **document.querySelectorAll()**

Accéder aux éléments dans un document avec JS et modifier leur contenu

Accéder à un élément à partir de son sélecteur CSS associé

La façon la plus simple d'accéder à un élément dans un document est de le cibler avec le sélecteur CSS qui lui est associé. Deux méthodes nous permettent de faire cela : les méthodes **querySelector()** et **querySelectorAll()** qui sont des méthodes relevant de **ParentNode** et qu'on va pouvoir implémenter à partir des **interfaces Document** et **Element**.

La méthode **querySelector()** retourne un objet Element représentant le premier élément dans le document correspondant au sélecteur (ou au groupe de sélecteurs) CSS passé en argument ou la valeur null si aucun élément correspondant n'est trouvé.

La méthode **querySelectorAll()** renvoie un objet appartenant à l'interface **NodeList**. Les objets **NodeList** sont des collections de nœuds.

L'objet **NodeList** renvoyé est une liste statique des éléments du document qui correspondent au sélecteur (ou au groupe de sélecteurs) CSS spécifiés.

Lorsqu'on utilise **querySelector() ou querySelectorAll()** avec un objet **Document**, la recherche se fait dans tout le document. Lorsqu'on utilise l'une de ces méthodes à partir d'un objet **Element**, la recherche se fait parmi les descendants de l'élément sur lequel on appelle la méthode en question.

```html
<html>
    <head>
        <title>Cours JavaScript</title>
        <meta charset="utf-8">
        <link rel="stylesheet" href="cours.css">
        <script src='cours.js' async></script>
    </head>

    <body>
        <h1 class='bleu'>Titre principal</h1>
        <p id='p1'>Un paragraphe</p>
        <div>
            <p>Un paragraphe dans le div</p>
            <p class='bleu'>Un autre paragraphe dans le div</p>
        </div>
        <p>Un autre paragraphe</p>
    </body>
</html>
```

Code HTML

```
//Sélectionne le premier paragraphe du document et change son texte avec la propriété textContent

document.querySelector('p').textContent = '1er paragraphe du document';

/*Sélectionne le premier paragraphe du premier div du document et modifie son texte*/

let documentDiv = document.querySelector('div');
documentDiv.querySelector('p').textContent = '1er paragraphe du premier div';

/*Sélectionne le premier paragraphe du document avec un attribut class='bleu‘ et change sa couleur en bleu
avec la propriété style */

document.querySelector('p.bleu').style.color = 'blue';

//Sélectionne tous les paragraphes du document

let documentPara = document.querySelectorAll('p');

//Sélectionne tous les paragraphes du premier div

let divPara = documentDiv.querySelectorAll('p');
```

Accéder à un élément en fonction de la valeur de son attribut id

La méthode **getElementById()** est une méthode qu'on va implémenter à partir d'un objet Document.

Cette méthode renvoie un objet **Element** qui représente l'élément dont la valeur de l'attribut **id** correspond à la valeur spécifiée en argument.

La méthode **getElementById()** est un moyen simple d'accéder à un élément en particulier (si celui-ci possède un id) puisque les id sont uniques dans un document.

Exemple:

```
//Sélectionne l'élément avec un id = 'p1' et modifie la couleur du texte

document.getElementById('p1').style.color = 'red';
```

Accéder à un élément en fonction de la valeur de son attribut class

La méthode **getElementsByClassName()** va renvoyer une liste des éléments possédant un attribut **class** avec la valeur spécifiée en argument. La liste renvoyée est un objet de l'interface **HTMLCollection** qu'on va pouvoir traiter comme un tableau.

En utilisant la méthode **getElementsByClassName()** avec un objet Document, la recherche des éléments se fera dans tout le document. En l'utilisant avec un objet Element, la recherche se fera dans l'élément en question.

```
<body>
    <h1 class='bleu' >Titre principal</h1>
    <p id='p1'>Un paragraphe</p>
    <div>
        <p class='bleu'>Un paragraphe dans le div</p>
        <p class='bleu'>Un autre paragraphe dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
</body>
```

```
//Sélectionne les éléments avec une class = 'bleu'
let bleu = document.getElementsByClassName('bleu');

//"bleu" est un objet de HTMLCollection qu'on va manipuler comme un tableau
for(valeur of bleu){
    valeur.style.color = 'blue';
}
```

Accéder à un élément en fonction de son identité

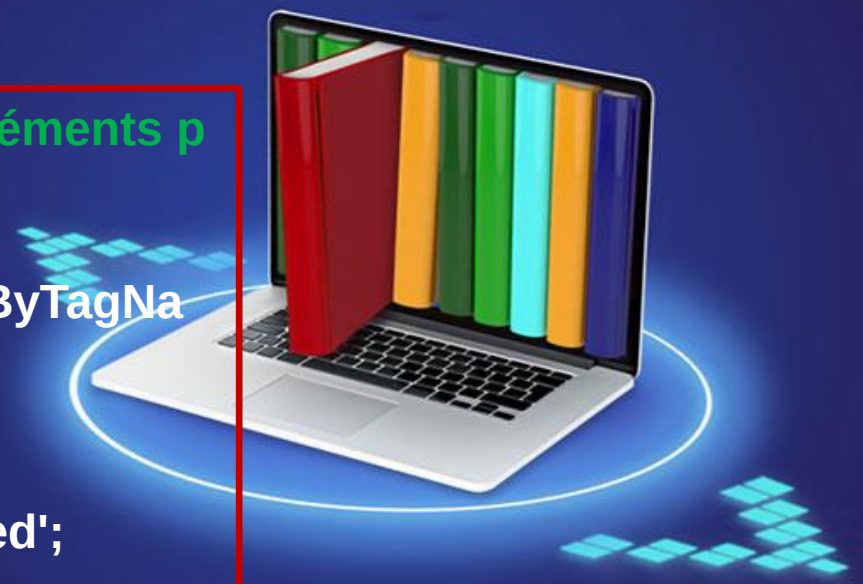
La méthode **getElementsByTagName()** permet de sélectionner des éléments en fonction de leur nom et renvoie un objet **HTMLCollection** qui consiste en une liste d'éléments correspondant au nom de balise passé en argument

Lorsqu'on utilise **getElementsByTagName()** avec un objet Document, la recherche se fait dans tout le document tandis que lorsqu'on utilise **getElementsByTagName()** avec un objet Element, la recherche se fera dans l'élément en question seulement.

Exemple: avec Document

```
<body>
    <h1 class='bleu'>Titre principal</h1>
    <p id='p1'>Un paragraphe</p>
    <div id='test'>
        <p>Un paragraphe dans le div</p>
        <p class='bleu'>Un autre
paragraphe dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
</body>
```

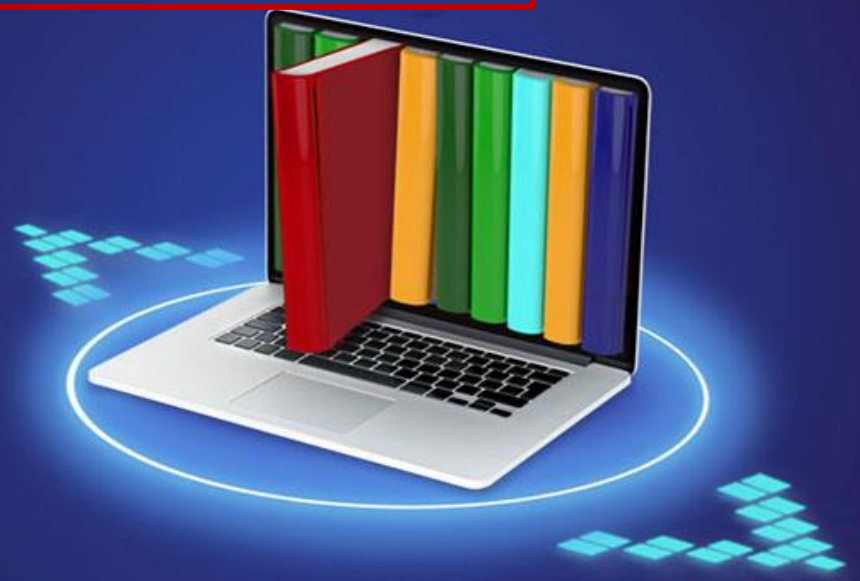
```
//Sélectionne tous les éléments p
du document
let parag =
document.getElementsByTagNa
me('p');

for(valeur of parag){
    valeur.style.color = 'red';
}
```



Exemple: avec Element

```
<body>
    <h1 class='bleu'>Titre principal</h1>
    <p id='p1'>Un paragraphe</p>
    <div id='test'>
        <p>Un paragraphe dans le div</p>
        <p class='bleu'>Un autre
paragraphe dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
  </body>
```

```
//Sélectionne tous les éléments p du div avec id test

let divelem = document.getElementById('test');
let pelement = divelem.getElementsByTagName('p');
for(valeur2 of pelement){
    valeur2.style.color = 'red';
}
```



Accéder à un élément en fonction de son attribut name

La méthode **getElementsByName()** renvoie un objet NodeList contenant la liste des éléments portant un **attribut name**. On utilise cette méthode pour sélectionner des éléments de formulaire par exemple.

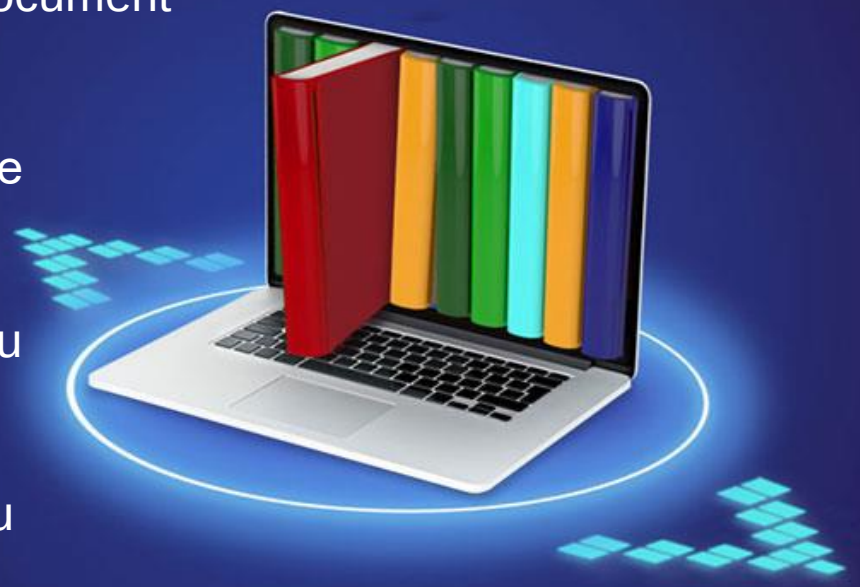
```
let elt =
document.getElementsByName("txt1");
for(valeur2 of elt){
    valeur2.style['color']= 'red'; }
```

```
 <body>
     <form>
         <input type="text"
name="txt1" value="nom">
         <input type="text"
name="txt2"
value="prenom">
        </form >
  </body>
```

Accéder directement à des éléments particuliers avec les propriétés de Document

L'interface Document fournit également des propriétés qui permettent d'accéder directement à certains éléments de la page HTML:

- La propriété **body** qui retourne le nœud représentant l'élément body ;
- La propriété **head** qui retourne le nœud représentant l'élément head ;
- La propriété **links** qui retourne une liste de tous les éléments a ou area possédant un href avec une valeur ;
- La propriété **title** qui retourne le titre (le contenu de l'élément title) du document ou permet de le redéfinir ;
- La propriété **url** qui renvoie l'URL du document sous forme de chaine de caractères ;
- La propriété **scripts** qui retourne une liste de tous les éléments script du document;
- La propriété **cookie** qui retourne la liste de tous les cookies associés au document sous forme de chaine de caractères ou qui permet de définir un nouveau cookie.



```
<body>
    <h1 class='bleu'>Titre principal</h1>
    <p id='p1'>Un paragraphe</p>
    <div>
      <p>Un paragraphe dans le div</p>
      <p class='bleu'>Un autre paragraphe dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
</body>
```

```
//Sélectionne l'élément body et applique une couleur jaune

document.body.style.color = ‘yellow';

//Modifie le texte de l'élément title

document.title= 'Le Document Object Model';
```

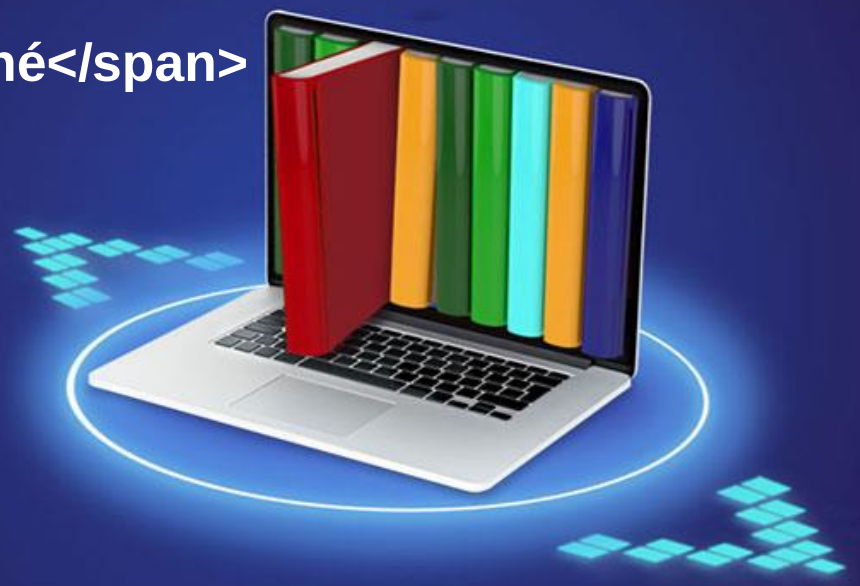
Accéder au contenu des éléments et le modifier

Accéder à un nœud en particulier permet d'effectuer différentes manipulations sur celui-ci, et notamment de récupérer le contenu de cet élément ou de le modifier.

Pour faire cela , nous utilisons l'une des propriétés suivantes :

- La propriété **innerHTML** de l'interface Element permet de récupérer ou de redéfinir la syntaxe HTML interne à un élément ;
- La propriété **outerHTML** de l'interface Element permet de récupérer ou de redéfinir l'ensemble de la syntaxe HTML interne d'un élément et de l'élément en soi ;
- La propriété **textContent** de l'interface Node représente le contenu textuel d'un nœud et de ses descendants.
- La propriété **innerText** de l'interface Node représente le contenu textuel visible sur le document final d'un nœud et de ses descendants.

```
<body>

    <h1>Titre principal</h1>
    <p id='p1'>Un paragraphe</p>
    <div>
        <p>Un paragraphe dans le div</p>
        <p id='texte'>Un autre paragraphe dans le div</p>
    </div>
    <p id='p2'>Un autre paragraphe
     <span style='visibility: hidden'>avec du contenu caché</span>
    </p>
    <p id='p3'></p>

</body>
```

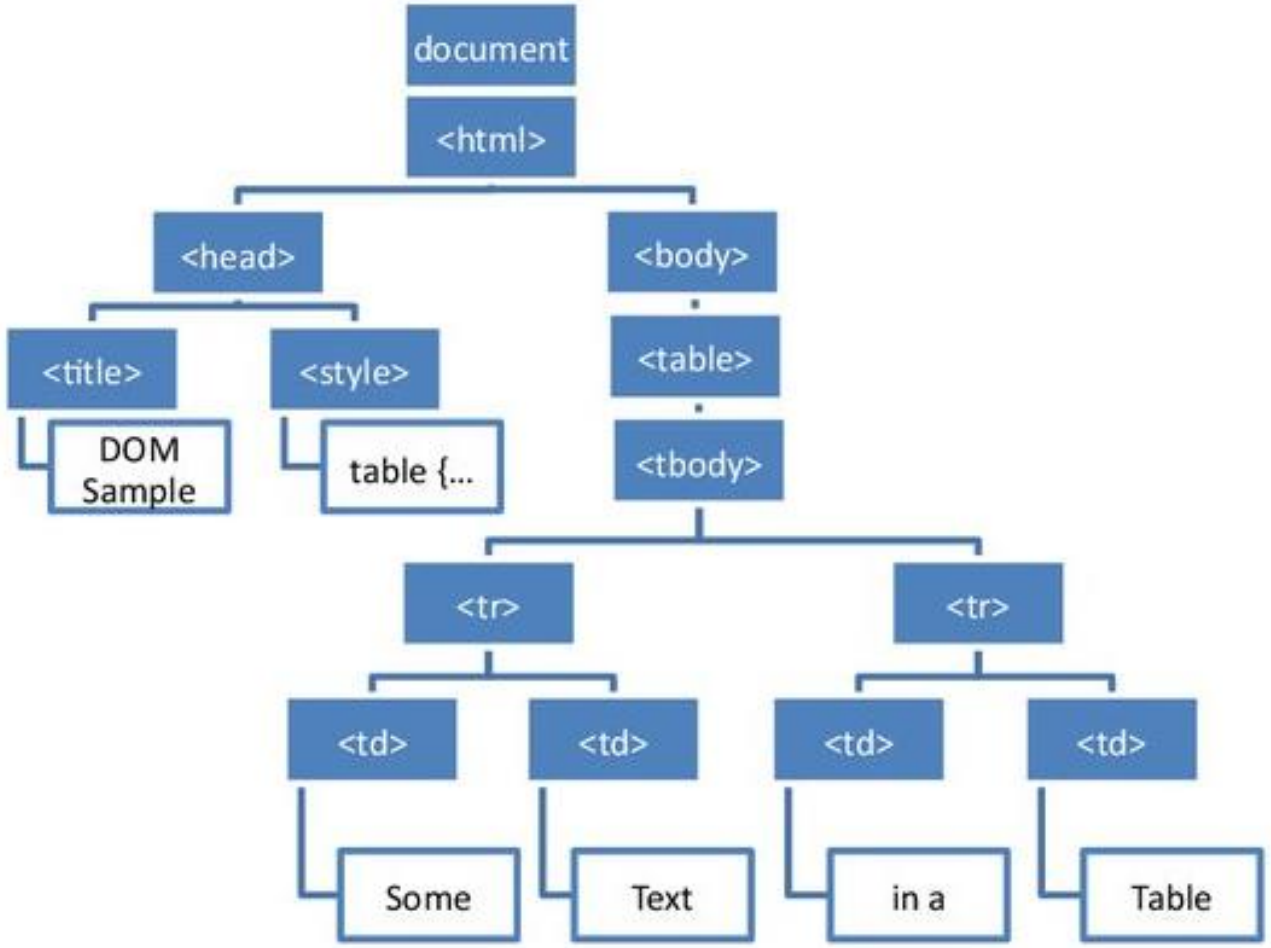


```
//On accède au HTML du 1er paragraphe du document et on le modifie
document.querySelector('p').outerHTML = '<h2>Je suis un titre h2</h2>';

//On accède au contenu HTML interne du div et on le modifie
document.querySelector('div').innerHTML +=
    '<ul><li>Elément n°1</li><li>Elément n°2</li></ul>';

//On accède au contenu textuel de l'élément avec un id='texte' et on le modifie
document.getElementById('texte').textContent = '<span>Texte modifié</span>';

//On accède au texte visible de l'élément avec l'id = 'p2'
let texteVisible = document.getElementById('p2').innerText;
alert (texteVisible);

//On accède au texte (visible ou non) de l'élément avec l'id = 'p2'
let texteEntier = document.getElementById('p2').textContent;
alert (texteEntier);

//Affiche les résultats du dessus dans l'élément avec l'id = 'p3'
document.getElementById('p3').innerHTML =
    'Texte visible : ' + texteVisible + '<br>Texte complet : ' + texteEntier;
```

Naviguer ou se déplacer dans le DOM en JavaScript grâce aux noeuds

DOM Tree

```html
<!DOCTYPE html>
<html>
    <head>
        <title>DOM Sample</title>
        <style type="text/css">
            table {
                border: 1px solid black;
            }
        </style>
    </head>
    <body>
        <table>
            <tbody>
                <tr>
                    <td>Some</td>
                    <td>Text</td>
                </tr>
                <tr>
                    <td>in a</td>
                    <td>Table</td>
                </tr>
            </tbody>
        </table>
    </body>
</html>
```



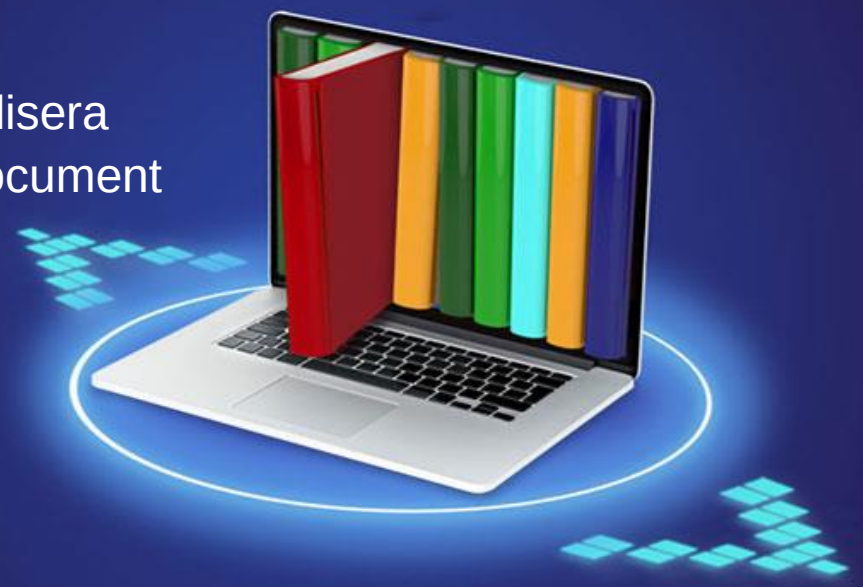
Accéder au parent ou à la liste des enfants d’un nœud

La propriété **parentNode** de l’interface **Node** renvoie le parent du nœud spécifié dans l’arborescence du DOM ou null si le nœud ne possède pas de parent.

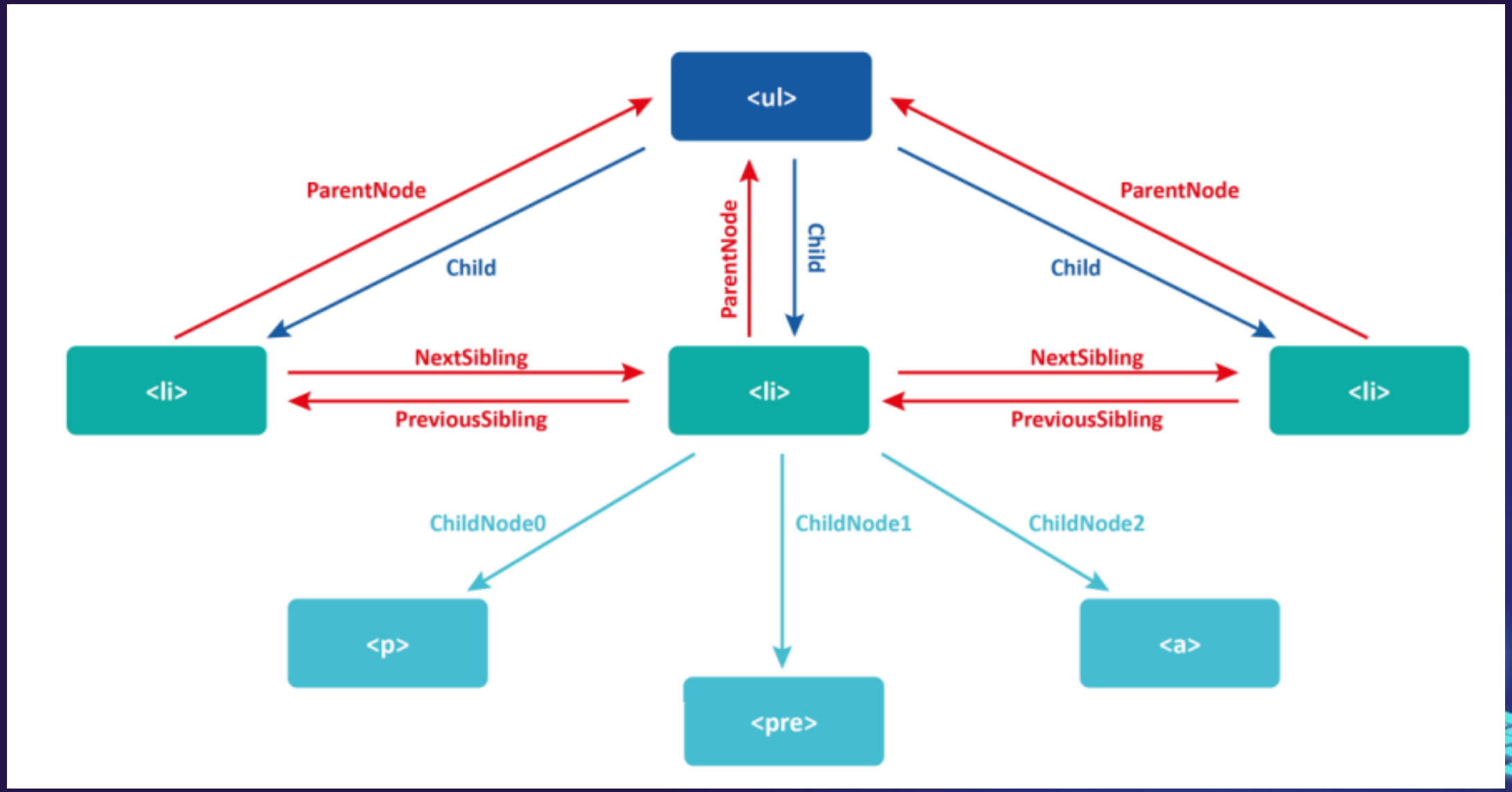
La propriété **childNodes** de cette même interface renvoie une liste des nœuds enfants de l’élément donné.

A noter que la propriété **childNodes** renvoie tous les nœuds enfants et cela quels que soient leurs types : nœuds élément, nœuds texte, nœuds commentaire, etc.

Si on ne souhaite récupérer que les nœuds enfants éléments, alors on utilisera plutôt la propriété **children** du **parentNode** (qui est implémenté par Document et par Element).



Relation entre les nœuds



Remarque:

Le parent d’un élément n’est pas forcément un nœud Element mais peut également être un nœud Document. La propriété **parentNode** renverra le parent d’un nœud quel que soit son type.

Pour n’accéder au parent que dans le cas où celui-ci est un nœud Element, on utilisera plutôt la propriété **parentElement** de Node qui ne renvoie le parent d’un nœud que s’il s’agit d’un nœud Element ou null sinon.

```
let p1 = document.getElementById('p1');
let p2 = document.getElementById('p2');

p2.parentNode.style.backgroundColor = 'Orange';

//On accède à tous les nœuds enfants de p1

let enfantsP1 = p1.childNodes;

/*Comme pour les tableaux pour manipuler un nœud enfant en particulier (le 1ier enfant a l'indice 0, le 2ieme l'indice 1, etc.)*/

enfantsP1[1].style.fontWeight = 'bold';

//On accède aux nœuds enfants éléments seulement de p1
let enfantsEltP1 = p1.children;

//On peut ensuite accéder aux différents enfants comme on le ferait avec un tableau

enfantsEltP1[0].style.textDecoration = 'underline';
```

```
<body>
    <h1>Titre principal</h1>
    <p id='p1'>Un paragraphe <span>avec un span</span></p>
    <div>
        <p id='p2'>Un paragraphe dans le div</p>
        <p>Un autre paragraphe dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
</body>
```

Accéder à un nœud enfant en particulier à partir d'un nœud parent

La propriété **firstChild** de l'interface Node renvoie le premier nœud enfant direct d'un certain nœud ou null s'il n'en a pas.

La propriété **lastChild**, au contraire, renvoie le dernier nœud enfant direct d'un certain nœud ou null s'il n'en a pas.

Notez que ces deux propriétés vont renvoyer les premiers et derniers nœuds enfants quels que soient leurs types (nœuds élément, nœuds texte ou nœuds commentaire).

Pour renvoyer le premier et le dernier nœud enfant de type élément seulement d'un certain nœud, on utilisera plutôt les propriétés **firstElementChild** et **lastElementChild** du ParentNode.

Exemple:

```
<body>
Premier paragraphe
    <h1>Titre principal</h1>
    <p id='p1'>Un paragraphe <span>avec un span</span></p>
    <div>
        <p id='p2'>Un paragraphe dans le div</p>
        <p>Un autre paragraphe dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
Dernier paragraphe
</body>
```

```
//On accède au premier nœud enfant de body
let bodyFirstChild = document.body.firstChild;

//On accède au dernier nœud enfant de body
let bodyLastChild = document.body.lastChild;

//On accède au premier nœud enfant élément de body
let bodyFirstElementChild = document.body.firstElementChild;

//On accède au dernier nœud enfant élémentde body
let bodyLastElementChild = document.body.lastElementChild;

alert(
    'Premier noeud enfant de body : ' + bodyFirstChild +
    '\nPremier noeud enfant élément de body : ' +
bodyFirstElementChild +
    '\nDernier noeud enfant de body : ' + bodyLastChild +
    '\nDernier noeud enfant élément de body : ' +
bodyLastElementChild
);
```

Accéder au nœud précédent ou suivant un nœud dans l'architecture DOM

La propriété **previousSibling** de l'interface Node renvoie le nœud précédent un certain nœud dans l'arborescence du DOM (en ne tenant compte que des nœuds de même niveau) ou null si le nœud en question est le premier.

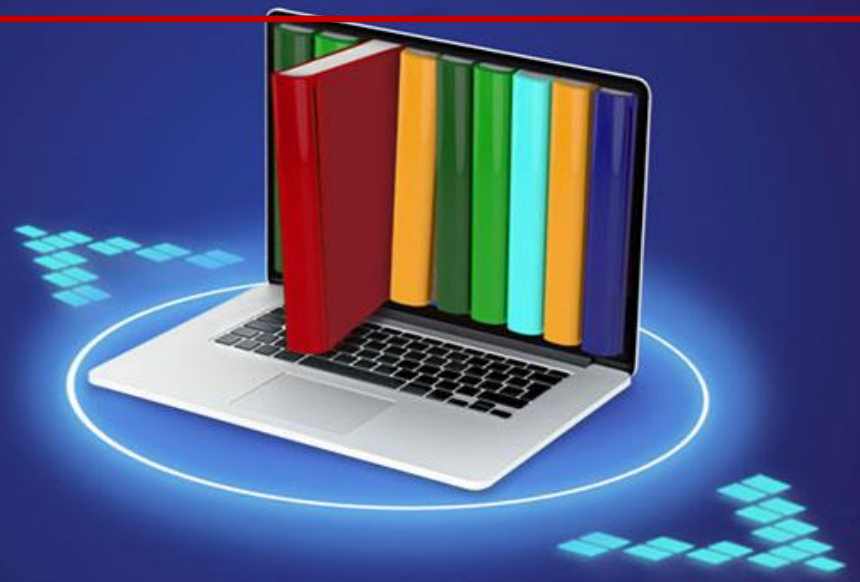
La propriété **nextSibling**, au contraire, renvoie elle le nœud suivant un certain nœud dans l'arborescence du DOM ou null si le nœud en question est le dernier.

Ces deux propriétés vont renvoyer le nœud précédent ou suivant un certain nœud, et cela quel que soit le type du nœud précédent ou suivant.

Si on souhaite accéder spécifiquement au nœud élément précédent ou suivant un certain nœud, on utilisera plutôt les propriétés **previousElementSibling** et **nextElementSibling**

```
<body>
    <h1>Titre principal</h1>
    <p id='p1'>Un paragraphe <span>avec un span</span></p>
    <div>
        <p id='p2'>Un paragraphe dans le div</p>
        <p>Un autre paragraphe dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
</body>
```

```
let p1 = document.getElementById('p1');

let p1Previous = p1.previousSibling;
let p1Next = p1.nextSibling;
let p1PreviousElement = p1.previousElementSibling;
let p1NextElement = p1.nextElementSibling;

console.log(p1Previous);
console.log(p1Next );

p1PreviousElementSibling.style.color = 'blue';
p1NextElementSibling.style.backgroundColor = 'orange';
```



Méthodes d’ajout et suppression d'éléments

Méthode	Description
document.**createElement**(element)	Créer un élément HTML
document.**removeChild**(element)	Supprimer un élément HTML
document.**appendChild**(element)	Ajouter un élément HTML enfant
document.replaceChild(new, old)	Remplacer un élément HTML
document.**write**(text)	Écrire dans un document HTML
document.**getElementById**(*id*).**onclick** = function(){*code*}	Ajouter un événement de clic à l’élément selectionné

Propriétés des éléments DOM

Méthode	Description
element.**innerHTML**	Permet de récupérer tout le contenu HTML d'un élément du DOM
element.**attribute**	Changer l'attribut d'un élément
element.**style.property**	Changer le style d'un élément
element.**textContent**	Renvoie tout le contenu textuel d'un élément du DOM, sans le balisage HTML
element.**classList**	Permet de récupérer la liste des classes d'un élément du DOM

Récapitulatif

Navigation entre les nœuds de l'arborescence DOM

❑ Navigation dans les nœuds enfants

- ✓ **firstChild** : Retourne le premier enfant de l'élément.
- ✓ **firstElementChild** : Retourne le premier élément enfant du parent.
- ✓ **lastChild** : Retourne le dernier enfant de l'élément.
- ✓ **lastElementChild** : Retourne le dernier élément enfant du parent.
- ✓ **childNodes** : Retourne tous les enfants de l'élément sous forme d'une collection.
- ✓ **children** : Renvoie tous les enfants qui sont des éléments sous forme d'une collection.

❑ Navigation dans les nœuds parents

- ✓ **parentNode** : Renvoie le nœud parent de l'élément.
- ✓ **parentElement** : Renvoie le nœud de l'élément parent de l'élément.

❑ Navigation dans les nœuds frères

- ✓ **nextSibling** : Renvoie le nœud frère correspondant au prochain enfant du parent.
- ✓ **nextElementSibling** : Renvoie l'élément frère correspondant au prochain enfant de son parent.
- ✓ **previousSibling** : Renvoie le nœud frère qui est un enfant précédent de son parent.
- ✓ **previousElementSibling** : Renvoie l'élément frère qui est un enfant précédent de son parent

Obtenir le nom d’un nœud, son type ou son contenu

Lorsqu’on accède à un certain nœud, on peut obtenir le nom du nœud, savoir ce qu’il contient et connaitre son type. Pour cela on peut utiliser les propriétés suivantes de l’interface Node :

- La propriété **nodeName** qui retourne une chaine de caractères contenant le nom du nœud (nom de la balise dans le cas d’un nœud de type Element ou #text dans le cas d’un nœud de type Text) ;
- La propriété **nodeValue** qui renvoie ou permet de définir la valeur du nœud. On pourra notamment utiliser cette propriété sur des nœuds #text pour obtenir le texte qu’ils contiennent ;
- La propriété **nodeType** renvoie un entier qui représente le type du nœud . Renvoie le type du nœud : 1 si c'est un élément, 2 si c'est un attribut, et 3 si c'est du texte.

```
let p1 = document.getElementById('p1');
let p1PreviousSibling = p1.previousSibling;
let p1PreviousElementSibling =
p1.previousElementSibling;

alert(
'Nom noeud p1 : ' + p1.nodeName +
'\nValeur noeud p1 : ' + p1.nodeValue +
'\nType noeud p1 : ' + p1.nodeType +
'\n\nNom p1PreviousSibling  : ' +
p1PreviousSibling.nodeName +
'\n\nType p1PreviousSibling  : ' +
p1PreviousSibling.nodeType +
'\n\nNom p1PreviousElementSibling : ' +
p1PreviousElementSibling.nodeName +
'\nValeur du premier noeud enfant de p1 : ' +
p1.firstChild.nodeValue
);
```

```
<body>
      <h1>Titre principal</h1>
      <p id='p1'>Un paragraphe <span>avec un
span</span></p>
      <div>
         <p id='p2'>Un paragraphe dans le div</p>
         <p>Un autre paragraphe dans le div</p>
      </div>
      <p>Un autre paragraphe</p>
  </body>
```

Ajouter, modifier ou supprimer des éléments du DOM avec JavaScript

Créer de nouveaux nœuds et les ajouter dans l'arborescence du DOM

Créer un nœud Element ou un nœud Text:

Pour créer un nouvel élément HTML en JavaScript, nous pouvons utiliser la méthode **createElement()** de l'interface Document. Cette méthode va prendre en argument le nom de l'élément HTML que l'on souhaite créer.

```
let newP = document.createElement('p');
```

Pour insérer du texte dans notre nœud élément, on va pouvoir par exemple utiliser la propriété textContent.

```
let newP = document.createElement('p');
newP.textContent = 'Paragraphe créé et inséré grâce au DOM';
```

On peut également créer directement un nœud de type texte en utilisant la méthode **createTextNode()** de Document et ensuite insérer ce nœud dans un nœud élément

```
Let newTexte = document.createTextNode('Texte écrit en JavaScript');
```

Insérer un nœud dans le DOM

Il existe différentes méthodes permettant d'insérer des nœuds dans d'autres nœuds. La différence entre ces méthodes consiste dans la position où le nœud va être inséré. On utilise les méthodes **prepend()** et **append()** du ParentNode.

Ces deux méthodes permettent d'insérer un nœud ou du texte avant le premier enfant d'un certain nœud ou après le dernier enfant de ce nœud.

```
<body>
<h1>Titre principal</h1>
<p id='p1'>Un paragraphe <span>avec un span</span></p>
<div> <p id='p2'>Un paragraphe dans le div</p>
<p>Un autre paragraphe dans le div</p> </div>
<p>Un autre paragraphe</p>
</body>
```

```
let b = document.body;
let newP = document.createElement('p');
let newTexte = document.createTextNode('Texte écrit en JavaScript');
newP.textContent = 'Paragraphe créé et inséré';
//Ajoute le paragraphe créé comme premier enfant de l'élément body
b.prepend(newP);
//Ajoute le texte créé comme dernier enfant de l'élément body
b.append(newTexte);
```

Remplacer un nœud dans le DOM

Pour remplacer un nœud, on utilise la méthode **replaceChild()** qui va remplacer un certain nœud par un autre.

Cette méthode prend en arguments le nœud de remplacement et le nœud qui doit être remplacé. Si le nœud de remplacement existe déjà dans le DOM, il sera d'abord retiré de son emplacement d'origine.

```
<body>
    <h1>Titre principal</h1>
    <p id='p1'>Un paragraphe
<span>avec un span</span></p>
    <div>
        <p id='p2'>Un paragraphe
dans le div</p>
        <p>Un autre paragraphe
dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
</body>
```

```
let b = document.body;
let p1 =
document.getElementById('p1');
let p2 =
document.getElementById('p2');
let p4 = b.lastElementChild;
//On accède au dernier paragraphe
let newP =
document.createElement('p');
//On crée un nouveau noeud
newP.textContent = 'Paragraphe créé et
inséré grâce au JS';

//On remplace p4 par newP
b.replaceChild(newP, p4);
```

Supprimer un nœud du DOM

Pour supprimer totalement un nœud du DOM, on utilise la méthode **removeChild()** de Node qui supprime un nœud enfant passé en argument d'un certain nœud parent de l'arborescence du DOM et retourner le nœud retiré.

On peut également utiliser la méthode **remove()** du ChildNode() qui permet de retirer un nœud de l'arborescence (cette façon de faire est plus récente que la précédente).

```
let b = document.body;
let p1 = document.getElementById('p1');
let p2 = document.getElementById('p2');
//Supprime p1 du DOM et renvoie le noeud supprimé
let eltDel = b.removeChild(p1);
//Supprime p2 du DOM
p2.remove() ;
alert('Noeud supprimé du DOM : ' + eltDel + '\nContenu : ' +
eltDel.textContent);
```

Manipuler les attributs et les styles des éléments via le DOM en JavaScript

Grâce au DOM, on peut tester si un élément possède un attribut, récupérer la valeur d'un attribut donné, ajouter, modifier ou supprimer des attributs et manipuler les styles CSS des éléments.

Tester la présence d'attributs

La méthode **hasAttribute()** de l'interface **Element** permet de tester la présence d'un attribut en particulier pour un élément. Cette méthode prend en argument le nom de l'attribut qu'on recherche et renvoie la valeur booléenne true si l'élément possède bien cet attribut ou false sinon.

Pour vérifier si un élément possède des attributs ou pas, on utilise la méthode **hasAttributes()**. Cette méthode retourne à nouveau une valeur booléenne (true si l'élément possède au moins un attribut ou false si l'élément ne possède pas d'attribut).

```
let p1 = document.querySelector('p');

let vide = document.getElementById('vide');

//Si p1 possède des attributs, hasAttributes() renvoie true

if(p1.hasAttributes()){
    vide.textContent = 'p1 possède des attributs';
}

//Si p1 possède un attribut id, hasAttribute() renvoie true

if(p1.hasAttribute('id')){
    vide.textContent += ' dont un attribut id';
}
```

```
<body>
    <h1>Titre principal</h1>
    <p id='p1' class='blue'>Un paragraphe</p>
    <div>
        <p id='p2'>paragraphe dans le div</p>
        <p>Un autre paragraphe dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
    <p id='vide'></p>
</body>
```

Récupérer la valeur ou le nom d’un attribut ou définir un attribut

La propriété **attributes** de l’interface Element renvoie la liste des attributs d’un (nœud) élément spécifié. La liste d’attributs est renvoyée sous la forme « clef / valeur » .

On va pouvoir récupérer les noms et valeurs de chaque attribue en utilisant une boucle for pour itérer dans cette liste et les propriétés name et value de l’interface Attr.

```
let p1 = document.querySelector('p');
let vide = document.getElementById('vide');

if(p1.hasAttributes()){
    let attP1 = p1.attributes;    // Liste des attributs de p1
    vide.innerHTML = 'Liste des attributs de p1 : <br>‘

    //La propriété length renvoie le nombre d'attributs

    for(let i = 0; i < attP1.length; i++){
        vide.innerHTML += attP1[i].name + ' = ' + attP1[i].value +
'<br>';
    }
}
```

Pour ajouter un nouvel attribut ou changer la valeur d'un attribut existant pour un élément, on utilise la méthode **setAttribute()** à laquelle on va passer en arguments le nom et la valeur de l'attribut à ajouter ou à modifier.

```
let p1 = document.querySelector('p');
let p2 = document.getElementById('p2');
let vide = document.getElementById('vide');

p2.setAttribute('class', 'blue');
vide.innerHTML += 'class : ' + p2.className + '<br>id : '
+ p2.id;
```

```
<head><style>.blue{color: blue;}</style> </head>
  <body>
     <h1>Titre principal</h1>
     <p id='p1' class='blue'>Un paragraphe
<span>avec un span</span></p>
     <div>
        <p id='p2'>Un paragraphe dans le div</p>
        <p>Un autre paragraphe dans le div</p>
     </div>
     <p>Un autre paragraphe</p>
     <p id='vide'></p>
  </body>
```

Supprimer un attribut

Pour supprimer un attribut d’un élément, on utilise la méthode **removeAttribute()** de l’interface Element. Cette méthode va prendre en argument le nom de l’attribut à supprimer.

```
let p1 = document.querySelector('p');
p1.removeAttribute('class');
```

Exercice:

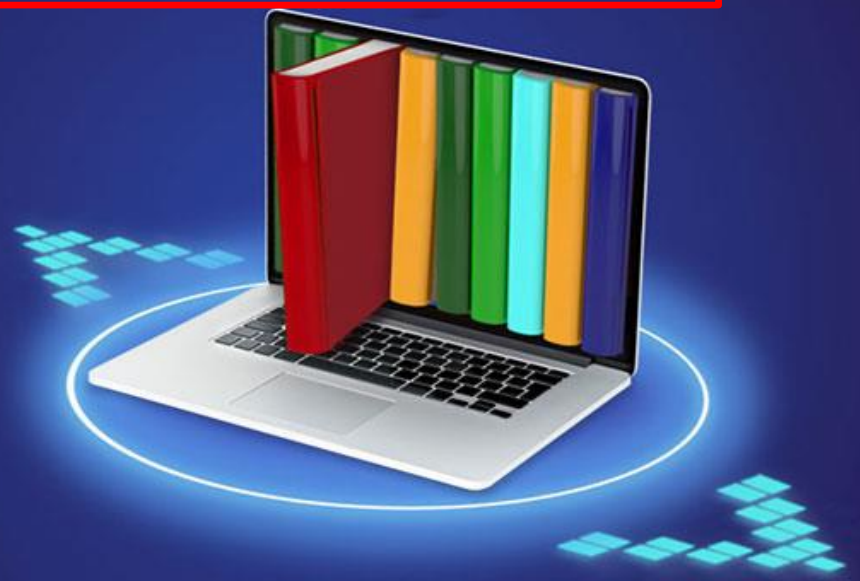
```
<head>
<title>Rick Astley</title>
  </head>
  <body>
    <h1>Title</h1>
    <p class="couplet" id="couplet1">
            We're no strangers<br/>
            We're no strangers<br/>
            </p>
    <p class="couplet">Gotta make you understand<br /></p>
    <p class="couplet refrain">Never never<br /></p>
    <p class="couplet">We've known each other for so long <br/></p>
    <p class="couplet">And if you ask me how I'm feeling <br /></p>
    <p class="couplet refrain">Never gonna give you up<br /></p>
    <div id="erreur">
      <p>Une erreur s'est produite lors de l'enregistrement</p>
    </div>
  </body>
```

Modifier les styles d'un élément

La propriété **style** permettre de définir les styles en ligne d'un élément. Cette propriété respecte la norme lower camel case : elle doit être écrite sans espace ni tiret, avec une majuscule au début de chaque mot sauf pour le premier : la propriété CSS **background-color**, par exemple, **s'écrira backgroundColor**.

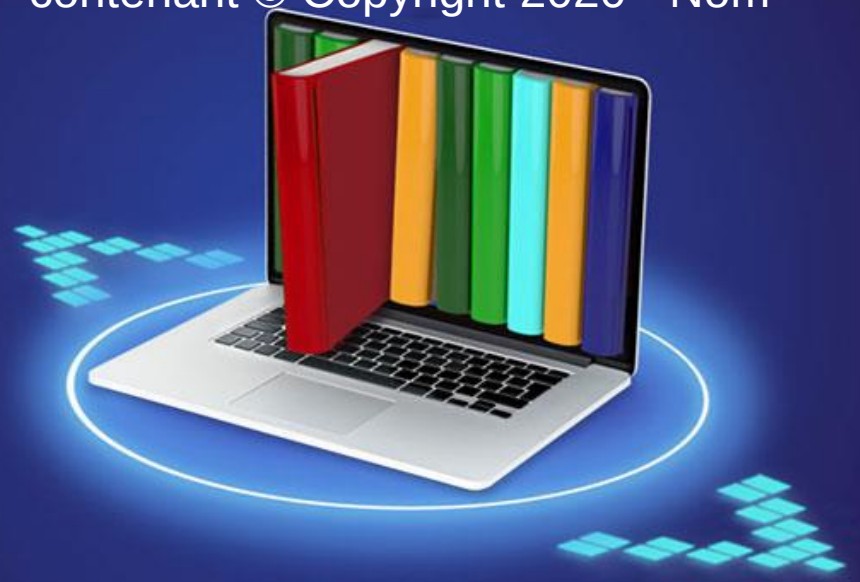
```
let p1 = document.querySelector('p');
let p2 = document.getElementById('p2');

p1.style.color = 'red';
p1.style.fontSize = '20px';
p2.style.backgroundColor = 'orange';
```

```
<body>
    <h1>Titre principal</h1>
    <p id='p1' class='blue'>Un paragraphe
<span>avec un span</span></p>
    <div>
        <p id='p2'>Un paragraphe dans le
div</p>
        <p>Un autre paragraphe dans le div</p>
    </div>
    <p>Un autre paragraphe</p>
    <p id='vide'></p>
  </body>
```



Travail à faire

1. Sélectionnez le premier fils de l'élement body et modifiez son contenu en Rick Astley.
2. Sélectionnez tous les éléments possédant la classe couplet et supprimer la première ligne qui est en double.
3. N'oubliez pas de supprimer le <br /> qui cause le retour à la ligne
4. Supprimer l'élément ayant pour id erreur
5. Ajoutez en bas de la page (dernier element de <body>), un <footer> contenant © Copyright 2020 - Nom



La gestion d'évènements en JavaScript et la méthode addEventListener

Les évènements sont des actions qui se produisent et auxquelles on va pouvoir répondre en exécutant un code. Par exemple, on va pouvoir afficher ou cacher du texte suite à un clic d'un utilisateur sur un élément, on change la taille d'un texte lors du passage de la souris d'un utilisateur sur un élément.

Présentation et première définition des évènements:

En JavaScript, un évènement est une action qui se produit et qui possède deux caractéristiques essentielles :

- C'est une action qu'on peut « écouter », c'est-à-dire une action qu'on peut détecter car le système va nous informer qu'elle se produit ;

- C'est une action à laquelle on peut « répondre », c'est-à-dire qu'on va pouvoir attacher un code à cette action qui va s'exécuter dès qu'elle va se produire.

Ces évènements peuvent être très différents les uns des autres :

- Le chargement du document est un évènement ;
- Un clic sur un bouton effectué par un utilisateur est un évènement ;
- Le survol d'un élément par la souris d'un utilisateur est un évènement ;

Définir des gestionnaires d’évènements

Pour écouter et répondre à un évènement, on définit **des gestionnaires d’évènements**.

Un **gestionnaire d’évènements** est divisé en deux parties : une partie qui va servir à écouter le déclenchement de l’évènement, et une partie gestionnaire en soi qui va être le code à exécuter dès que l’évènement se produit.

En JavaScript, il existe trois grandes façons d’implémenter un gestionnaire d’évènements :

- On peut utiliser des attributs HTML de type évènement (non recommandé) ;
- On peut utiliser des propriétés JavaScript liées aux évènements ;
- On peut utiliser la méthode addEventListener() (recommandé).

1 **Écouteur d'événement (Event Listener) :**

L'écouteur d'événement est un objet qui attend qu’un certain événement se produise (un clic, un mouvement de souris, etc.).

2 **Gestionnaire d'événements :**

Le gestionnaire d'événements correspond généralement à une fonction appelée suite à la production de l’événement.

1. Utiliser les attributs HTML pour gérer un évènement

L'utilisation d'attributs HTML pour prendre en charge un évènement est la méthode la plus ancienne. Cette façon de faire ne devrait plus être utilisée aujourd'hui. Cependant, de nombreux sites utilisent encore ce type de syntaxe.

L'idée est d'insérer un attribut HTML lié à l'évènement qu'on souhaite gérer directement dans la balise ouvrante d'un élément à partir duquel on va pouvoir détecter le déclenchement de cet évènement.

Ces attributs HTML de **« type évènement »** possèdent souvent le nom de l'évènement qu'ils doivent écouter et gérer précédé par **« on »** comme par exemple :

- L'attribut **onclick** pour l'évènement « clic sur un élément » ;
- L'attribut **onmouseover** pour l'évènement « passage de la souris sur un élément » ;
- L'attribut **onmouseout** pour l'évènement « sortie de la souris d'élément » ;

On va passer en valeur de ces attributs le code JavaScript qu'on souhaite exécuter (généralement une fonction) suite au déclenchement de l'évènement en question.

Dès que l'évènement va être détecté, le code présent dans l'attribut va être exécuté.

```
<body>
    <h1>Titre principal</h1>
    <p>Un premier paragraphe</p>
    <button onclick="alert('Bouton cliqué')">Cliquez moi !</button>
    <div onmouseover="this.style.backgroundColor='orange'"
     onmouseout="this.style.backgroundColor='white'">
      <p>Un paragraphe dans un div</p>
      <p>Un autre paragraphe dans le div</p>
    </div>
  </body>
```

2. Utiliser les propriétés JavaScript pour gérer un évènement

Chaque évènement est représenté en JavaScript par un objet basé sur **l'interface Event**. L'interface Event est l'interface de base commune pour tout évènement qui se produit dans le DOM. Certains types d'évènements sont ensuite définis plus précisément dans des interfaces qui héritent de Event.

Ces gestionnaires d'évènements sont des propriétés qui sont de la forme « on » + nom de l'évènement géré, c'est-à-dire qui ont des noms similaires aux attributs HTML vus précédemment. Le code à exécuter est généralement sous forme de fonction anonyme.

```
//On sélectionne le premier button et le premier div du document

let b1 = document.querySelector('button');
let d1 = document.querySelector('div');

//On utilise les propriétés gestionnaires d'évènement avec nos éléments

b1.onclick = function(){alert('Bouton cliqué')};
d1.onmouseover = function(){this.style.backgroundColor ='orange'};
d1.onmouseout = function(){this.style.backgroundColor='white'};
```

3. Utiliser la méthode addEventListener() pour gérer un évènement

C'est la manière recommandée car c'est la plus flexible et la plus performante. La méthode **addEventListener()** est une méthode de l'interface **EventTarget** qu'on va souvent utiliser avec des objets Element. L'interface Element hérite de l'interface Node qui hérite elle-même de EventTarget.

On va passer deux arguments à cette méthode : le nom d'un évènement qu'on souhaite prendre en charge ainsi que le code à exécuter (qui prendra souvent la forme d'une fonction) en cas de déclenchement de cet évènement. Notez qu'on va par ailleurs pouvoir utiliser la méthode addEventListener() pour réagir plusieurs fois et de façon différente à un même évènement

```
//On sélectionne le premier button et le premier div du document
let b1 = document.querySelector('button');
let d1 = document.querySelector('div');

//On utilise la méthode addEventListener pour gérer des évènements

b1.addEventListener('click', function(){alert('Bouton cliqué')});
d1.addEventListener('mouseover',
function(){this.style.backgroundColor ='orange'});
d1.addEventListener('mouseover', function(){this.style.fontWeight
='bold'});
d1.addEventListener('mouseout',
function(){this.style.backgroundColor='white'});
```

Supprimer un gestionnaire d'évènements avec removeEventListener()

La méthode **removeEventListener()** de l'interface EventTarget permet de supprimer un gestionnaire d'évènement déclaré avec addEventListener().
Il suffiit de passer en argument le type d'évènement ainsi que le nom de la fonction passée en argument de addEventListener().

```
<body>
  <p class="style1">Cet élément possède l'événement "mouseover"</p>
  <button id="btn" onclick="SupprimerEvnt()">Supprimer  evt</button>
  <script>
  let elt = document.querySelector(".style1");
  //Sélectionner l'élement button
  element.addEventListener("mouseover", fct1);
  // Ajouer un évenement de type « mouseover »
  function fct1(){
  alert("Evénement déclenché!");
  }
  function SupprimerEvnt(){
  element.removeEventListener("mouseover", fct1);
  }
  </script>
  </body>
```

Supprimer un gestionnaire d'évènements avec removeEventListener()

La méthode **removeEventListener()** de l'interface **EventTarget** permet de supprimer un gestionnaire d'évènement déclaré avec **addEventListener().** Pour cela, il suffit de passer en argument le type d'évènement ainsi que le nom de la fonction passée en argument de **addEventListener().**

```
//On sélectionne le premier bouton et le premier div du document let

b1 = document.querySelector('button');
let d1 = document.querySelector('div');
function changeCouleur()
{ this.style.backgroundColor ='orange'; }

//On utilise la méthode addEventListener pour gérer des évènements

b1.addEventListener('click', function(){alert('Bouton cliqué')});
d1.addEventListener('mouseover', changeCouleur);
d1.addEventListener('mouseover', function()
{this.style.fontWeight ='bold'});

//On supprime un évènement

d1.removeEventListener('mouseover', changeCouleur);
```

Création d’un DOMMenu Objet

L'objet DOMMenu en HTML représente l' élément <menu>.

Les attributs :

- Label : prend une valeur textuelle, spécifie le label du menu.
- Type : prend l’une des valeurs (list, toolbar, contex). Son rôle est de spécifier le type du menu à afficher.

```
var menuObject = document.createElement("MENU")
```

Remarque: Cet élément n’est plus supporté par les principaux navigateurs

Création d’un DOMMenu Objet

Exemple :

```
<menu type="context" id="monMenu">
<menuitem label="Actualiser" onclick="window.location.reload();"
icon="ico_reload.png"></menuitem>
<menu label="Partager sur...">
<menuitem label="Twitter" icon="ico_twitter.png"
onclick="window.open('//twitter.com/intent/tweet?text=
'+window.location.href);">
</menuitem>
<menuitem label="Facebook" icon="ico_facebook.png"
onclick="window.open('//facebook.com/sharer/sharer.php?
u='+window.location.href);">
</menuitem>
</menu>
<menuitem label="Envoyer cette page"
onclick="window.location='mailto:?body='+window.location.href;">
</menuitem>
</menu>
```

La propagation des évènements en JavaScript

element.addEventListener(événement, fonction de rappel, useCapture);

- Elément : un élément HTML auquel l’événement est attaché.
- Événement : le nom de l'événement.
- Fonction de rappel : la fonction qui va gérer l'événement.
- UseCapture : paramètre booléen facultatif qui prend par défaut la valeur false (spécifie s'il faut utiliser le bouillonnement d'événements ou la capture d'événements).

Les phases de capture et de bouillonnement:

Lorsqu’un évènement se déclenche, celui-ci va en fait naviguer à travers le DOM et passer à travers les différents gestionnaires d’évènement disposés dans le document. On dit également que l’évènement se « propage » dans le DOM.

Cette propagation va se faire selon différentes phases qu’on appelle phase de **capture** et phase de **bouillonnement**.

```
<body>
        <h1>Titre principal</h1>
        <p>Un premier paragraphe</p>
        <div id='d1'>
            <p id='d1p1'>Un paragraphe dans un div</p>
            <p id='d1p2'>Un autre paragraphe dans le div</p>
        </div>
</body>
```

```
//On sélectionne les éléments du document avec les ids d1 et d1p1

let d1 = document.querySelector('#d1');

let d1p1 = document.querySelector('#d1p1');

d1.addEventListener('click', function()

{alert('div cliqué')},true);

d1p1.addEventListener('click', function(){

alert('paragraphe cliqué')});
```

L’évènement **click** va se propager en partant de l’ancêtre le plus lointain du paragraphe, c’est-à-dire en partant de l’élément html puis en traversant les ancêtres de l’élément p un à un (body puis div) jusqu’à arriver à cet élément p.

Une fois arrivé à l’élément p, l’objet évènement va faire le chemin inverse et remonter dans l’arborescence du DOM de cet élément p jusqu’à l’ancêtre le plus lointain, c’est-à-dire traverser l’élément div, puis l’élément body, puis finalement l’élément html.

Cette première phase de descente dans l’arbre du DOM est ce qu’on appelle la phase de **capture.** La deuxième phase de remontée est appelée phase de bouillonnement.

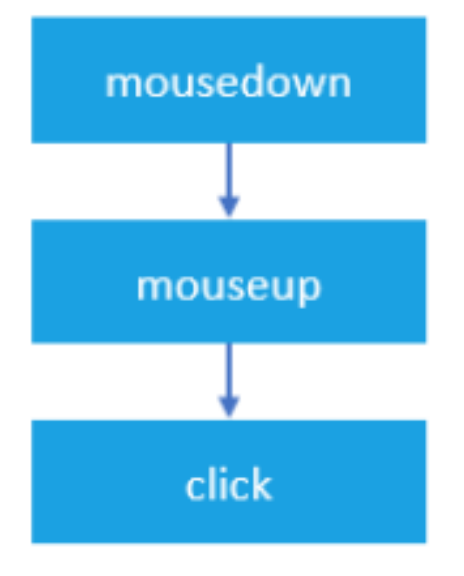
Par défaut, la valeur de cet argument est false ce qui indique que le gestionnaire d’évènement doit ignorer la phase de capture.

Pour lui demander de réagir à la phase de capture, on va donc devoir lui passer la valeur **true**.

Les événements de la souris JavaScript

Les événements de la souris sont déclenchés quand elle interagit avec les éléments de la page. Quand la souris est enfoncée, trois événements se déclenchent dans l'ordre suivant :

- **Mousedown** : se déclenche lorsqu’on appuie sur le bouton gauche de la souris.
- **Mouseup** : se déclenche lorsqu'on relâche le bouton de la souris.
- **Click** : se déclenche lorsqu’un événement **mousedown** et un **mouseup** sont détectés sur l'élément.



L’événements de la souris dblclick

L'événement **dblclick** se déclenche lorsqu’on double-clique sur un élément en utilisant la souris. Il est déclenché par deux événements click.

L'événement dblclick a quatre événements déclenchés dans l'ordre suivant :

1. mousedown
2. mouseup
3. click
4. mousedown
5. mouseup
6. click
7. dblclick

.

Les événements de la souris JavaScript

- **mousemove**: L'événement **mousemove** se déclenche à plusieurs reprises lorsqu'on déplace le curseur de la souris autour d'un élément.

- **mouseover / mouseout:**

✓ L'événement **mouseoverse** déclenche lorsque le curseur de la souris se déplace à l'extérieur d'un élément.

✓ L'événement **mouseout** se déclenche lorsque le curseur de la souris survole un élément, puis se déplace vers autre élément.

- **mouseenter / mouseleave**

✓ L'événement **mouseenter** se déclenche lorsque le curseur de la souris se déplace de l'extérieur à l'intérieur d'un élément.

✓ L'événement **mouseleave** se déclenche lorsque le curseur de la souris se déplace de l'intérieur à l'extérieur d'un élément.

▪ Obtenir les coordonnées de l'écran

- Les propriétés **screenX** et **screenY** de l'événement passées au gestionnaire d'événements de la souris renvoient les coordonnées (dans l'écran) correspondant à l'emplacement de la souris par rapport à l'ensemble de l'écran.
- Les propriétés **clientX** et **clientY** fournissent les coordonnées horizontales et verticales dans la zone cliente de l'application où l'événement de la souris s'est produit.

```
<body>
    <p>Move your mouse to see its position.</p>
    <p id="screen-log"></p>
</body>
```

```
let screenLog = document.querySelector("#screen-log");

document.addEventListener("mousemove", logKey);


function logKey(e) {

  screenLog.innerText = `

    Screen X/Y: ${e.screenX}, ${e.screenY}

    Client X/Y: ${e.clientX}, ${e.clientY}`;

}
```

Les événements du clavier JavaScript

Il existe trois principaux événements du clavier :

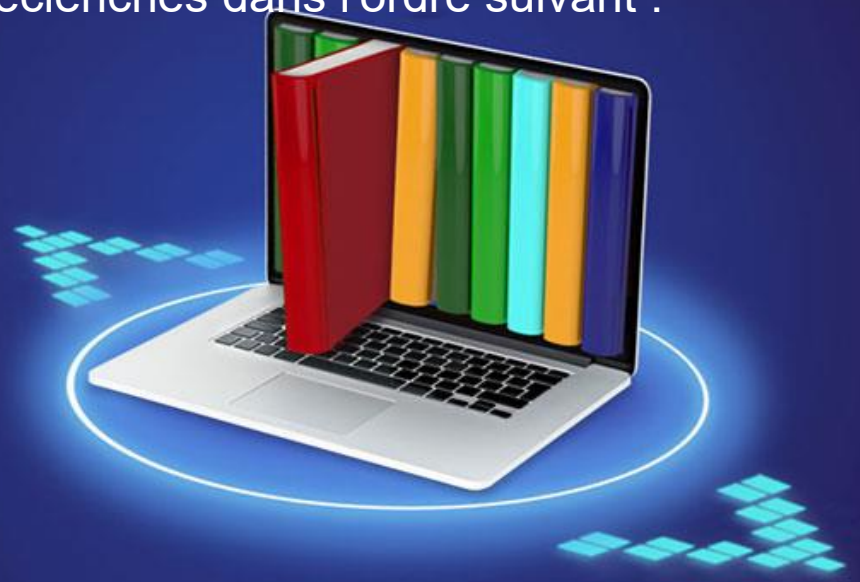
- **Keydown** : déclenché lorsqu'on appuie sur une touche du clavier. Cet événementse déclenche plusieurs fois pendant que la touche est enfoncée.
- **Keyup** : déclenché lorsqu'on relâche une touche du clavier.
- **Keypress** : déclenché lorsqu'on appuie sur les caractères comme a, b, ou c, et pas sur les touches fléchées gauche, etc. Cet événement se déclenche également à plusieurs reprises lorsqu'on maintient la touche du clavier enfoncée.

Lorsqu'on appuie sur une touche du clavier, les trois événements sont déclenchés dans l'ordre suivant :

1. keydown

2. keypress

3. Keyup



Gestion des événements du clavier

Étape 1 : sélectionner l'élément sur lequel l'événement clavier se déclenchera (généralement d'une zone de texte).
Étape 2 : associer à l'élément la méthode **addEventListener()** pour enregistrer un gestionnaire d'événements.

Exemple :
soit la zone de texte suivante : **<input type="text" id="message">**
Dans le code suivant, les trois gestionnaires d'événements seront appelés à l'enfoncement d'une touche de caractère :

```
let msg = document.getDocumentById('#message');
msg.addEventListener("keydown", (event) => {
// traitement keydown
});
msg.addEventListener("keypress", (event) => {
// traitement keypress
});
msg.addEventListener("keyup", (event) => {
// traitement keyup
});
```

Les propriétés de l'événement clavier

L'événement clavier possède deux propriétés importantes :

- **key** : renvoie le caractère qui a été enfoncé.
- **Code** : renvoie le code de touche physique.

Par exemple, le clic sur la touche z du clavier : **event.key** retourne « z » et les « **event.code** » retourne « **KeyZ** »

```
<input type="text" id="message">
<script>
let zone = document.getElementById('message');
zone.addEventListener('keydown', (event) => {
console.log(`key=${event.key},code=${event.code}`);
});
</script>
```

Gérer les éléments d’un formulaire

Soumission d’un formulaire/ Soumission d’un événement

La méthode **submit()** du DOM est utilisée pour soumettre les données du formulaire à l'adresse spécifiée dans l'attribut action. Cette méthode se comporte comme le bouton de soumission du formulaire et ne prend pas de paramètres.

```
<form id="FORM1" method="post" action="code.php">
<label>Nom <input type="text" name="nom"></label><br>
<label>Age <input type="text" name="Age"><label> <br>
<input type="submit" onclick="SubmitForm()" value="SUBMIT">
<input type="button" onclick="ResetForm()" value="RESET">
</form>
<p id="message"></p>
<script>
function SubmitForm() {
document.getElementById("FORM1").submit();
}
function ResetForm() {
document.getElementById("FORM1").reset();
document.getElementById("message").innerHTML="Formulaire
réinitialisé";
}
</script>
```

Empêcher la soumission d'un formulaire

La fonction **preventDefault()** empêche l’exécution de l’action par défaut de l’événement.

```
<form onsubmit="EnvoyerForm(event)">
<input type="text">
<button type="submit">Envoyer</button>
</form>

<script type="text/JavaScript">
function EnvoyerForm(event){
event.preventDefault();
}
</script>
```

Validation d’un formulaire

La validation des données

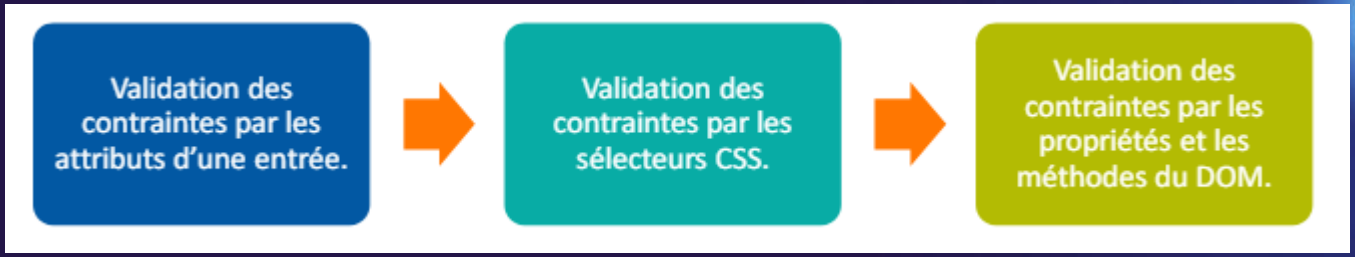
La validation des données consiste à s'assurer que l'entrée de l'utilisateur est conforme aux données attendues. Parmi les types de vérifications à faire on cite :

- ✓ l'utilisateur a-t-il rempli tous les champs obligatoires ?
- ✓ l'utilisateur a-t-il saisi une date valide ?
- ✓ l'utilisateur a-t-il saisi du texte dans un champ numérique ?

On distingue deux types de validation :

- **La validation côté serveur** : effectuée par un serveur Web, une fois que les données sont envoyées au serveur.
- **La validation côté client** : effectuée par un navigateur Web, avant que les données ne soient envoyée à un serveur Web.

HTML5 a introduit un nouveau concept de validation HTML appelé « validation des contraintes » qui est basée sur :



Validation des contraintes HTML

Validation des contraintes en utilisant les sélecteurs CSS

Sélecteur	Description
:disabled	Sélectionner les éléments désactivés
:invalid	Sélectionner les éléments dont la valeur est invalide
:optional	Sélectionner les éléments d'entrée sans attribut "requis" spécifié
:required	Sélectionner les éléments d'entrée avec l'attribut "requis" spécifié
:valid	Sélectionner les éléments d'entrée avec des valeurs valides

Validation des contraintes par les attributs

Attribut	Description
disabled	L'input doit être désactivé
max	Spécifier la valeur maximale d'un élément input
min	Spécifier la valeur minimale d'un élément input
pattern	Spécifier un modèle de chaine (Regex)
required	Saisie obligatoire
type	Spécifier le type d'un élément input

Validation d’un formulaire

les éléments d’un formulaire:

Exemple : Si un champ de formulaire (nom) est vide, la fonction affiche un message et renvoie false pour empêcher la soumission du formulaire

```
<form name="myForm" id = "myForm"  onsubmit="return validerForm()" method="post">
Nom: <input type="text" name="nom">
<input type="submit" value="Submit">
</form>
```

```
function validerForm() {
let x = document.getElementById("myForm" )["nom"].value;
if (x == "") {
alert("Le champ “nom” doit être saisi");
return false;
}
}
```

Validation automatique des formulaires HTML

La validation du formulaire HTML peut être effectuée automatiquement par le navigateur : Si un champ de formulaire (nom) est vide, l'attribut required empêche la soumission du formulaire

```
<form action="/code.php" method="post">
<input type="text" name="nom" required>
<input type="submit" value="Submit">
</form>
```

Les expressions régulières

Une **expression régulière** (ou expression rationnelle) est un élément existant dans la plupart des langages de programmation. En JavaScript, les expressions régulières sont implémentées au travers de l'objet natif RegExp.

Il s'agit d'une chaîne de caractères permettant la recherche et le remplacement de motifs. C'est une expression quasi mathématique composée d'une combinaison de codes représentants des schémas possibles de chaînes.

Quel est le jargon des expressions régulières ?

Terme	Explication
regex/regexp	Une façon courte de dire expression régulière
modèle, motif ou *pattern*	La chaîne correspondant à la recherche voulue
options ou modificateurs	Des propriétés que l'on peut ajouter à une regex pour modifier son fonctionnement
expression	Les parties de la chaîne testée correspondant au modèle
capture	Résultat des méthodes `exec()` ou `match()`

Comment créer un objet regex ?

Il existe deux façons de créer des expressions régulières:

1) La création à l'aide du constructeur :

var regex = new RexExp(modèle, options);

2) La création littérale :

var regex = /modèle/options;

Les crochets :

Les crochets correspondent à des ensembles de caractères. On place à l'intérieur tous les caractères que l'on attend à l'emplacement donné de la chaîne.

Expression	Explication	Exemple
`[abc]`	N'importe quel caractère dans les crochets	`[abcdef]`
`[^abc]`	N'importe quel caractère sauf ceux des crochets	`[^abcdef]`
`[a-z]`	N'importe quel caractère dans l'intervalle	`[d-y]`, `[4-7]`, `[C-M]`
`[^a-z]`	N'importe quel caractère sauf ceux dans l'intervalle	`[^d-y]`, `[^4-7]`, `[^C-M]`

Le caractère tiret **("-")** permet d'indiquer une plage de caractères (par exemple, /[a-z]/ correspond à toutes les lettres de "a" à "z"). Si on doit rechercher le caractère tiret, on doit l'échapper en le faisant précéder d'un antislash ("\").

Modificateur	Description
[abc]	Trouver l'un des caractères entre les crochets ➔ a ou b ou c
[0-9]	Trouver l'un des chiffres entre les crochets ➔ 0, 1, 2, ... ou 9
(x|y)	Trouvez l'une des alternatives séparées par | ➔ x ou y

Quantificateurs d'expressions régulières:

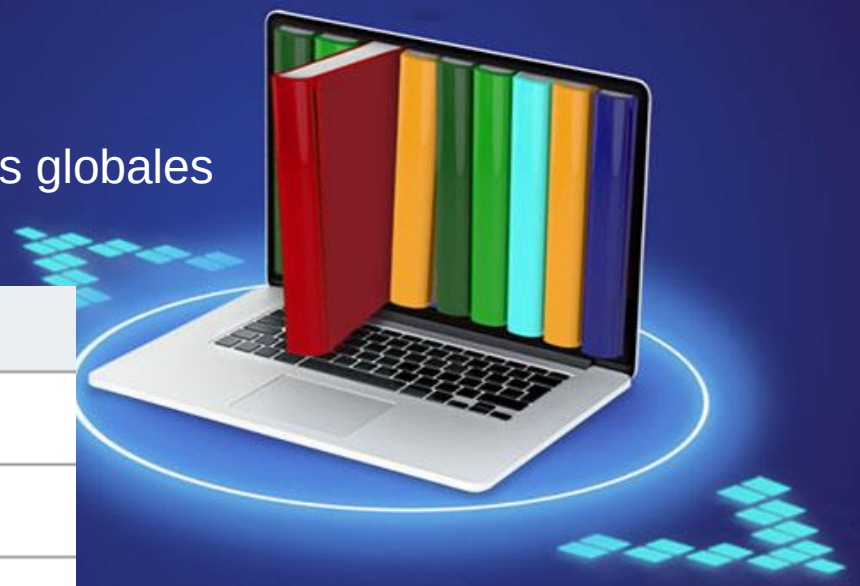
Les quantificateurs définissent les quantités

Expression	Description
n+	Correspond à toute chaîne contenant au moins un caractère n.
n*	Correspond à toute chaîne contenant zéro ou plusieurs occurrences du caractère n.
n?	Correspond à toute chaîne contenant zéro ou une occurrence du caractère n.

Modificateurs d'expressions régulières

Les modificateurs peuvent être utilisées pour effectuer des recherches plus globales insensibles à la casse :

Option	Explication
`g`	Cherche le modèle sur toute la chaîne
`i`	Ignore la casse
`m`	Fait que `^` devient le début d'une ligne et que `$` devient la fin d'une ligne



Les métacaractères:

Les métacaractères sont des caractères ayant une signification particulière :

Expression	Description
\d	Trouver un chiffre
\s	Trouver un caractère d'espacement
\b	Trouver une correspondance au début ou à la fin d'un mot : \bMOT ou MOT\b
\uxxxx	Trouver le caractère Unicode spécifié par le nombre hexadécimal xxxx

Exemple:

```
alert( "\u00A9" ); // ©, le symbole de Copyright

alert( "\u044F" ); // я, la lettre de l'alphabet cyrillique

alert( "\x7A" ); // affiche la lettre Z
```

Les assertions dans les expressions régulières

Les assertions sont utilisées pour indiquer les débuts et les fins des lignes et des mots :

Expression	Explication	Exemple

`/^`	Marque le début de la chaîne	`/^pommes/.test('pommes bleues'); // true`
		`/^pommes/.test('des pommes'); // false`

`$/`	Marque la fin de la chaîne	`/pommes$/.test('des pommes'); // true`
		`/pommes$/.test('pommes bleues'); // false`

- **Les méthodes de l'objet RegExp:**

Les méthodes utilisées dans les expressions régulières sont listées dans le tableau ci-dessous :

Méthode	Description
exec()	Cherche une correspondance (match) d'un pattern dans une chaîne de caractères. Retourne un tableau ou null.
test()	Cherche une correspondance (match) d'un pattern dans une chaîne de caractères. Retourne true ou false.
match()	Retourne null ou un tableau contenant toutes les correspondances.
matchAll()	Retourne un itérateur contenant toutes les correspondances.
search()	Teste une correspondance dans une chaîne de caractères. Retourne -1 ou l'index de la correspondance.
replace()	Cherche une correspondance dans une chaîne et la remplace par une sous-chaîne.
replaceAll()	Recherche toutes les correspondances dans une chaîne et les remplace par une sous-chaînes.
split()	Décompose une chaîne en un tableau de sous-chaînes selon une expression régulière.

Les regex - Utilisation des méthodes search() et replace()

Exemple 1 :

Utilisez une chaîne pour rechercher "JS" dans une chaîne :

```
let texte = "Cours JS";
let pos = texte.search("JS");
console.log(pos); //renvoie 6
```

Exemple 2 :

La méthode replace() remplace une valeur spécifiée par une autre valeur dans une chaîne :

```
let texte = "Cours JS";
let NvTexte = texte.replace("JS", "JavaScript");
console.log(NvTexte); //cours JavaScript
```

Exemple 3 :

Utilisez une expression régulière insensible à la casse pour remplacer « JS » par « JavaScript » dans une chaîne :

```
let texte = "Un cours de JS";
let NvTexte = texte.replace(/js/i, "JavaScript");
console.log(NvTexte);  //Un cours de JavaScript
```

Affiche le même résultat pour la chaine « Un cours de js »

Exemple 4 :

Utilisez une expression régulière pour effectuer une recherche dans une chaîne :

```
let Expr = /[A-Z]/;
let texte = "un cours de JavaScript";
let index = texte.search(Expr);
console.log(index); //12, indice de « J »
```

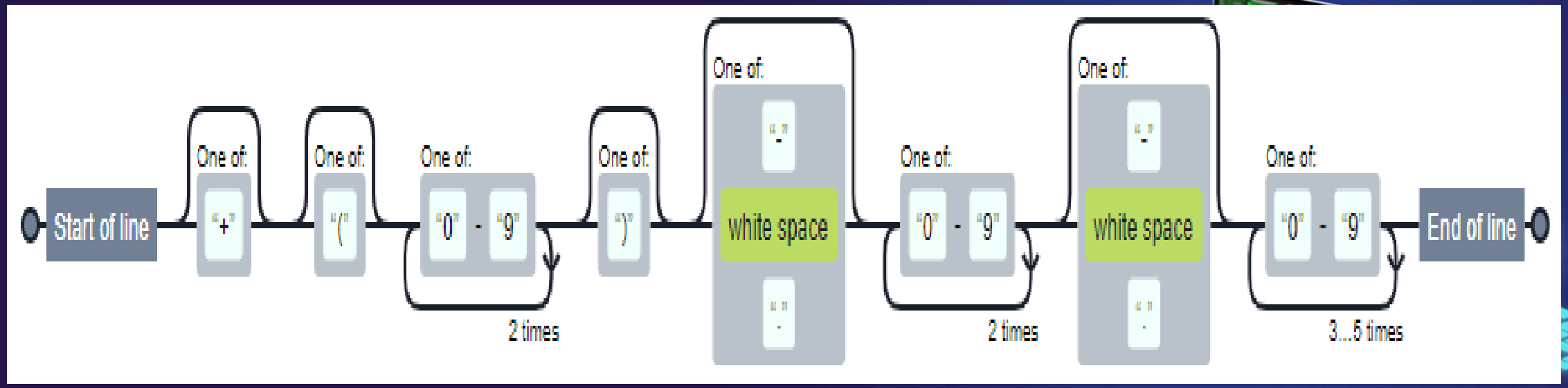
Affiche « -1 » si Expr=/[0-9]/

Les regex - Utilisation des méthodes test(), exec() et match()

Exemple 1 :

Ce code affiche correct si le numéro de téléphone est écrit sous la forme ###-###-####

```
let tel =/^[\+]?[(]?[0-9]{3}[)]?[-\s\.]?[0-9]{3}[-\s\.]?[0-9]{4,6}$/
var OK = tel.exec("+(919)367788755");
if (!OK)
alert('incorrect');
else
alert('ok');
```



Exemple 2 :

Tester si une chaîne contient une sous-chaîne :

```
let text = "un cours JS";
let pattern = /js/;
let result = pattern.test(text);
alert(result); //renvoie True
```

Exemple 3 :

Utilisation de la fonction match()

```
let regex = /cours(?= js)/g;
alert('cours js'.match(regex)); // [ 'cours' ]
alert('cours html'.match(regex)); // null
alert('C\'est le cours js pour vous'.match(regex)); // [ 'cours' ]
alert('C\'est le premier cours du mois.'.match(regex)); // null
```

Manipulation JSON

JSON : définition et caractéristiques

JSON (JavaScript Objet Notation) est un langage léger d’échange de données textuelles. Pour les ordinateurs, ce format se génère et s’analyse facilement. Pour les humains, il est pratique à écrire et à lire grâce à une syntaxe simple et à une structure en arborescence. JSON permet de représenter des données structurées (comme XML par exemple).

Caractéristiques de JSON :

- Facile à utiliser - l'API JSON offre une mise en œuvre avancée des différents types et structures de données, ce qui aide à simplifier les cas d'utilisation.
- Performance et rapidité - JSON consomme très peu de mémoire.
- Outil gratuit - la bibliothèque JSON est open source et gratuite.
- Dépendance - la bibliothèque JSON ne nécessite pas l’utilisation d’une autre bibliothèque pour le traitement.
- JSON est supporté par les navigateurs ;
- JSON est pris en charge par tous les principaux framework JavaScript ;
- JSON permet de transmettre des données structurées à l'aide d'une connexion réseau ;
- JSON est compatible avec les langages de programmation.

JSON : Syntaxe

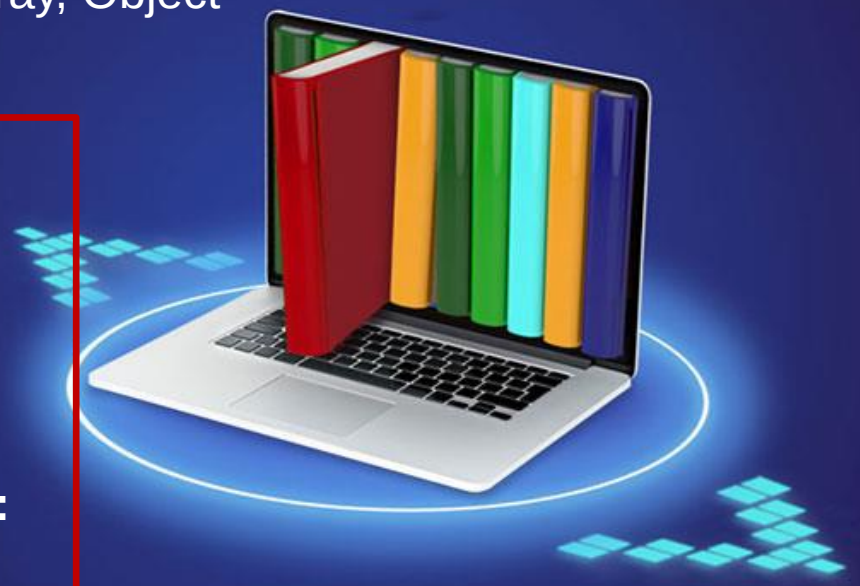
Règles générales de la syntaxe JSON:

- Les données JSON sont écrites entre accolades ;
- Les données sont représentées sous forme de paires de clé – valeur ;
- Les clés doivent être mises entre guillemets (double quotes) ;
- La clé et la valeur doivent être séparées par deux points (:) ;
- La virgule (,) est utilisée pour séparer les données ;
- Les crochets tiennent les tableaux ;
- Les accolades retiennent les objets.

Les types JSON : Number, String (entre guillemets), Boolean, Null, Array, Object

Exemple :

```
{ "nom" : "saidi",
"prenom" : "ali",
"age" :40,
"interets" :null,
"experience" : ["CSS","JS","HTML"],
"addresse" : {"Rue" : "Sidi Maarouf","Ville": "Casablanca","codeP" :
10000 }}
```



Manipulation des données JSON

Pour traiter et afficher les données JSON dans les pages web, on a souvent besoin de les convertir en objets Javascript et vice versa.

- **Analyse syntaxique (parse)** : Convertir une chaîne de caractères en un objet natif.
- **Linéarisation (stringification)** : Convertir un objet natif en chaîne de caractères.

En Javascript, les méthodes utilisées sont :

- **JSON.parse** permet de convertir JSON vers un objet JavaScript.
- **JSON.stringify** permet de convertir des objets JavaScript vers des données JSON.

```
//Création d'un string JSON

let jsonData = '{"nom":"Saidi", "prenom":"Ali"}';

document.write(typeof(jsonData)+'<br>'); //string

//Convertir JSON vers Javascript

let jsObject = JSON.parse(jsonData);

document.write(typeof(jsObject)+'<br>');//object

document.write(jsObject+'<br>'); //[object object]

document.write(jsObject.nom + ""+jsObject.prenom+'<br>'); //Saidi Ali
```

```
//Création d'un objet Javascript

var jsObject = {nom:"Saidi", prenom:"ali"};
document.write(typeof(jsObject)+'<br>'); //Object

//Convertir JavaScript vers JSON
var jsonString = JSON.stringify(jsObject);

document.write(typeof(jsonString)+'<br>');//string
document.write(jsonString);//{"nom":"Saidi","prenom":"ali"}

//remarquer la présence des "" dans les clés
```

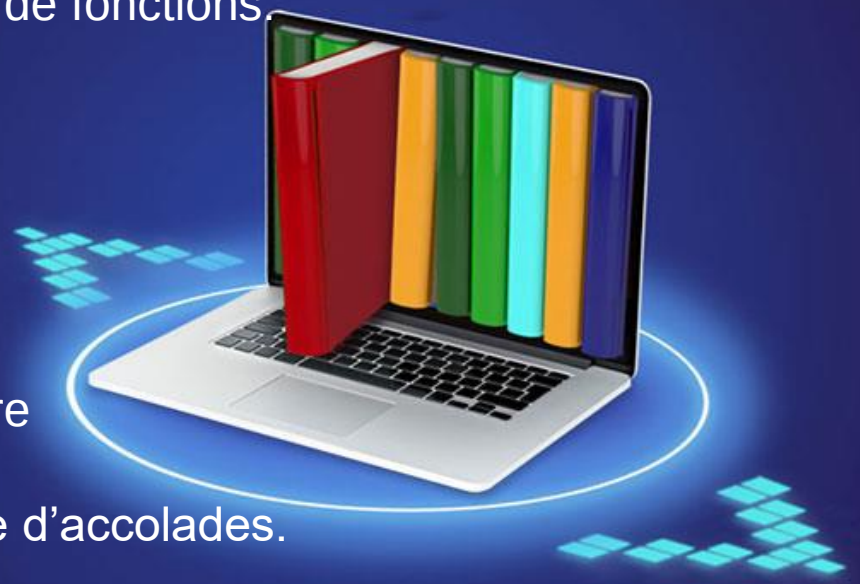
Les expressions lambdas ou Les fonctions fléchées:

Il existe différentes syntaxes permettant de créer une fonction en JavaScript. On peut créer une fonction en utilisant :

- une déclaration de fonction ;
- une expression de fonction ;
- une fonction fléchée ;

Les déclaration de fonctions: on utilise principalement la déclaration de fonctions:

```
function disBonjour(){
   alert('Bonjour');
}
```



On déclare une fonction avec le mot clef **function**, on donne un nom à notre

fonction et on fait suivre ce nom par un couple de parenthèses et un couple d’accolades.

Les expressions de fonctions:

Pour créer une expression de fonction, on utilise une **fonction anonyme** qu'on assigne ensuite à une variable. Pour appeler une fonction créée comme cela, on va pouvoir utiliser la variable comme une fonction, c'est-à-dire avec un couple de parenthèses après son nom.

```
let disBonjour= function(){
    alert('Bonjour');
};
```

Les expressions de fonctions fléchées : syntaxe et intérêts

Les fonctions fléchées sont des fonctions qui possèdent une syntaxe très compressée , ce qui les rend très rapide à écrire. Les fonctions fléchées utilisent le signe => qui leur a donné leur nom à cause de sa forme de flèche.

//Expression de fonction classique :

```
let somme = function(a, b) {
    return a + b;
};
```

//Equivalent en fonction fléchée :

```
let somme = (a, b) => a + b
```

Les fonctions fléchées n’ont pas besoin du couple d’accolades classique aux fonctions pour fonctionner et n’ont pas besoin non plus d’une expression return puisque celles-ci vont automatiquement évaluer l’expression à droite du signe => et retourner son résultat.

N.B: Si une fonction fléchée n’a besoin que d’un argument pour fonctionner, alors on pourra également omettre le couple de parenthèses.

```
//Expression de fonction classique :

let double = function(n){
    return n * 2
}

//Equivalent en fonction fléchée :

let double = n => n * 2;
```

Les méthodes setTimeout() et clearTimeout()

Les méthodes **setTimeout()** et **clearTimeout() de** **L'objet Window** permettent de déclencher une fonction après un laps de temps déterminé.

Syntaxe : nom_du_compteur = setTimeout("fonction_appelée()", temps en milliseconde)

Exemple :

- Lancer la fonction démarrer() après 5 secondes.

setTimeout("demarrer()",5000)

- Pour arrêter le temporisateur avant l'expiration du délai fixé :

clearTimeout(nom_du_compteur) ;

Exemple de seTimeout()

```
<html>
<body>

<h1>La méthode setTimeout() </h1>

<p>Veuillez patientez 5 secondes:</p>

<h2 id="demo"></h2>

<script>
const myTimeout = setTimeout(attendre, 5000);

function attendre() {
  document.getElementById("demo").innerHTML = "C’est parti!"
}
</script>

</body>
</html>
```

Exemple de clearInterval()

```
<body>

<h2>Les méthodes setInterval() and clearInterval()</h2>

<p id="demo"></p>
<button onclick="Stop()">arrêter</button>
<script>
const myInterval = setInterval(temps, 1000);

function temps() {
  const date = new Date();
  document.getElementById("demo").innerHTML =
date.toLocaleTimeString();
}
function Stop() {
  clearInterval(myInterval);
}
</script>
</body>
```

La méthode **setInterval** () appelle une fonction ou évalue une expression à des intervalles spécifiés (en millisecondes). Elle continue d'appeler la fonction jusqu'à ce que la méthode **clearInterval** () soit appelée ou que la fenêtre soit fermée.

```
<body>

<h2>la méthode setInterval() </h2>

<p id="demo"></p>
<script>
setInterval(myTimer, 3000);

function myTimer() {
  const date = new Date();
  document.getElementById("demo").innerHTML =
date.toLocaleTimeString();
}
</script>
</body>
```

Introduction à l’asynchrone en JavaScript

Une première définition des termes « synchrone » et « asynchrone »

Dans la vie de tous les jours, on dit que deux actions sont synchrones lorsqu’elles se déroulent en même temps ou de manière synchronisée. Au contraire, deux opérations sont asynchrones si elles ne se déroulent pas en même temps ou ne sont pas synchronisées.

En informatique, on dit que deux opérations sont synchrones lorsque la seconde attend que la première ait fini son travail pour démarrer. Ce qu’il faut retenir de cette définition est le concept de dépendance (la notion de « synchronisation » dans la première définition donnée de synchrone au-dessus) : le début de l’opération suivante dépend de la complétude de l’opération précédente.

Au contraire, deux opérations sont qualifiées d’asynchrones en informatique lorsqu’elles sont indépendantes c’est-à-dire lorsque la deuxième opération n’a pas besoin d’attendre que la première se termine pour démarrer.

L’importance de l’asynchrone en programmation

Par défaut, le JavaScript est un langage synchrone, bloquant et qui ne s’exécute que sur un seul fil d’éxecutin. Cela signifie que :

- Les différentes opérations vont s’exécuter les unes à la suite des autres (elles sont synchrones) ;
- Chaque nouvelle opération doit attendre que la précédente ait terminé pour démarrer (l’opération précédente est « bloquante ») ;
- Le JavaScript ne peut exécuter qu’une instruction à la fois (il s’exécute sur un thread, c’est-à-dire un « fil » ou une « tache » ou un « processus » unique).
- Cela peut rapidement poser problème dans un contexte Web

Exemple: imaginons qu'une des fonctions ou de boucles prenne beaucoup de temps à s'exécuter. Tant que cette fonction n'a pas terminé son travail, la suite du script ne peut pas s'exécuter (elle est bloquée) et le programme dans son ensemble parait complètement arrêté du point de vue de l'utilisateur.

```
//L'exécution de cette boucle devrait prendre un certain temps
while (x <= 10000000){
   x++;
}
```

Pour éviter de bloquer totalement le navigateur et le reste du script, on aimerait que ce genre d'opérations se déroule de manière asynchrone, c'est-à-dire en marge du reste du code et qu'ainsi le reste du code ne soit pas bloqué.

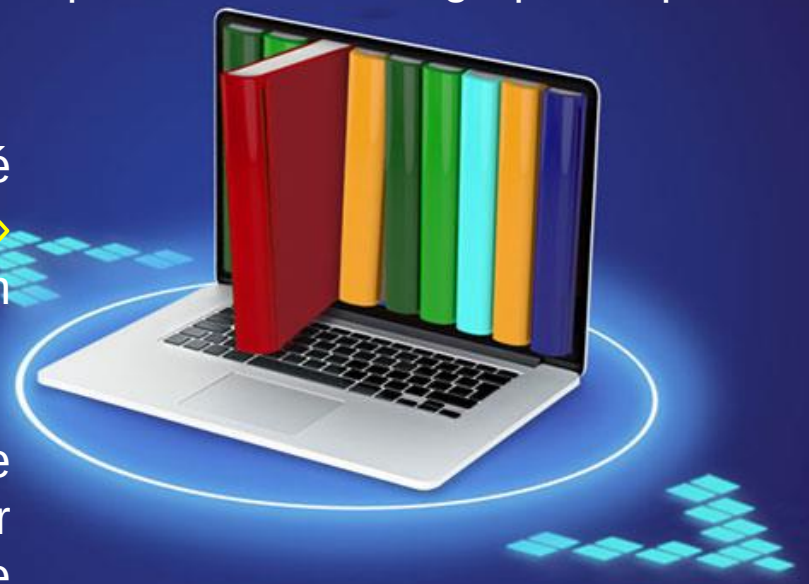
Cela est aujourd'hui possible puisque les machines disposent de plusieurs cœurs, ce qui leur permet d'exécuter plusieurs tâches de façon indépendante et en parallèle et que le JavaScript nous fournit des outils pour créer du code asynchrone.

Les fonctions de rappel : à la base de l'asynchrone en JavaScript

Au cours de ces dernières années les machines sont devenues de plus en plus puissantes et les scripts de plus en plus complexes et de plus en plus gourmands en ressources. Dans ce contexte, il faisait tout à fait sens pour le JavaScript de fournir des outils pour permettre à certaines opérations de se faire de manière asynchrone.

En JavaScript, les opérations asynchrones sont placées dans des files d'attentes qui vont s'exécuter après que le fil d'exécution principal ou la tâche principale ait terminé ses opérations. Elles ne bloquent donc pas l'exécution du reste du code JavaScript.

L'idée principale de l'asynchrone est que le reste du script puisse continuer à s'exécuter pendant qu'une certaine opération plus longue ou demandant une réponse / valeur est en cours. Cela permet un affichage plus rapide des pages et en une meilleure expérience utilisateur.



Le premier outil utilisé en JavaScript pour générer du code asynchrone a été les **fonctions de rappel**. En effet, une fonction de rappel ou « callback » en anglais est une fonction qui va pouvoir être rappelée à un certain moment.

L'idée est de passer une fonction de rappel en argument d'une autre fonction. Cette fonction de rappel va être rappelée à un certain moment par la fonction principale et pouvoir s'exécuter, sans forcément bloquer le reste du script .

Résumé:

Les fonctions JavaScript sont exécutées dans l'ordre où elles sont appelées. Pas dans l'ordre où elles sont définies ;

- Une fonction peut appeler une fonction dans son code, directement ou en utilisant les fonctions de rappel ;
- Une fonction de rappel est une fonction passée en paramètre d'une autre fonction en tant qu'argument ;
- La fonction de rappel est invoquée à l'intérieur de la fonction externe pour exécuter des instructions précises

```
function affichage(s) {
console.log(s);
}
function calcul(num1, num2, myCallback) {
let somme = num1 + num2;
myCallback(somme);
}
calcul(1, 5, affichage);
```

N.B: Ne pas utiliser les parenthèses dans l'appel de la fonction

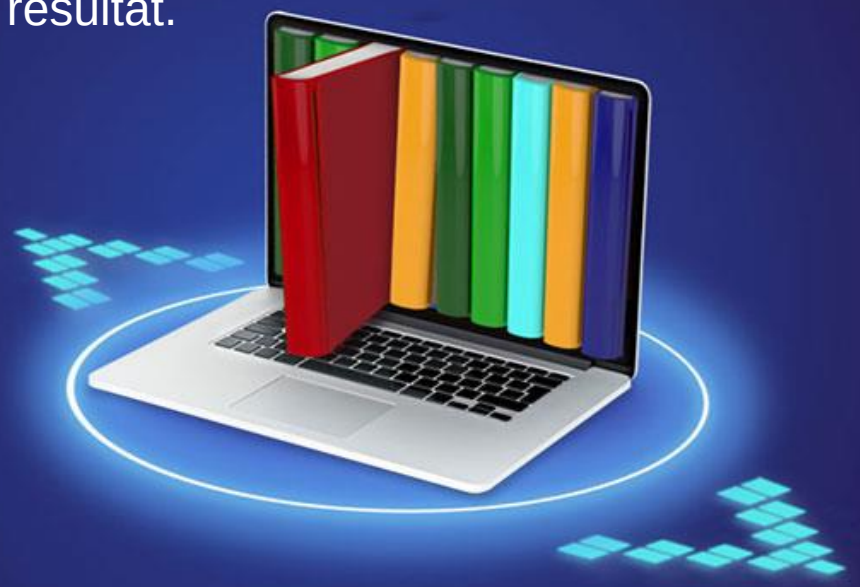
Gestion des exceptions

Lorsqu'une erreur se produit, JavaScript arrête et crée un objet d'erreur (error) avec deux propriétés : nom et un message (varie selon le navigateur) ➔ JavaScript lance une exception, ont dit qu'une erreur est générée.

Pour gérer les exceptions en JavaScript on utilise les mots clés suivants :

- **try** : tester un bloc de code pour les erreurs ;
- **catch** : gérer l'erreur ;
- **throw** : créer des erreurs personnalisées ;
- **Finally** : permet l'exécution de code, après try et catch, quel que soit le résultat.

```
try {
//Bloc du code à exécuter
}
catch(err) {
//Bloc du code qui gère l'exception
}
```



L'instruction throw

L'instruction **throw** permet de créer une erreur personnalisée. L'exception à générer peut être de type String, Number, un Boolean ou un Object

Exemple :

```
if(a > 200)
throw "Too big"; // exception de type string
else
throw 500; // exception de type number
```

L'utilisation de l'instruction **throw** avec **try** et **catch** permet de contrôler le déroulement du programme et de générer des **messages d'erreurs personnalisés**

Exemple de validation d'entrée:

L’utilisateur doit saisir une valeur comprise entre 1 et 5. On distingue 4 cas :

- x est vide : exception levée avec le message « Vide » ;
- x n’est pas un nombre : exception levée avec le message « Ce n’est pas un numéro » ;
- x < 5 : exception levée avec le message « Trop petit » ;
- x >10 : exception levée avec le message « Trop grand ».

```
let x = Number(prompt("Donnez un numéro entre 5 et 10"));
try {
if(x == "") throw "Vide";
if(isNaN(x)) throw "Ce n’est pas un numéro";
x = Number(x);
if(x < 5) throw "Trop petit";
if(x > 10) throw "Trop grand";
}
catch(err)
{
console.log(err);
}
```

L'objet error:

L’objet **error** de JavaScript fournit des informations sur l'erreur quand elle se produit. Cet objet fournit deux propriétés : **name** et **message**.

Valeurs de nom d'erreur :

Nom de l'erreur	Description
EvalError	Erreur dans la fonction eval()
RangeError	Nombre hors limite
ReferenceError	Référence inconnue
SyntaxError	Erreur de syntaxe
TypeError	Une erreur de type s'est produite
URIError	URI incorrecte

Exemple :

```
let x = 5;
try {
  x = y + 1;  // y n’est pas référencé en mémoire
}
catch(err) {
  console.log(err.name);
}

Output : ReferenceError
```

Manipuler JQUERY

JQuery est une bibliothèque JavaScript open-source inventée par John Resig en 2006. Cette bibliothèque est compatible avec les différents navigateurs web.

Le rôle de **JQuery est de simplifier l'utilisation de JavaScript et la manipulation du DOM** sur les sites Web. En effet, les traitements nécessitant l'écriture de nombreuses lignes de code JavaScript sont encapsulés dans des méthodes appelées dans une seule ligne de code.

La bibliothèque jQuery contient les fonctionnalités suivantes :

- Manipulation du HTML et DOM
- Manipulation du CSS
- Méthodes d'événement HTML
- Effets et animations
- AJAX

JQUERY : Installation

On peut utiliser deux manières pour utiliser JQuery dans les pages html :

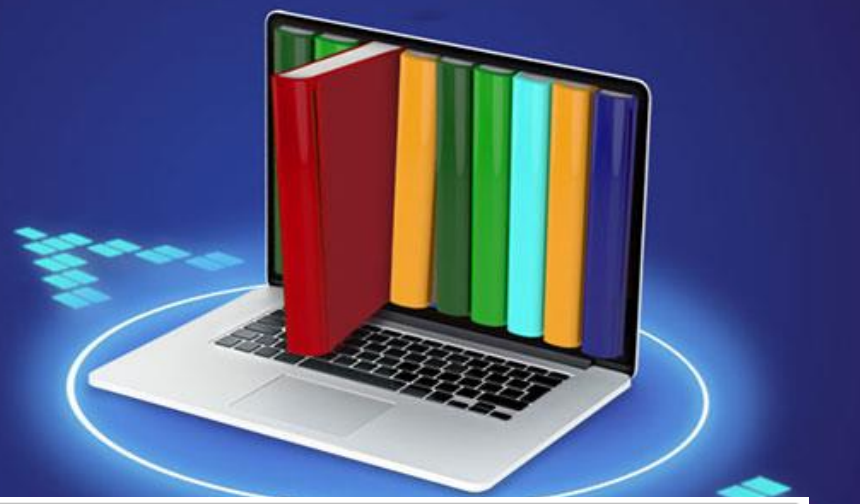
Méthode 1 : Téléchargement de Jquery

Il existe deux versions de JQuery téléchargées depuis le site jQuery.com.

- Version de production : version compressée pour la phase de l'hébergement.
- Version de développement : version non compressée et lisible, pour la phase de développement et de tests.

La bibliothèque jQuery téléchargée correspond à un fichier JavaScript. Pour l'utiliser il faut le référencer avec la balise <script> dans la section <head> :

```
<head>
    <script src="jquery-3.5.1.min.js"></script>
</head>
```



Méthode 2 : JQuery via CDN (Content Delivery Network)

On peut inclure JQuery à partir d'un CDN (Content Delivery Network) sans besoin de télécharger les fichiers.
Exemple d'utilisation de JQuery hébergé par Google :

```
<head>
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>
</head>
```

Syntaxe de JQUERY

La syntaxe de JQuery est basée sur les sélecteurs : **$(sélecteur). action ()**

- $() est un raccourci vers la fonction jQuery() qui trouve des éléments dans une page et crée des objets jQuery qui référencent ces éléments.

Par exemple : $('p') et jQuery('p') : sélectionne tous les éléments p (paragraphe).

- Sélecteur correspond à sélecteur CSS pour interroger (ou rechercher) des éléments HTML.
- Action correspond à une action à effectuer sur le(s) élément(s) sélectionnés.

Exemples :

- $(this).hide() : masque l'élément courant.
- $("p").hide() : masque tous les éléments <p>.
- $(".test").hide() : masque tous les éléments avec class="test".
- $("#test").hide() : masque l'élément avec id="test"

L'événement ready pour le document

Les méthodes jQuery se trouvent dans l’événement ready de l’objet document qui permet d'empêcher l'exécution du code jQuery avant la fin du chargement du document.

```
$(document).ready(function(){
// Méthodes jQuery...
});
```

Exemples d'actions qui peuvent échouer si les méthodes sont exécutées avant le chargement complet du document :

- Masquer un élément qui n'est pas encore crée.
- Obtenir la taille d'une image qui n'est pas encore chargée.

Fonctions essentielles et chaînage

Les instructions JQuery peuvent être écrites soit l'une après l'autre (dans des lignes différentes) ou en utilisant la technique de chaînage.

Le chaînage permet d'exécuter plusieurs actions JQuery l'une après l'autre (dans la même ligne), sur le même élément.

Exemple : Les méthodes css(), slideUp() et slideDown() sont appelées sur le paragraphe identifié par l'ID « p1 ». Ainsi, l'élément "p1" devient d'abord rouge, puis il glisse vers le haut, puis vers le bas :

Syntaxe 1 :

```
$("#p1").css("color", "red").slideUp(2000).slideDown(2000);
```

Syntaxe 2 :

```
$("#p1").css("color", "red")
  .slideUp(2000)
  .slideDown(2000);
```

Comportement des liens hypertextes avec jquery

Désactiver un lien href en JQUERY:

La méthode **removeAttr()** de JQuery permet de supprimer l'attribut « href » du lien, qui devient non cliquable.

```
<head>
   <meta charset="utf-8">
   <title>Désactiver un lien</title>
   <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>
   <script type="text/javascript">
     $(document).ready(function(){
       $(".menu a").each(function(){
         if($(this).hasClass("disabled")){
           $(this).removeAttr("href");
         }
       });
     });
   </script>
 </head>
```

```
<body>
  <ul class="menu">
    <li><a href="https://www.ofppt.ma/">Lien1</a></li>
    <li><a href="https://www.ofppt.ma/">Lien2</a></li>
    <li><a href="https://www.ofppt.ma/" class="disabled">Lien3</a></li>
  </ul>
</body>
```

Activer un lien href en JQUERY

La méthode **attr()** de JQuery permet d'jouter l'attribut « href » à un lien

```html
<head>
   <meta charset="utf-8">
   <title>Désactiver un lien</title>
   <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>
   <script type="text/javascript">
     $(document).ready(function(){
       $(".menu a").each(function(){
         if($(this).hasClass("disabled")){
           $(this).attr("href","https://www.google.com/");
         }
       });
     });
   </script>
 </head>
```

```html
<body>
  <ul class="menu">
    <li><a href="https://www.ofppt.ma/">Lien1</a></li>
    <li><a href="https://www.ofppt.ma/">Lien2</a></li>
    <li><a class="disabled">Lien3</a></li>
  </ul>
</body>
```

Les événements en JQUERY

Les événements DOM ont une méthode jQuery équivalente.

Exemple 1 : La méthode click()

Attribuer un événement de clic et une fonction à tous les paragraphes d'une page. La fonction masque l'élément cliqué.

```
$("p").click(function(){
    // actions de l'évènement
        $(this).hide();
  });
```

Exemple 2 : La méthode dbclick()

Attribuer un événement de double clic et une fonction à tous les paragraphes d'une page. La fonction est exécutée lorsque l'utilisateur double-clique sur le paragraphe.

```
$("p").dblclick(function(){
    $(this).hide();
  });
```

Exemple :

```
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
  $("#hide").click(function(){
    $("p").hide();
  });
  $("#show").dblclick(function(){
    $("p").show();
  });
});
</script>
</head>
<body>
<p>If you click on the "Hide" button, I will disappear.</p>
<button id="hide">Hide</button>
<button id="show">Show</button>
</body>
</html>
```

Exemple 3 : La méthode mouseenter()

```
$("#p1").mouseenter(function(){
    alert("Vous êtes sur le paragraphe p1!");
  });
```

Exemple 4 : La méthode mouseleave()

```
$("#p1").mouseleave(function(){
    alert("vous avez quitté le paragraphe p1!");
  });
```

Exemple :

```
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
  $("p").mouseenter(function(){
    $("p").css("background-color", "yellow");
  });
  $("p").mouseleave(function(){
    $("p").css("background-color", "lightgray");
  });
});
</script>
</head>
<body>

<p>Move the mouse pointer over this paragraph.</p>

</body>
</html>
```

Intérêt des plugins

Intégration de plugins existants:

- Un plugin est un code écrit dans un fichier JavaScript standard. Il fournit des méthodes jQuery utiles qui peuvent être utilisées avec les méthodes de la bibliothèquejQuery.

- L'installation des modules JQuery additionnels, appelés JQuery plugins permet d'étendre les fonctionnalités offertes par JQuery et gagner en termes de rapidité du développement en réutilisant des codes existants.

- Il existe de nombreux sites proposant des plugins jQuery. Parmi ces sites, on peut utiliser le site officiel https://jquery.com/plugins.

- Pour intégrer un plugin dans une page HTML, il faut télécharger le plugin à partir du site dédié puis le référencer dans la page HTML.

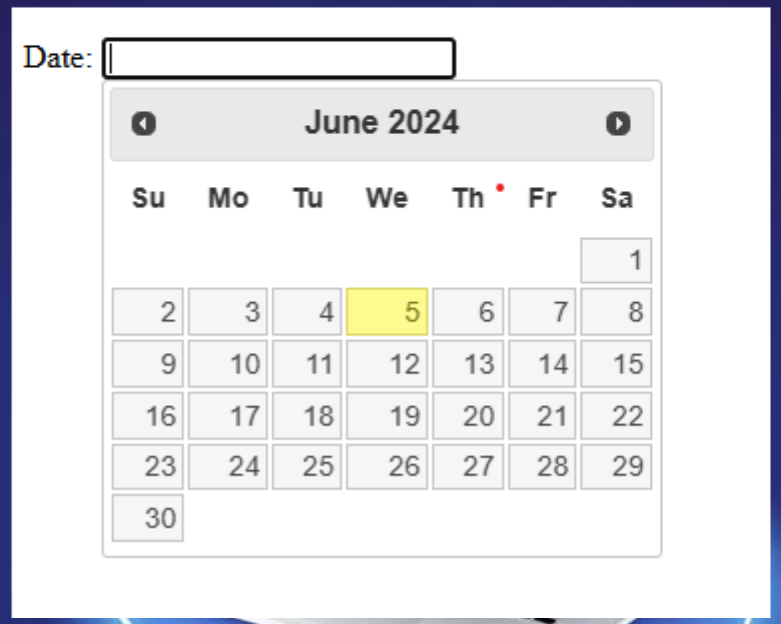
Liste des plugins

Intégrer le plugin jquery.plug-in.js téléchargé dans le fichier HTML :

```
<script src = "jquery.plug-in.js" type =
"text/javascript"></script>
```

Utilisation de plugins existants

```
<html lang="en"><head>
<link rel="stylesheet" href="https://code.jquery.com/ui/1.13.3/themes/base/jquery-ui.css">
<script src="https://code.jquery.com/jquery-3.7.1.js"></script>
  <script src="https://code.jquery.com/ui/1.13.3/jquery-ui.js"></script>
  <script>
  $( function() {
    $( "#datepicker" ).datepicker();
  } );
  </script>
</head>
<body>

<p>Date: <input type="text" id="datepicker"></p>

</body></html>
```



Qu'est-ce qu'AJAX ?

Introduction à AJAX:

AJAX signifie **Asynchronous JavaScript and XML**. L'AJAX n'est pas un langage de programmation mais correspond plutôt à un ensemble de techniques utilisant des technologies diverses pour envoyer et récupérer des données vers et depuis un serveur de façon asynchrone, c'est-à-dire sans avoir à recharger la page

C'est une technologie basée sur :

- **XMLHttpRequest** est un objet navigateur prédéfini (un objet disponible dans tous les navigateurs par défaut) qui nous permet d'effectuer des requêtes HTTP en utilisant du JavaScript.
- JavaScript et DOM HTML (pour afficher les données).

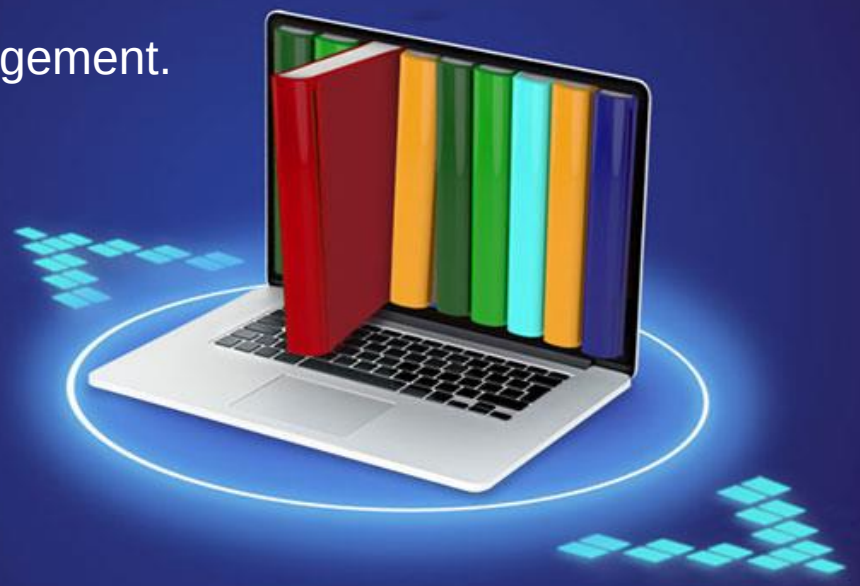
AJAX permet de :

- Lire les données d'un serveur web (après le chargement d'une page web) ;
- Mettre à jour une page web sans la recharger ;
- Envoyer les données à un serveur web .

AJAX, histoire et présentation

La première formalisation du terme **AJAX** en tant que tel date de 2005 mais les techniques utilisées ont commencé à être mises en place dès la fin des années 1990. A cette époque, la plupart des sites Web étaient entièrement conçus à base de HTML et la moindre action de l'utilisateur (envoi ou demande de données) résultait par le chargement d'une nouvelle page envoyée par le serveur. Ce processus était inefficace, lent, et peu agréable pour l'utilisateur.

C'est à cette époque où les développeurs ont commencé à développer le chargement de données asynchrone qui a débouché sur l'objet **XMLHttpRequest** et sur l'AJAX qui l'utilise largement.



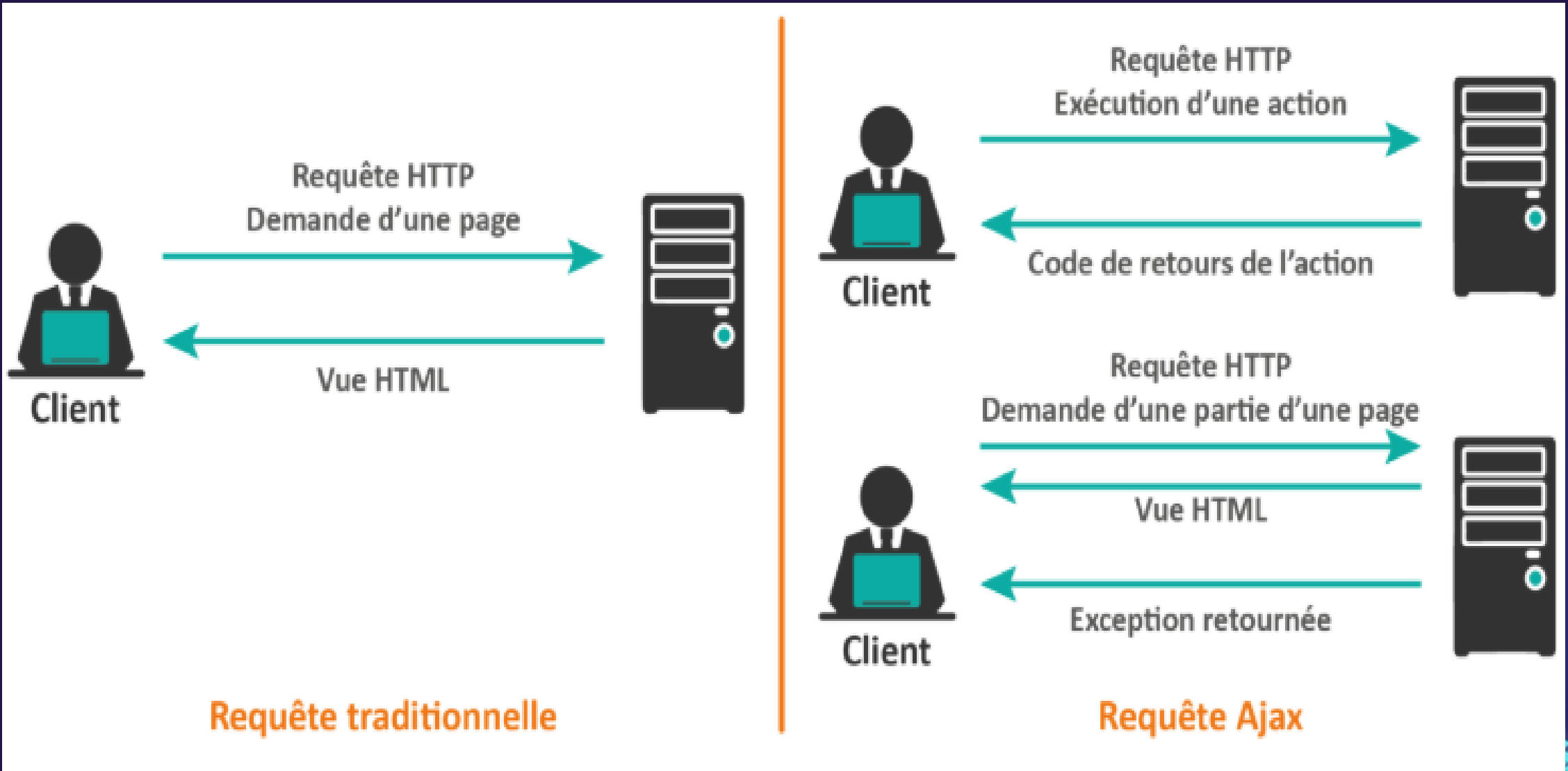
L'AJAX permet d' envoyer et récupérer des données d'un serveur de manière asynchrone (en arrière-plan) sans interférer avec l'affichage et le comportement de la page existante. Grosso-modo, l'AJAX nous permet de modifier de manière dynamique le contenu d'une page, c'est-à-dire sans qu'il soit nécessaire de recharger l'intégralité de la page.

A sa création, l’AJAX utilisait les technologies suivantes qui lui ont donné son nom :

- Le XML pour l’échange de données avec le serveur ;
- L’objet XMLHttpRequest pour la communication asynchrone ;
- Le JavaScript pour afficher les données de manière dynamique et permettre à l’utilisateur d’interagir avec les nouvelles informations ;
- Le HTML et le CSS pour la présentation des données.

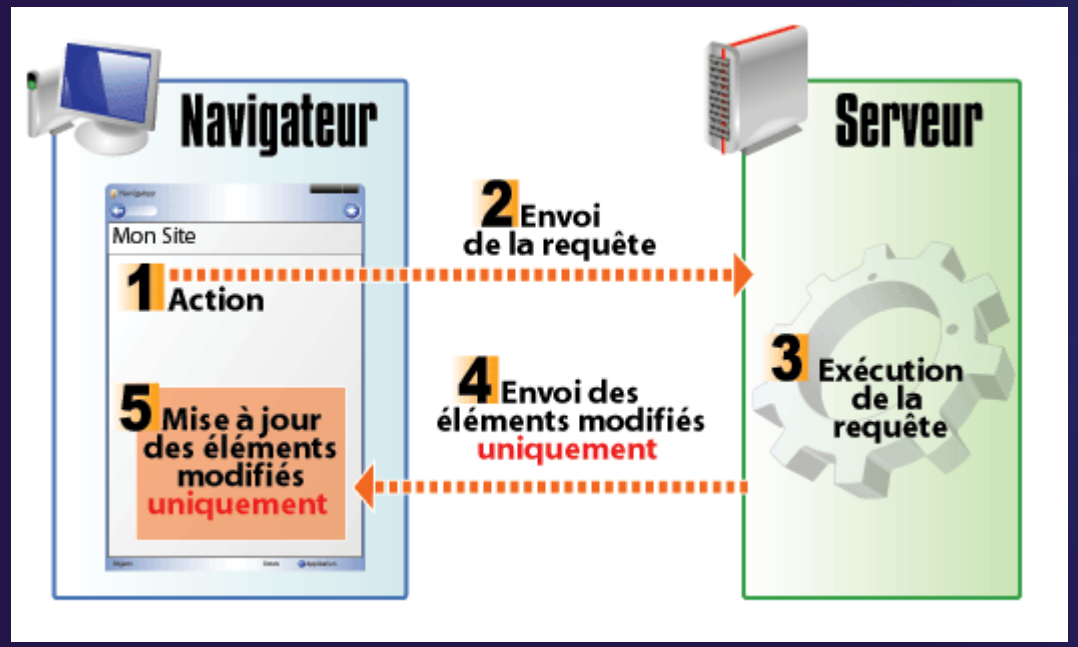
Aujourd’hui, le XML a été largement délaissé au profit du JSON qui est une notation qui permet d’échanger des données relativement simplement tandis que l’objet **XMLHttpRequest** est lentement en train de laisser sa place à la nouvelle **API Fetch.**

“L’AJAX” est aujourd’hui un terme générique utilisé pour désigner toute technique côté client (côté navigateur) permettant d’envoyer et de récupérer des données depuis un serveur et de mettre à jour dynamiquement le DOM sans nécessiter l’actualisation complète de la page.


Requête HTTP
Demande d'une page
Client
Vue HTML
Requête traditionnelle
Requête HTTP
Exécution d'une action
Client
Code de retours de l'action
Requête HTTP
Demande d'une partie d'une page
Vue HTML
Client
Exception retournée
Requête Ajax

L'objet XMLHttpRequest

- L’objet **XMLHttpRequest** de la technologie AJAX est un objet qui permet d’envoyer des requêtes HTTP au serveur, de recevoir des réponses et de mettre à jour la page Web.
- En mode asynchrone, cette mise à jour se réalise sans devoir recharger la page et donc de façon totalement transparente pour l’utilisateur.
- L’objet **XMLHttpRequest** est basé sur le principe d’échange de données entre le client (la page web) et le serveur (sur lequel se trouve la page ou la source de données à laquelle la page Web veut accéder)



Fonctionnement de l’objet XMLHttpRequest

L’objet **XMLHttpRequest (XHR)** est créé par le moteur JavaScript du navigateur pour initier des requêtes (demandes) HTTP à partir de l’application vers le serveur, qui à son tour renvoie la réponse au navigateur.

- **Utiliser la propriété readyState et le gestionnaire onreadystatechange:**

Une autre méthode consiste à observer la valeur de la propriété **readyState** de l’objet **XMLHttpRequest** pour déterminer l’état d’avancement de la requête.

Les valeurs possibles de **readyState** sont les suivantes :

Si on choisit d’utiliser **readyState** pour suivre l’avancement de notre requête, on va alors utiliser un gestionnaire d’événement **onreadystatechange** qui va être appelé dès que la valeur de **readyState** change.

On passe une fonction anonyme à ce gestionnaire pour gérer la réponse. Dans cette fonction anonyme, on teste déjà que la valeur de **readyState** est bien égale à **4** ou à **DONE**. Cela signifie qu’on a reçu la réponse du serveur dans son intégralité et qu’on va donc pouvoir l’exploiter.

Valeur	Etat	Description
0	UNSENT	Le client a été créé mais open() n’a pas encore été appelée
1	OPENED	open() a été appelée
2	HEADERS_RECEIVED	send() a été appelée et l’en-tête et le statut sont disponibles
3	LOADING	Les données sont en train d’être téléchargées
4	DONE	L’opération est complète

Créer un objet XMLHttpRequest

Syntaxe de création d'un objet XMLHttpRequest :

```
variable = new XMLHttpRequest();
```

Méthodes d'objet XMLHttpRequest :

Méthode	Description
new XMLHttpRequest()	Créer un nouvel objet XMLHttpRequest
abort()	Quitter la requête courante
getAllResponseHeaders()	Retourner toutes les informations du header
getResponseHeader()	Retourner une information spécifique du header
open(method,url,async,user,psw)	Specifier la requête : • **method**: GET ou POST • **url**: emplacement du fichier • **async**: true (asynchrone, c'est à dire JavaScript n'a pas à attendre la réponse du serveur) ou false (synchrone) • **user**: nom d'utilisateur (optionel) • **psw**: mot de passe (optionnel)
send()	Envoyer la requête au serveur (utilisé dans les requêtes GET)
send(string)	Envoyer la requête au serveur (utilisé dans les requêtes POST)

Propriétés de l'objet XMLHttpRequest:

Propriété	Description
onreadystatechange	Définit une fonction à appeler lorsque la propriété **readyState** change
readyState	Contient le statut de XMLHttpRequest
responseText	Renvoie les données de réponse sous forme de chaîne
responseXML	Renvoie les données de réponse sous forme de données XML
status	Renvoie le numéro d'état d'une requête 200: "OK" 403: "Forbidden" 404: "Not Found"
statusText	Renvoie le texte d'état (par exemple "OK" ou "Not Found")

Fonctionnement d'AJAX: Envoyer une demande à un serveur

Les méthodes **open()** et **send()** de l'objet **XMLHttpRequest** permettent d'envoyer une requête à un serveur

Syntaxe :

```
open(method, url, async)
```

Exemple 1 :

```
let xhttp = new XMLHttpRequest();
xhttp.open("GET", "code.php", true);
xhttp.send();
```

Exemple 2 : Envoi des données dans la requête GET

```
let xhttp = new XMLHttpRequest();
xhttp.open("GET", "code.php?prenom=Hassan&nom=FILALI", true);
xhttp.send();
```

Exemple 3 :

```
let xhttp = new XMLHttpRequest();
xhttp.open("POST", "code.php", true);
xhttp.setRequestHeader("Content-type", "application/x-www-form-
urlencoded");
xhttp.send("prenom=Hassan&nom=FILALI");
```

La propriété onreadystatechange

La propriété **onreadystatechange** de l'objet **XMLHttpResponse** permet de définir une fonction à exécuter quand une réponse est reçue.
La propriété **onreadystatechange** exécute la fonction chaque fois que **readyState** change de valeur : Lorsque readyState est 4 et status est 200, la réponse est prête.

Exemple 1 :

```
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("demo").innerHTML = this.responseText;
    }
  };
  xhttp.open("GET", "ajax_info.txt", true);
  xhttp.send();
```

Exemple 2 :

```
function loadDoc() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            document.getElementById("demo").innerHTML =
            this.responseText;
        }
    };
    xhttp.open("GET", "ajax_info.txt", true);
    xhttp.send();
}
```

Utilisation d'une fonction Callback

- Une fonction de rappel est une fonction passée en paramètre à une autre fonction.
- Les fonctions de rappel sont utilisées quand plusieurs tâches AJAX doivent être exécutées dans un site web : il faut créer une fonction pour exécuter l'objet **XMLHttpRequest** et une fonction de rappel pour chaque tâche AJAX.
- L'appel de la fonction doit contenir l'URL et la fonction à appeler lorsque la réponse est prête

Exemple

```html
<div id="demo">

<h1>Objet XMLHttpRequest</h1>

<button type="button"
onclick="loadDoc('ajax_info.txt', myFunction)">
Change Content
</button>
</div>

<script>

</script>
```

```js
function loadDoc(url, cFunction) {
  var xhttp;
  xhttp=new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      cFunction(this);
    }
  };
  xhttp.open("GET", url, true);
  xhttp.send();
}
function myFunction(xhttp) {
  document.getElementById("demo").innerHTML =
  xhttp.responseText;
}
```

Exemple : testajax.html

```
<html>
<head>
    <meta charset="UTF-8">
    <title>Document</title>
    <style>table, th, td { border: 1px solid;}</style>
</head>
<body>
    <h1 style="margin:20px">Exemple Ajax</h1>
    <button id="list" style="margin:20px">Afficher la liste des utilisateurs</button>
    <div id="tab" style="margin:20px; display: none;">
      <h2>Liste des utilisateurs</h2>
        <table>
          <tr>
            <td>Prénom</td>
            <td>Nom</td>
            <td>Email</td>
            <td>Teléphone</td>
            <td>Site Web</td>
            <td>Entreprise</td>
            <td>Adresse</td>
          </tr>
```

Exemple : testajax.html => Suite

```
<tr>
        <td class="name"></td>
        <td class="username"></td>
        <td class="email"></td>
        <td class="phone"></td>
        <td class="website"></td>
        <td class="company"></td>
        <td class="address"></td>
      </tr>

    </table>
  </div>
  <script src="https://code.jquery.com/jquery-3.6.0.js"></script>
```

Exemple : testajax.html => Suite

```
 <script>
 $('#list').click(() => {
//créer une nouvelle requête
let xhr = new XMLHttpRequest();
//ouvrir la requête
xhr.open('get', 'ajax.json', true);
 //requête arrivée
 xhr.onload = () => {
    //convert ir string en fichier json object
    let response = JSON.parse(xhr.response);
    $('#tab').show();
    $('.name').html(response.name);
    $('.username').html(response.username);
    $('.email').html(response.email);
    $('.phone').html(response.phone);
    $('.website').html(response.website);
    $('.company').html(response.company);
    $('.address').html(response.address);
};
//Gestion des erreurs
xhr.onerror = () => { alert("Echec") }
//Envoyer la requête
xhr.send()
})
  </script></body></html>
```

Exemple : ajax.json


```
{
  "id": 1,
  "name": "Cours JS",
  "username": "DEV 103",
  "email": "formationdev@ofppt.ma",
  "phone": "05-36-68-02-45",
  "website": "ofppt.ma",
  "company": "OFPPT",
  "address": "Adresse de test"
}
```

Exercice 1: 24pts

1) Ecrire le code html permettant de créer l'interface suivante: **3pts**

Choisir un article : Les billes Quantité : Ajouter

Référence	Quantité	Prix	Action

Total à payer:

La liste articles contient les éléments suivants: *Les billes, La play doh, Ecran magique, Barbie*

Chacun de ces articles possède un prix comme suit:

Article	Prix
Les billes	60
La play doh	100
Ecran magique	230
Barbie	250

Office de la Formation Professionnelle et de la promotion du travail
Service contrôle qualité
Direction Régionale de l'Oriental

2) L'utilisateur pourra choisir un article, indiquer la quantité à acheter, puis clique sur le bouton **Ajouter** pour exécuter une fonction **Ajouter()** permettant d'ajouter l'article au panier.**10pts**
On doit afficher le prix de chaque article et la quantité, puis un bouton **supprimer**, comme dans l'illustration :

Choisir un article : Ecran magique Quantité : 2 Ajouter

Référence	Quantité	Prix	Action
La play doh	1	100	Supprimer
Barbie	2	230	Supprimer
Ecran magique	2	250	Supprimer

Total à payer:1060DH

A chaque ajout d'un article, le montant à payer doit être calculé et affiché:

Dans cet exemple: 100*1 + 230*2 + 2* 250 =1060 **5pts**

3) Lorsqu'on clique sur le bouton **supprimer**, la ligne correspondante à l'article acheté **va être supprimée du panier. Et le total à payer doit être diminué** **6pts**

Choisir un article : Ecran magique Quantité : 2 Ajouter

Référence	Quantité	Prix	Action
Barbie	2	230	Supprimer
Ecran magique	2	250	Supprimer

Total à payer:960DH

Exercice 2: 16pts . . .

1) Créer le formulaire suivant: **2pts**

Nom : *

Prénom : *

Genre : ○ F ○ M

Ville: Rabat

Année de naissance :

Langage préférée: * ☐ C# ☐ Python ☐ Java

Mot de passe : *

Confirmation du mot de passe :

Quel est le résulat de : 8+9=

Valider Effacer

Service contrôle qualité
Direction Régionale de l'Oriental

Indication sur **Quel est le résultat de 8+9:**

Les valeurs 8 et 9 sont générées **aléatoirement**. Donc vous devez générer deux chiffres entre 0 et 9 de manière aléatoire. **1pt**

2) Lorsqu'on clique sur le bouton valider, il faut:

   a) Vérifier que les champs marqués avec * sont saisie, sinon afficher un message en bas du formulaire, **"vous devez saisir les champs qui sont obligatoires" et leur bordure sera en rouge (3pts)**

   b) Le mot de passe composé d'au moins 5 caractères (lettre et/ou chiffres et/ou _) se terminant obligatoirement par # .
   Si le mot de passe n'est pas correcte, afficher un message **"mot de passe n'est pas valide" et sa bordure en rouge (3pts)**

   c) Le champ confirmation du mot de passe doit être **égal** au mot de passe, sinon afficher un message **"le mot de passe et la confirmation ne se correspondent pas" et la bordure du mot de passe et la confirmation seront en rouge (2pts)**

d) L'utilisateur doit saisir le résultat de l'addition demandé, si la valeur donnée par n'est pas correcte, afficher le message **"la somme est incorrecte"** **(2pts)**

e) Lorsque les données sont saisies, on doit afficher un message en bas du formulaire comme suit:

> ***Mr/Mme Nom prénom*** vous êtes âgé de : ***x*** ans, vous habitez ***ville***. *Vos langages préférés sont:* ***Langage 1*** et ***Langage 2*** ...

Si le genre est F afficher Mme, si le genre est M afficher Mr. **(3pts)**

Exercice:

Créer une page html avec une feuille de style associée.

Créer un formulaire avec les champs suivants et les labels associés :

- état civil (M. ou Mme) ;
- nom ;
- prénom ;
- nom complet (désactivé) ;
- date de naissance ;
- niveau en JavaScript ;
- couleur préférée ;
- mot de passe ;
- mot de passe (confirmation) ;
- afficher le mot de passe (oui ou non) ;
- description (texte) ;
- bouton réinitialiser (pour réinitialiser le formulaire) ;
- bouton envoyer (pour envoyer le formulaire).

Formulaire d'inscription

Etat civil M.

Nom

Prénom

Nom complet

Date de naissance dd/mm/yyyy

Niveau en javascript

Couleur preferee

Mot de passe

Mot de passe (confirmation)

Afficher mot de passe

Description

Reinitialiser Envoyer

Travail à faire:

1.Faire en sorte que le champ « date de naissance » ne puisse contenir que des dates entre aujourd'hui et il y a 100 ans.
2. A l'aide du CSS, créer une mise en page similaire à celle de la figure ci-dessous. `
A partir de maintenant, on utilisera jQuery pour les questions qui suivent. `
3. Le champ « nom complet » doit changer de valeur en fonction de l'état civil, du nom et du prénom (ex. si l'utilisateur a entré respectivement « M. », « Dupont » et « Toto », il faudra afficher « M. Dupont Toto »).
4. La valeur numérique du champ « niveau en JavaScript » doit apparaître à côté de celui-ci.
5. Lorsqu'on sélectionne l'option « afficher le mot de passe », les deux champs précédents affichent le mot de passe en clair.

6. Le bouton « envoyer » est désactivé lorsque l'un des champs n'est pas rempli.
7. Le nom des champs vides apparaît en rouge.
8. Le bouton « envoyer » est désactivé lorsque les deux champs « mot de passe » ne coïncident pas.
9. Le champ « mot de passe (confirmation) » apparaît en rouge tant que son contenu n'est pas égal à celui du champ « mot de passe ».